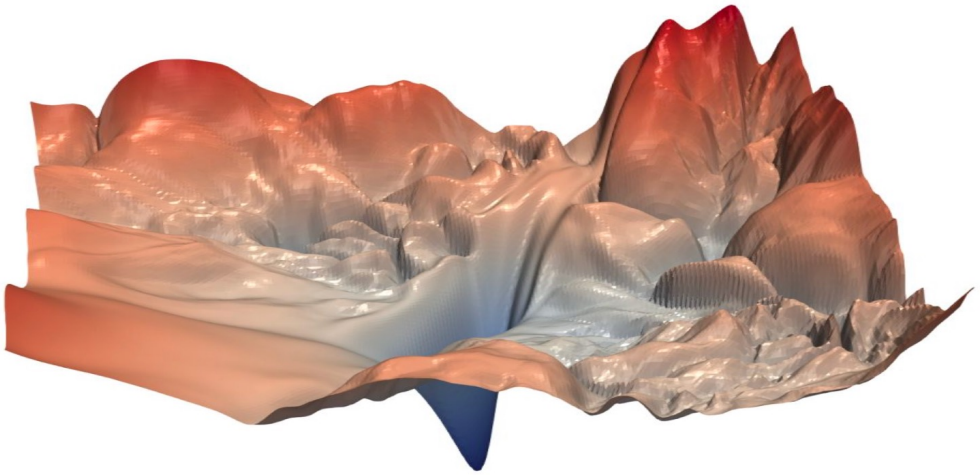
Machine Learning



Lecture 4: Training deep networks & convolutional networks

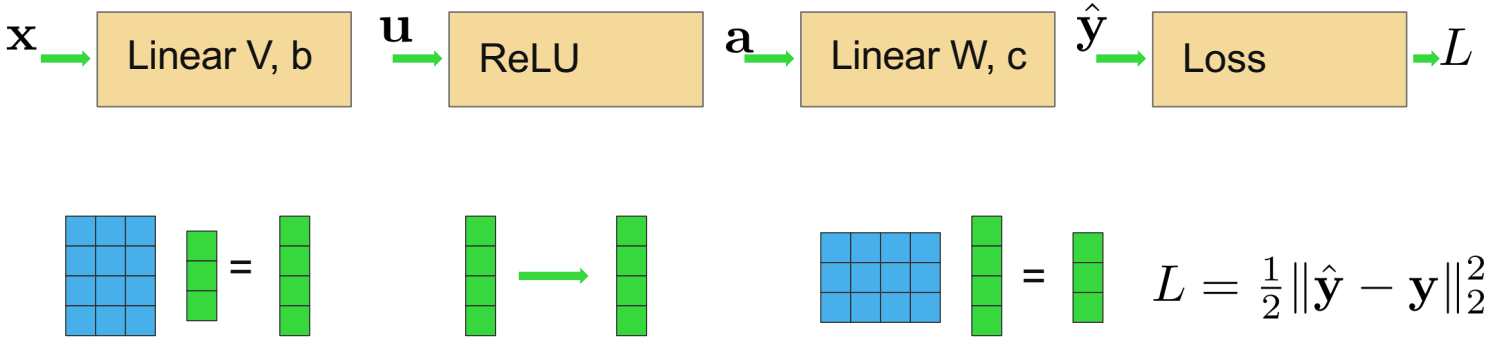
Iasonas Kokkinos

Imperial College London

i.kokkinos@ic.ac.uk

Forward pass through a ReLU MLP

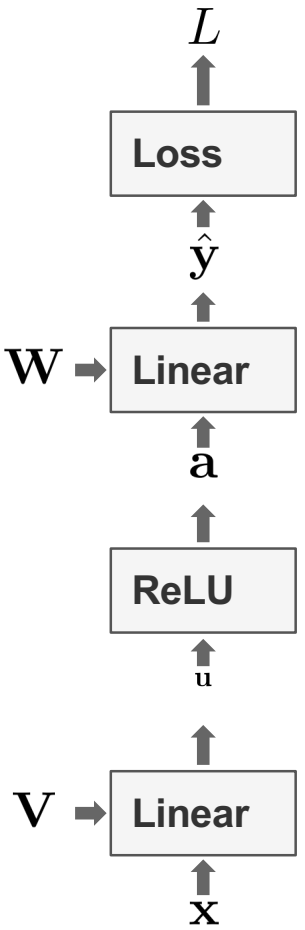
The same cached values will be reused in the backward pass.



$$L = \frac{1}{2}\|\hat{\mathbf{y}} - \mathbf{y}\|_2^2$$

One scalar

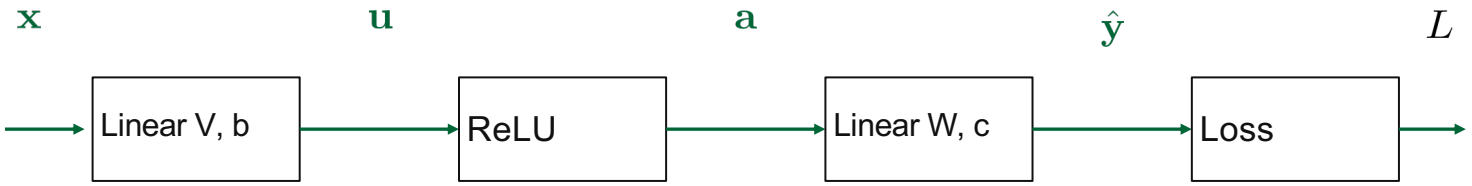
$$\mathbf{u} = \mathbf{V}\mathbf{x} + \mathbf{b} \quad \mathbf{a} = \mathrm{ReLU}(\mathbf{u}) \quad \hat{\mathbf{y}} = \mathbf{W}\mathbf{a} + \mathbf{c}$$



Cache x, u and a; the prediction is ŷ = Wa + c.

The completed backward pass

Cache x, u and a. Compute all gradients at the same fixed parameter values.



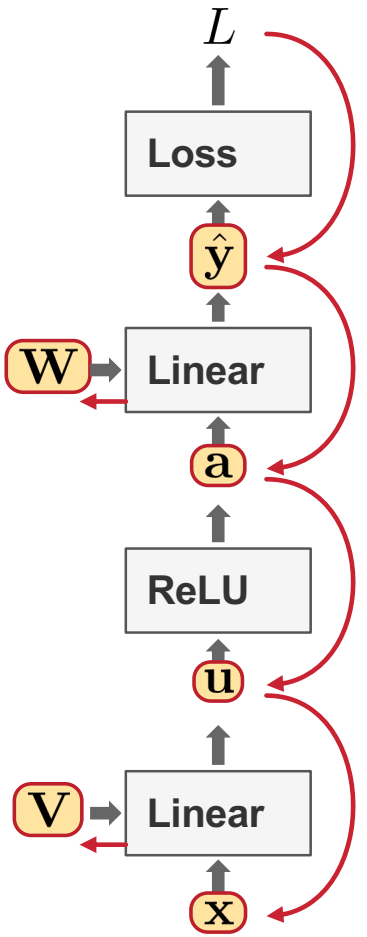
Output $\mathbf{g}_{\hat{y}} = \hat{\mathbf{y}} - \mathbf{y}$

Output linear $\nabla_{\mathbf{W}} L = \mathbf{g}_{\hat{\mathbf{y}}} \mathbf{a}^T \qquad \nabla_{\mathbf{c}} L = \mathbf{g}_{\hat{\mathbf{y}}}$

$\mathbf{g}_{\mathbf{a}} = \mathbf{W}^T \mathbf{g}_{\hat{\mathbf{y}}}$

ReLU $\mathbf{g}_u = \mathbf{g}_a \odot \mathbb{I}(\mathbf{u} > 0)$

First linear $\nabla_{\mathbf{V}} L = \mathbf{g}_{\mathbf{u}} \mathbf{x}^T \qquad \nabla_{\mathbf{b}} L = \mathbf{g}_{\mathbf{u}}$



Backpropagation computes gradients. The optimiser then changes the parameters.

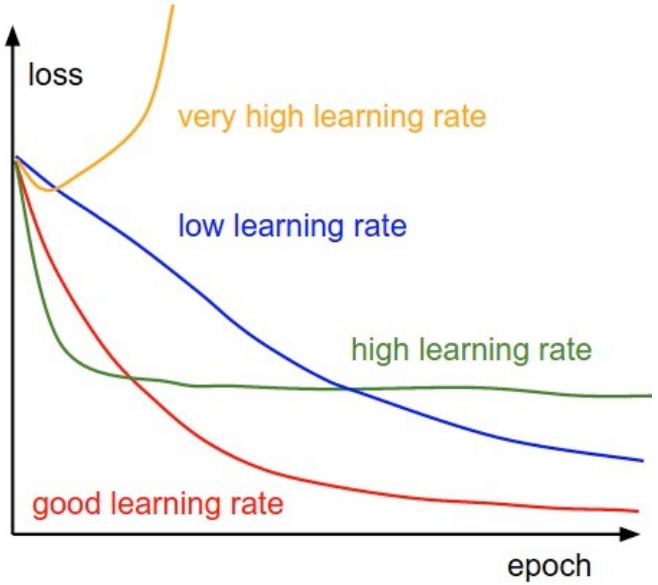
Gradient descent and SGD

Optimisation

- **Gradient descent and SGD**
- Momentum and Adam
- Depth issues
- Scale issues

Generalisation

Convolutional networks

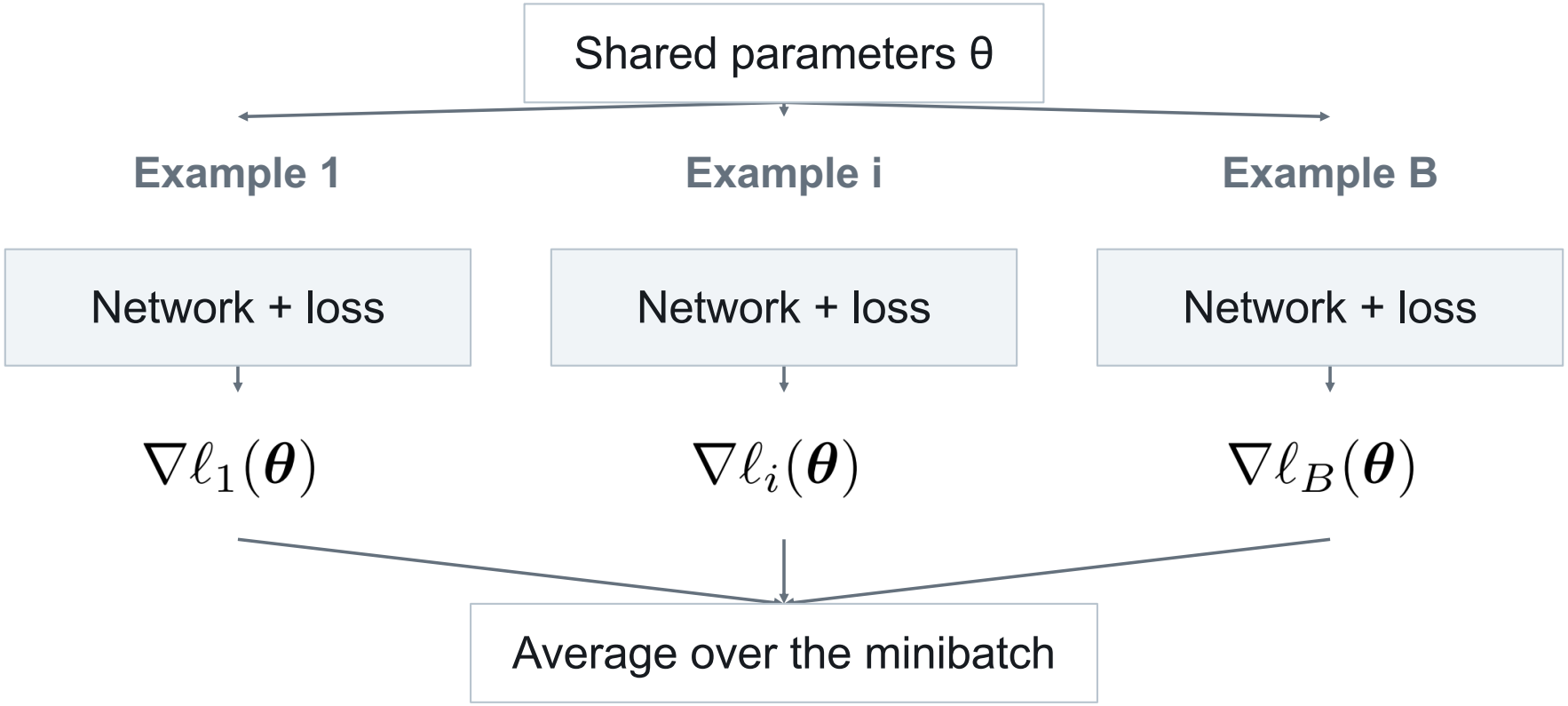


Reading: Gradient descent and SGD

K. Murphy (2022), §§8.2, 8.4: gradient descent and SGD

The optimiser averages per-example gradients

Backpropagation differentiates each example's loss. The optimiser combines these gradients to update the shared parameters.



$$\widehat{\mathbf{g}}_t = \frac{1}{B_t} \sum_{i \in \mathcal{B}_t} \nabla \ell_i(\boldsymbol{\theta}_t)$$

Gradient descent steps against the gradient

$$L(\boldsymbol{\theta}) = \frac{1}{N}\sum_{i=1}^{N} \ell_i(\boldsymbol{\theta}), \qquad J(\boldsymbol{\theta}) = L(\boldsymbol{\theta}) + \lambda\Omega(\boldsymbol{\theta})$$

Gradient descent:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta_t \frac{\partial J}{\partial \boldsymbol{\theta}}(\boldsymbol{\theta}_t)$$

θ contains all trainable weights and biases; ℓ_i is the loss for example i.
L is the data loss; Ω penalises weights; J = L + λΩ.

Column gradient of the objective:

$$\nabla J(\boldsymbol{\theta}_t) = \frac{1}{N}\sum_{i=1}^{N} \nabla \ell_i(\boldsymbol{\theta}_t) + \lambda \nabla\Omega(\boldsymbol{\theta}_t)$$

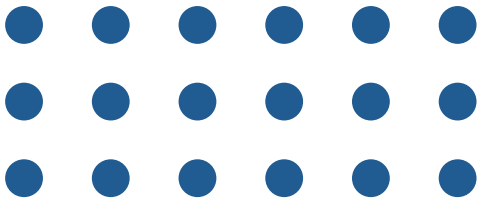
One-example backpropagation

A full-gradient update averages contributions from all N training examples.

Minibatches give cheap, unbiased gradients

Full gradient

All N training examples

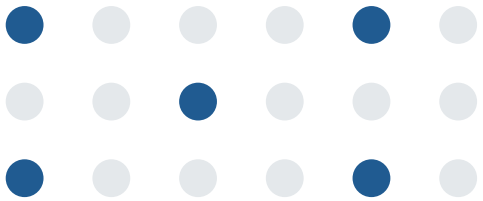


Average N loss gradients

$$\nabla J(\boldsymbol{\theta}_t) = \frac{1}{N}\sum_{i=1}^{N} \nabla \ell_i(\boldsymbol{\theta}_t) + \lambda \nabla \Omega(\boldsymbol{\theta}_t)$$

Minibatch gradient

B_t sampled examples



Average B_t loss gradients

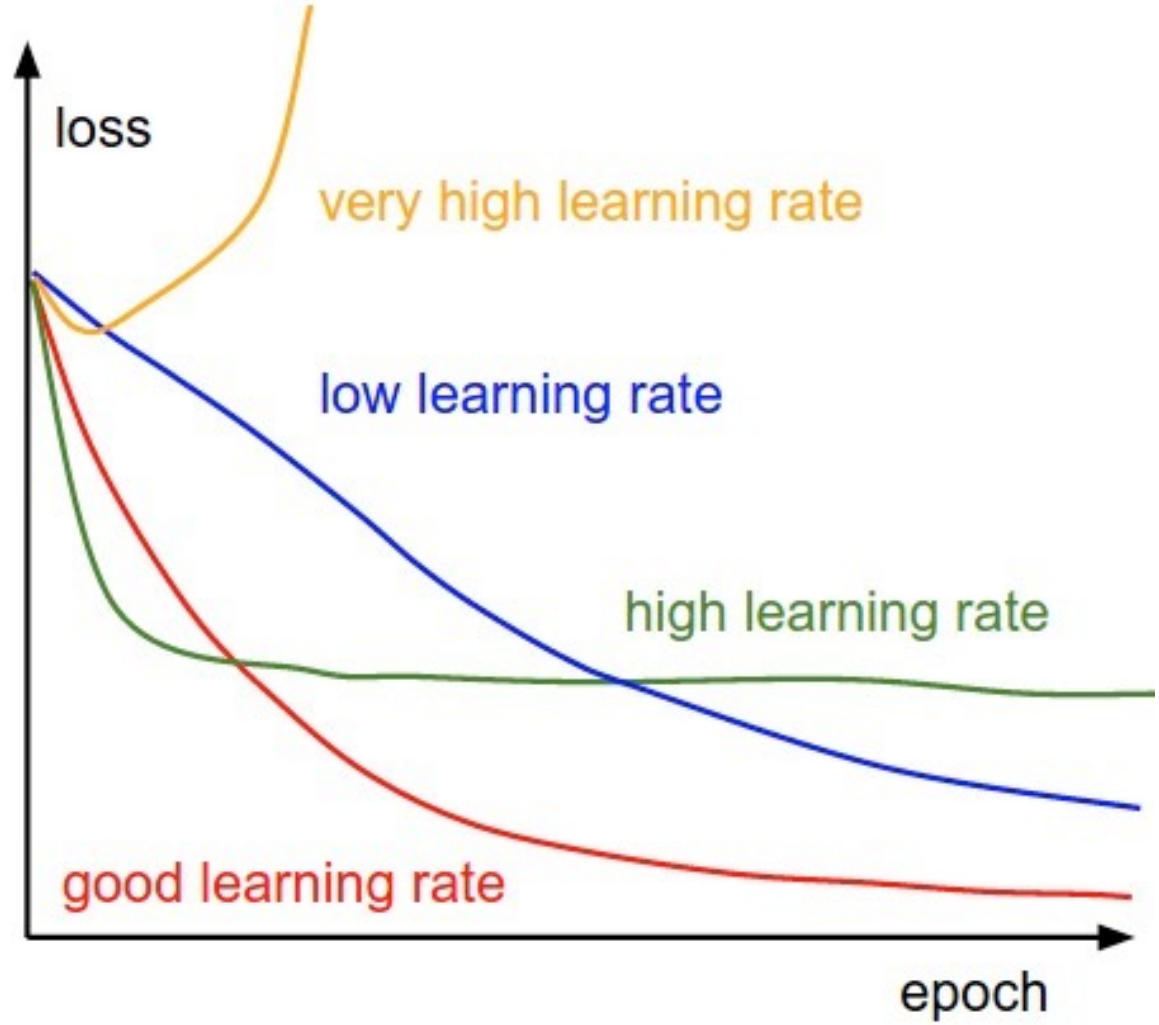
$$\mathbf{g}_t = \frac{1}{B_t}\sum_{i \in \mathcal{B}_t} \nabla \ell_i(\boldsymbol{\theta}_t) + \lambda \nabla \Omega(\boldsymbol{\theta}_t)$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta_t \mathbf{g}_t$$

Uniform sampling with replacement gives an unbiased estimate. B = 1 gives single-example SGD.

One shuffled pass is an epoch. Without replacement, N/B updates cover the full training set.

The learning rate makes or breaks training



$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta_t \frac{\partial J}{\partial \boldsymbol{\theta}}(\boldsymbol{\theta}_t)$$

Figure: Stanford CS231n, Neural Networks Part 3 (Karpathy).

Decaying the learning rate reduces noise

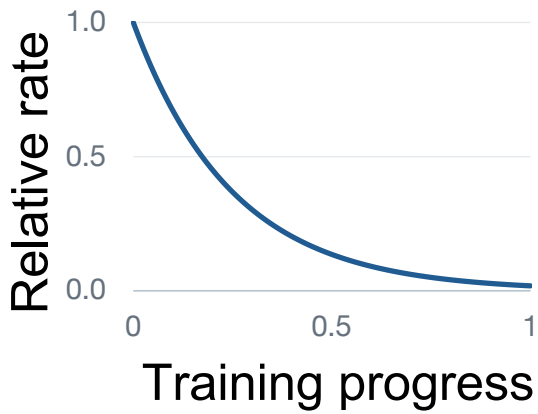
Use larger updates early and smaller updates later to reduce stochastic fluctuations.

Step decay



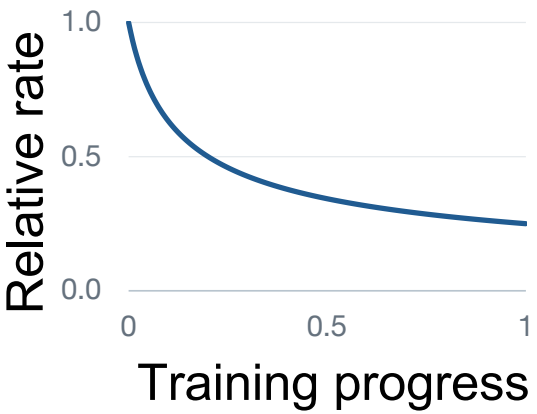
$$\eta_t = \eta_0 \gamma^{\lfloor t/K \rfloor}, \quad 0 < \gamma < 1$$

Exponential decay



$$\eta_t = \eta_0 e^{-\kappa t}$$

Polynomial decay



$$\eta_t = \eta_0 (1 + \kappa t)^{-\alpha}$$

Step decay uses fixed milestones. Reduce-on-plateau lowers the rate when a monitored loss stops improving.

Murphy (2022), Probabilistic Machine Learning: An Introduction, §8.4.3. Illustrative schedule parameters.

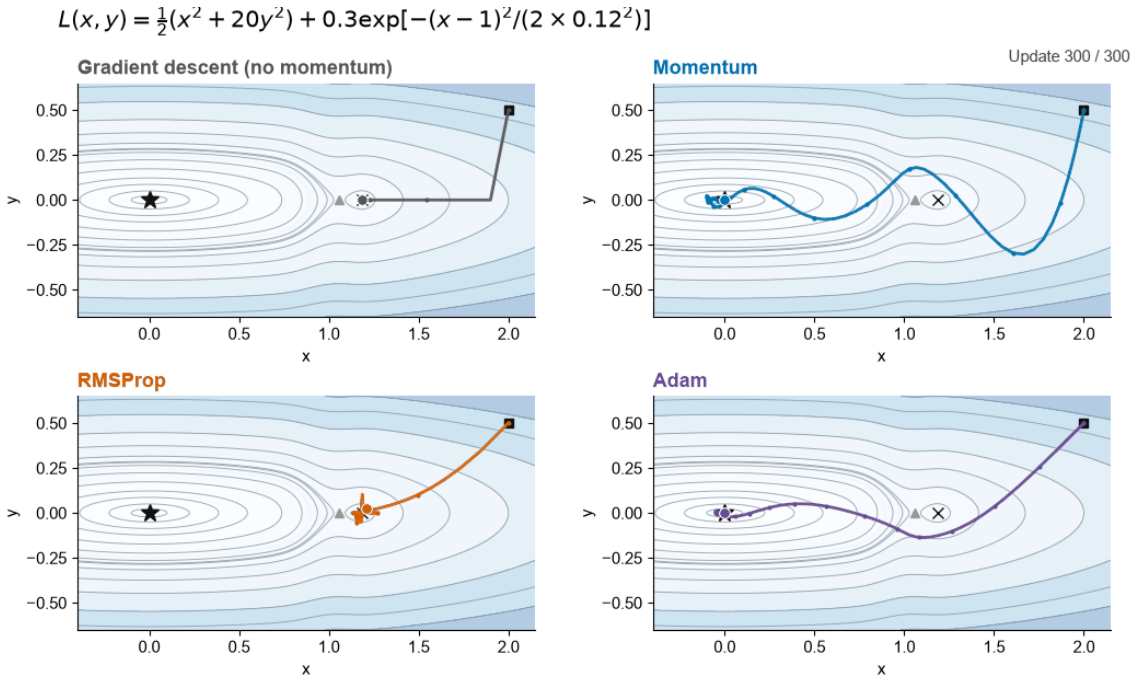
GD Issues: Momentum and Adam

Optimisation

- Gradient descent and SGD
- **Momentum and Adam**
- Depth issues
- Scale issues

Generalisation

Convolutional networks



Reading: Momentum and Adam

K. Murphy (2022), §8.2.4: Momentum methods; §8.4.6: Preconditioned SGD
Kingma and Ba (2015), Adam: A Method for Stochastic Optimization

Feature learning makes optimisation harder

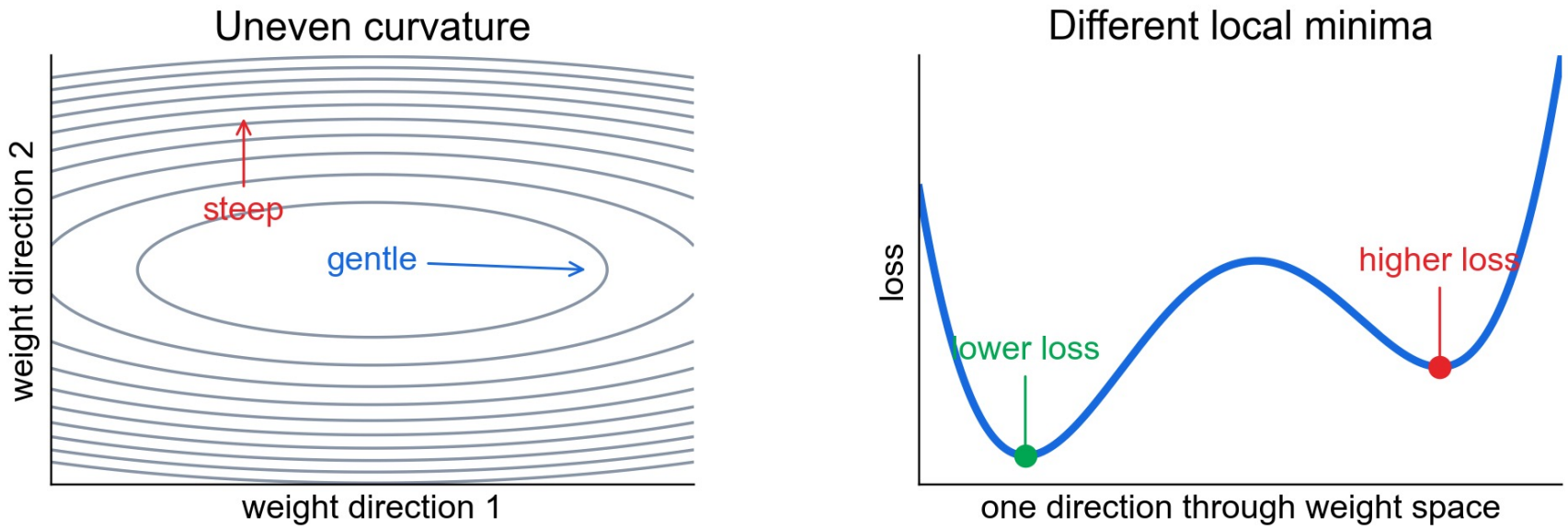
MLP: changing one layer changes the inputs and gradients seen by other layers. Joint training is generally nonconvex.

Initialisation, input scaling, activations and learning rates now matter together.

Next: two small examples of what makes this optimisation hard.

Two obstacles: curvature and minima

Two difficulties can slow gradient-based learning. These sketches show the ideas.

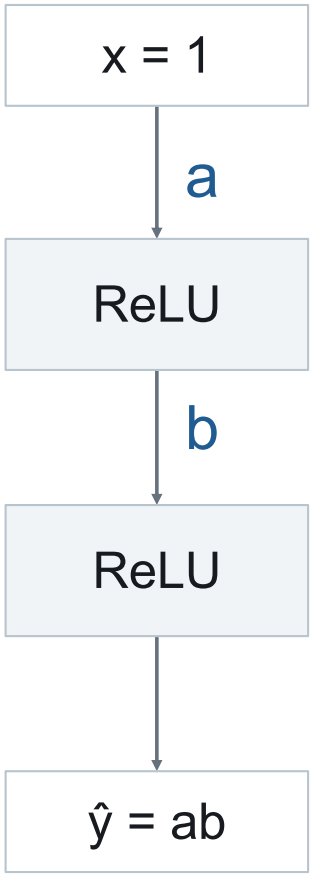


Uneven curvature: steep across a valley, gentle along it.
Different local minima: the starting point can determine the final loss.

Illustrative loss landscapes. Contours on the left use equal loss increments.

A two-weight network example

One training example: input x = 1, target y = 1. Biases are fixed at zero.



Both ReLUs are in their linear regime

Assume a > 0 and b > 0.
Each ReLU passes its positive input through.

$$\hat{y} = \mathrm{ReLU}(b\ \mathrm{ReLU}(ax))$$

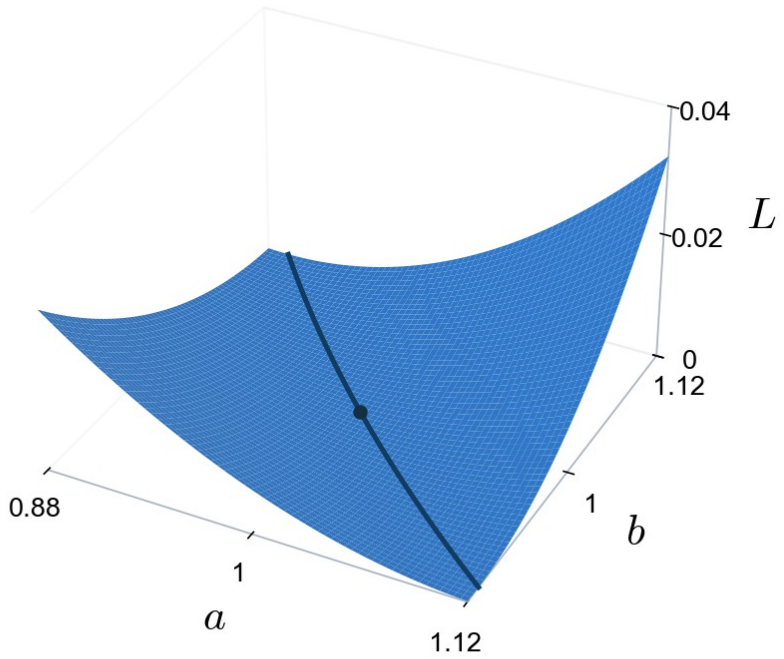
$$L(a,b) = \frac{1}{2}(y - abx)^2 = \frac{1}{2}(1 - ab)^2$$

The prediction is linear in x, but multiplicative in the trainable weights a and b.

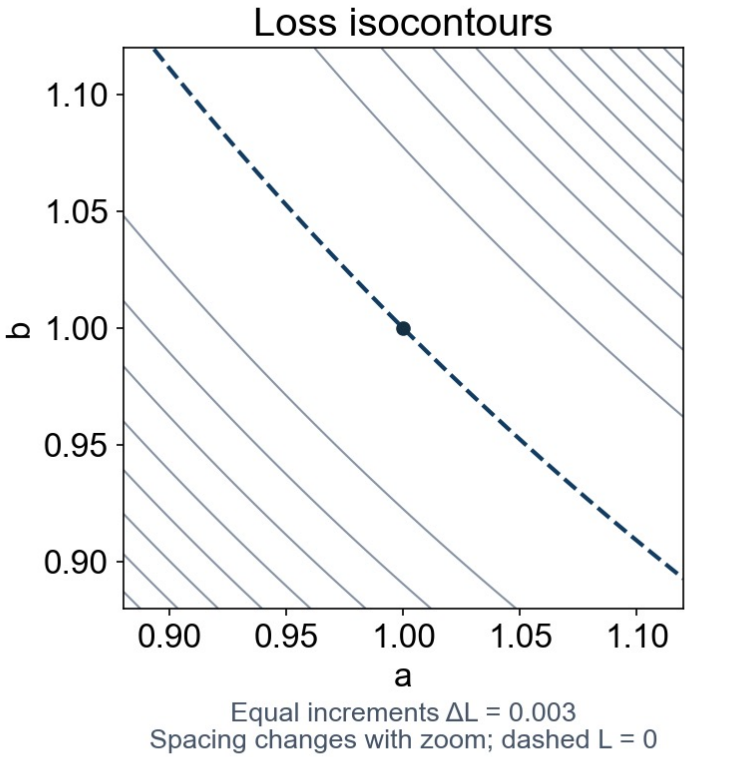
Zooming out over the loss landscape

From near (1,1) to 0 < a, b ≤ 4, within the active ReLU regime.

$$L(a,b) = \tfrac{1}{2}(1-ab)^2$$



Largest visible loss
0.0324



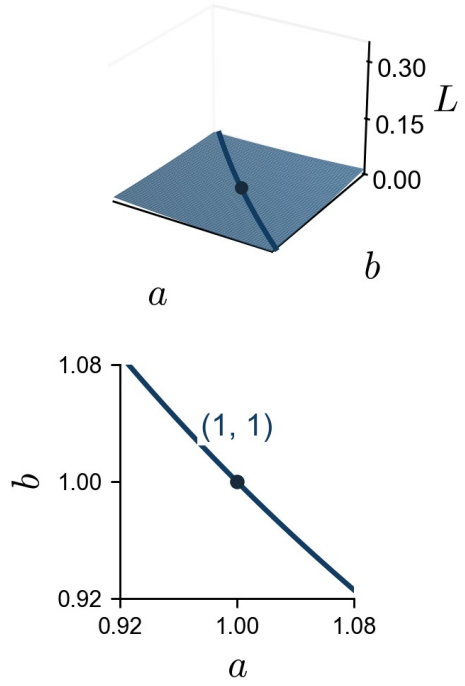
$$a, b \in [0.88, 1.12]$$

a > 0, b > 0. Axis ranges expand. Blue shading shows surface orientation, not loss.

Same fit, different local geometry

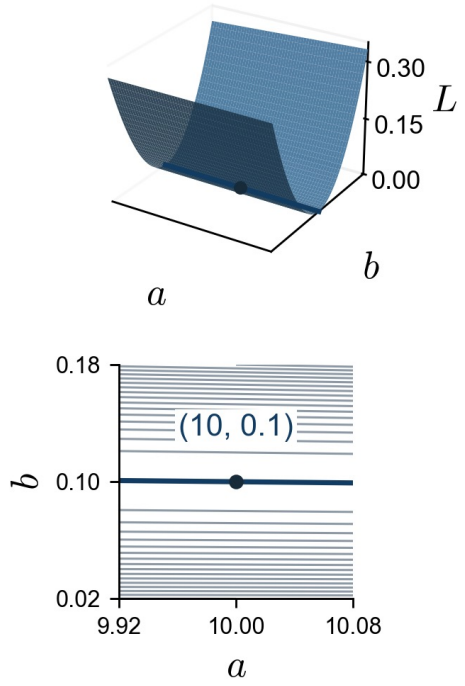
Same prediction ab = 1. Equal ±0.08 weight windows and equal loss scales.

Curvature at (1, 1)



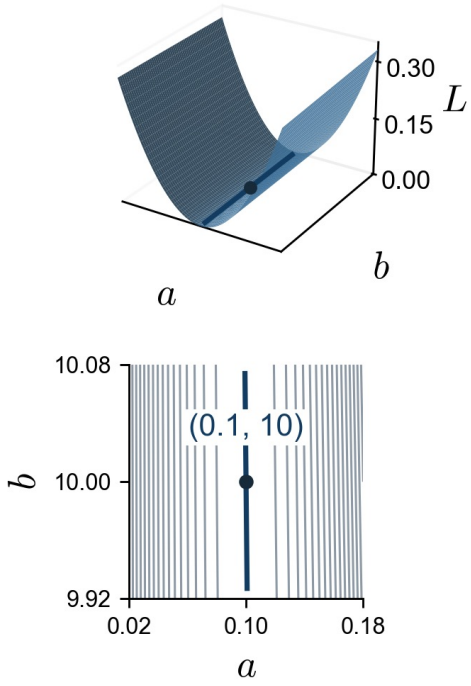
Shallow diagonal valley

Curvature at (10, 0.1)



Steep across b

Curvature at (0.1, 10)



Steep across a

Contours: L = 0.02, 0.04, … . Dark curve: L = 0. Denser lines mean steeper loss.

Lecture 2: Learning rate & convergence

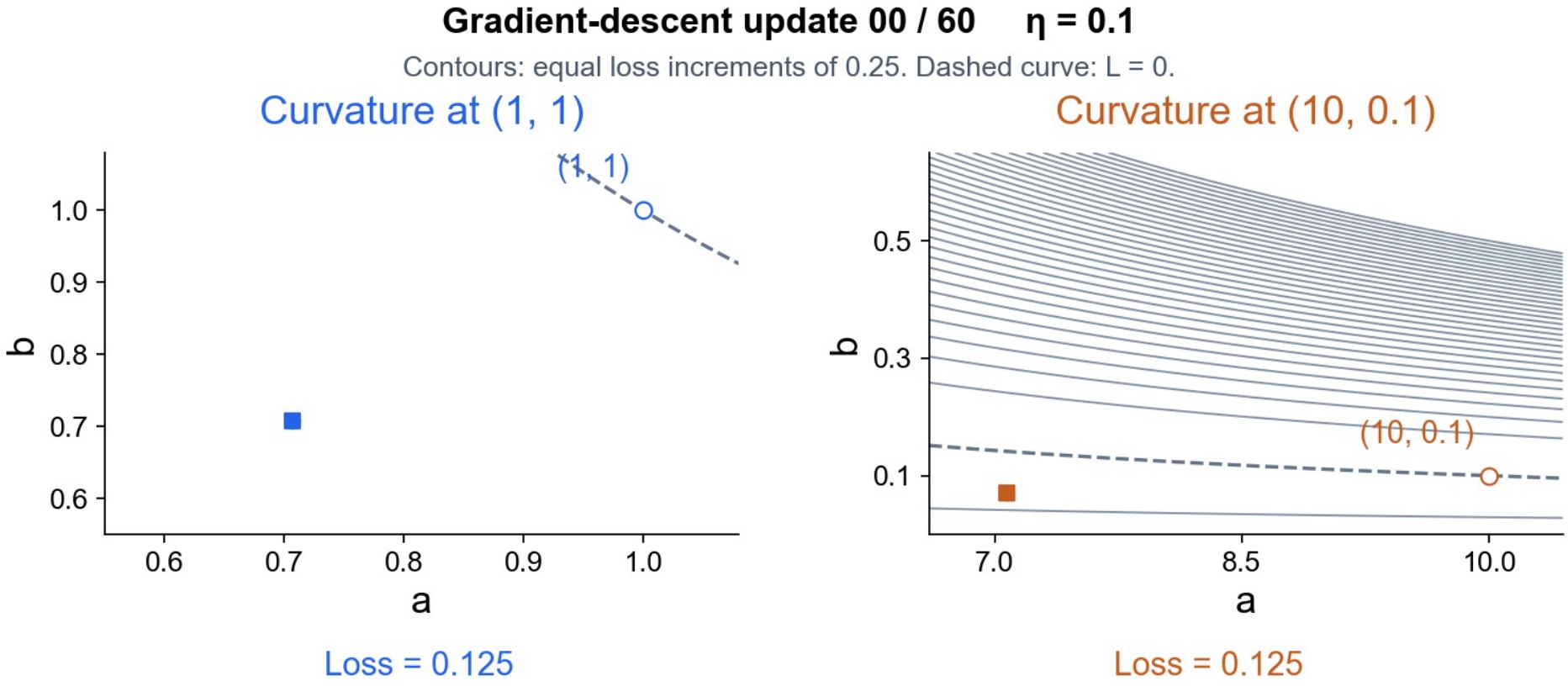
$$L(w) = \tfrac{1}{2}w^2, \qquad L'(w) = w, \qquad w_{t+1} = (1-\eta)w_t$$

One scalar parameter w; the minimum is at w = 0.

0 < η < 1: approaches zero without changing sign.
η = 1: reaches the minimum in one step.
1 < η < 2: alternates signs but still approaches zero.
η = 2: oscillates; η > 2: diverges (unless already at zero).

One learning rate, two different outcomes

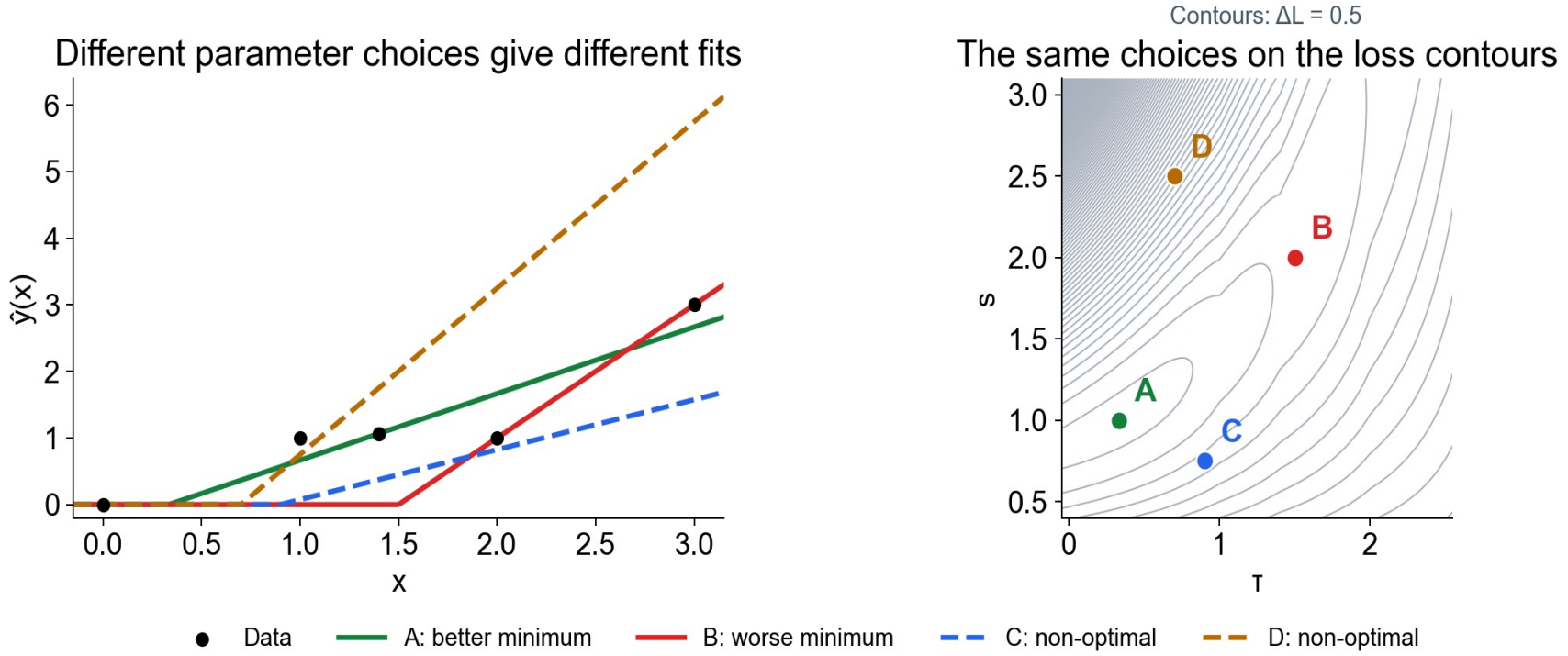
Starts (0.71, 0.71) and (7.07, 0.071): 10× and 0.1× the first, so both predict ab = 0.5 with the same loss. Same η = 0.1.



Click to play. One learning rate cannot suit both: the steep start needs smaller steps.

Obstacle 2: a ReLU fit with two minima

Fit $\hat{y}(x) = s\,\mathrm{ReLU}(x - \tau)$ to five points with $L = \tfrac{1}{2}\sum(\hat{y} - y)^2$. Only s and τ are trained.

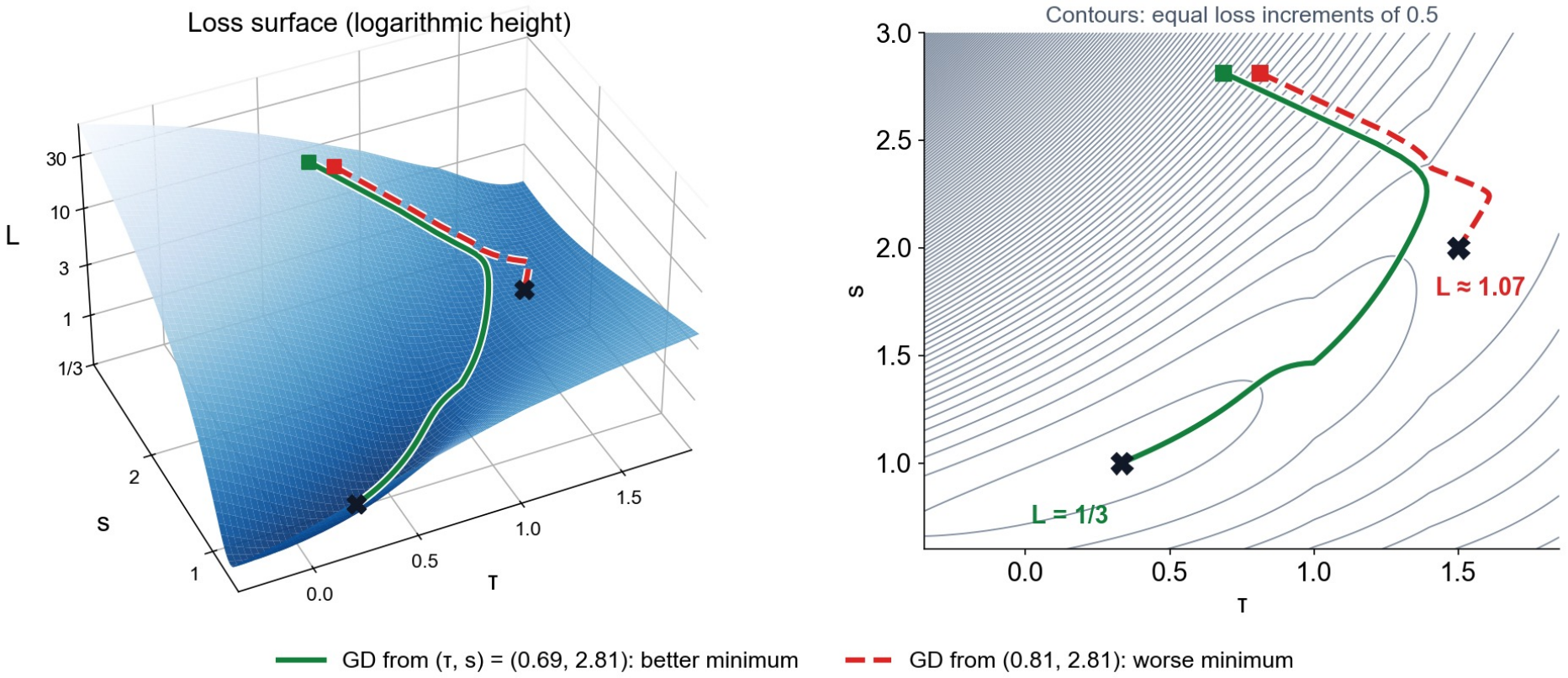


Solid fits A and B are local minima. Dashed fits C and D are non-optimal: their coloured points locate them on the same loss surface.

Both minima have positive-definite Hessians: they are strict local minima.

GD can reach either local minimum

The loss L(s, τ) of the five-point fit. Two small-step GD runs start side by side (squares).



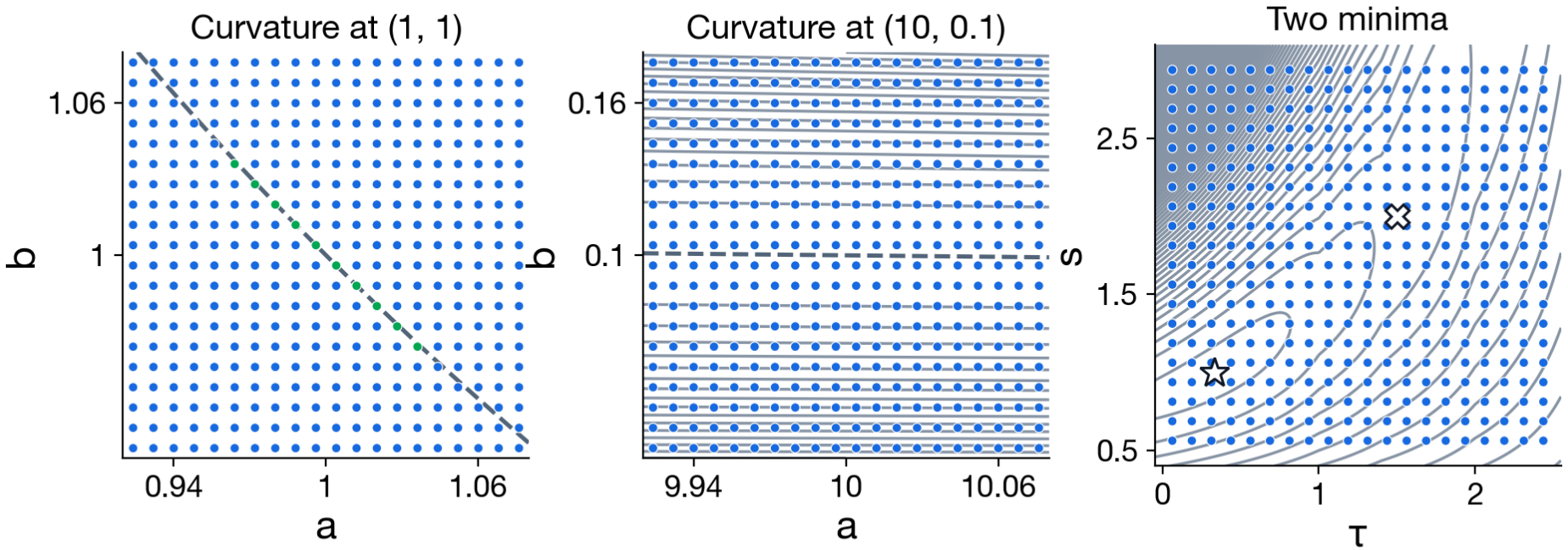
Nearby starts can lead to different minima.

Saved runs from the five-point grid used on the following slides: 400 GD updates each.

GD: one step size cannot fit both

η = 0.1: fine near (1, 1), but balls shoot out of the steep valley and switch the ReLU off. Right: many end in the worse minimum.

GD: update 0 of 100 (curvature), 0 of 400 (two minima)

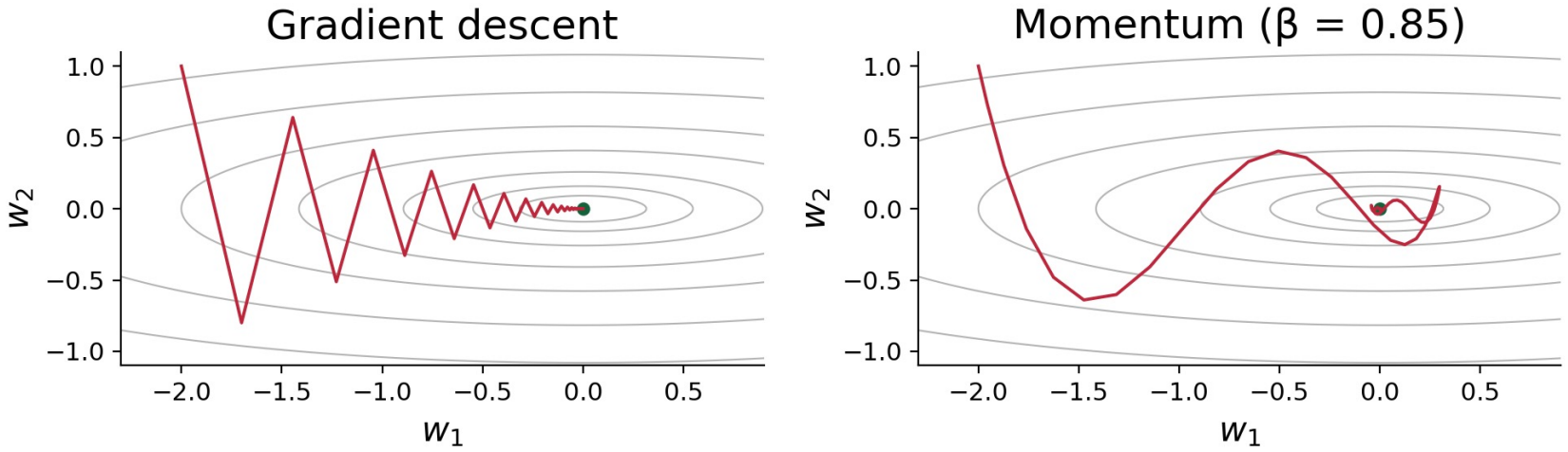


Contours: ΔL = 0.02 (curvature); ΔL = 0.5 (two minima). Dashed: L = 0.

Green: at the best minimum now. Triangles: outside the view.

Click to play. At the end, balls return to their starts, coloured by outcome.

Momentum damps oscillation across a valley



Same quadratic, start and rate. Green marks the minimum.

(S)GD $$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta_t \mathbf{g}_t$$

Momentum $$\mathbf{m}_t = \beta \mathbf{m}_{t-1} + (1-\beta)\mathbf{g}_t$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta_t \mathbf{m}_t$$

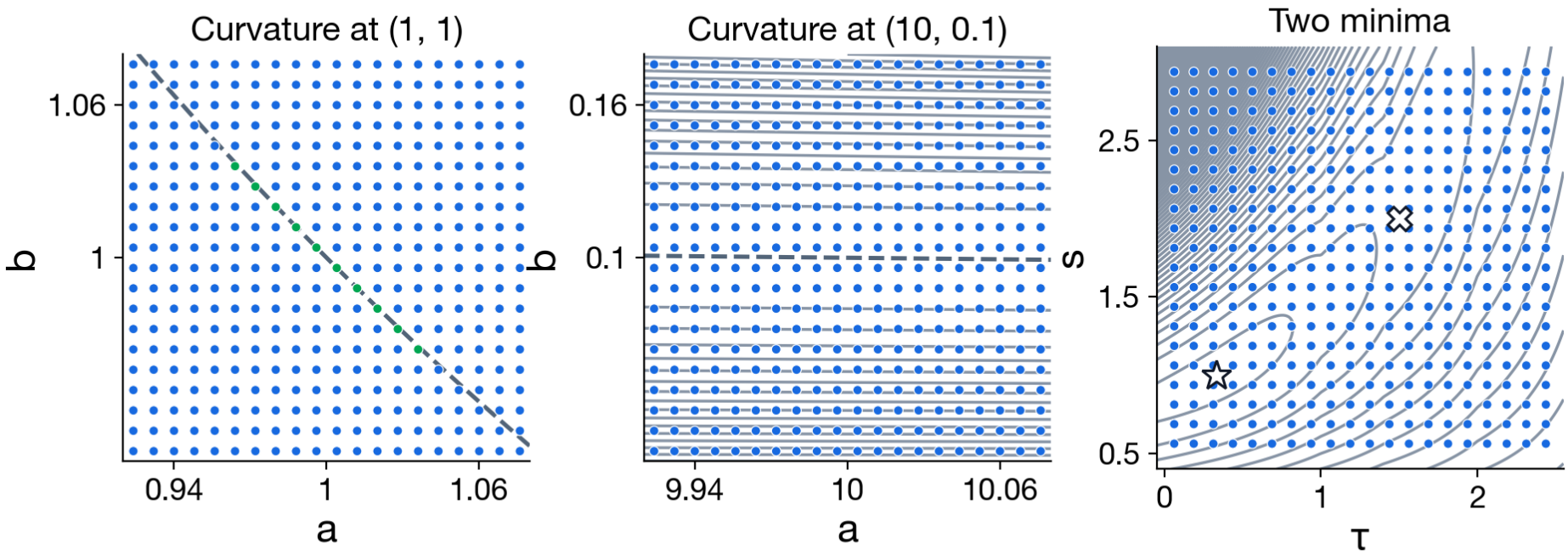
Across the valley, successive gradients point in opposite directions and cancel in m_t. Along it they agree and add up.

g_t is the gradient estimate; m_t, its moving average, is its first moment.
$m_0 = 0$ and $0 \le \beta < 1$. This convention includes the factor $(1-\beta)$.

Momentum tames curvature and humps

Same η = 0.1, with β = 0.8: every curvature start converges, and more balls cross the hump to the better minimum.

Momentum: update 0 of 100 (curvature), 0 of 400 (two minima)



Contours: ΔL = 0.02 (curvature); ΔL = 0.5 (two minima). Dashed: L = 0.

Green: at the best minimum now. Triangles: outside the view.

Click to play. At the end, balls return to their starts, coloured by outcome.

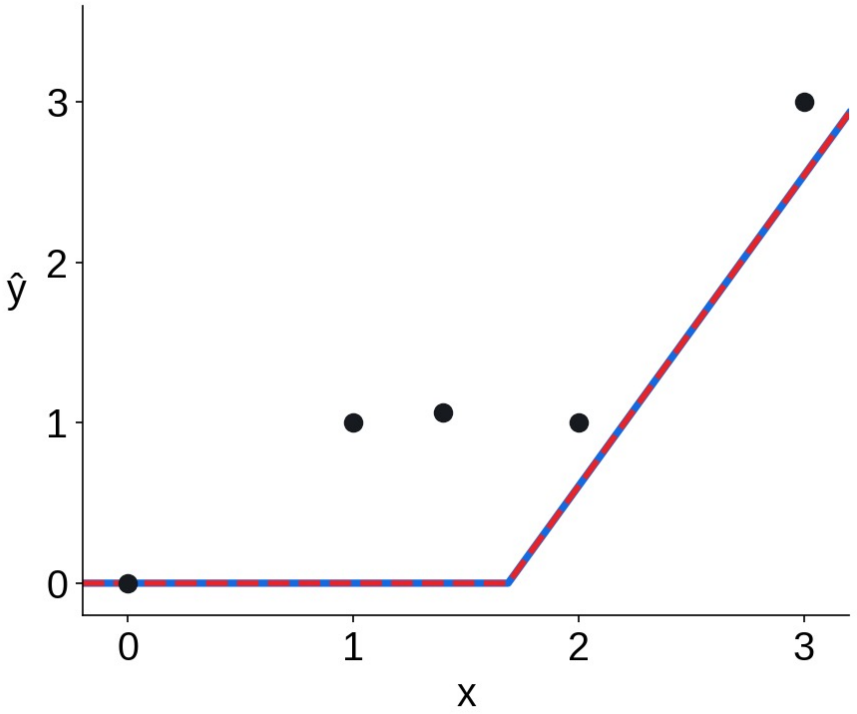
One start: the fit and its path

$\hat{y}(x) = s\,\mathrm{ReLU}(x - \tau)$ $\quad L = \tfrac{1}{2}\Sigma(\hat{y} - y)^2$

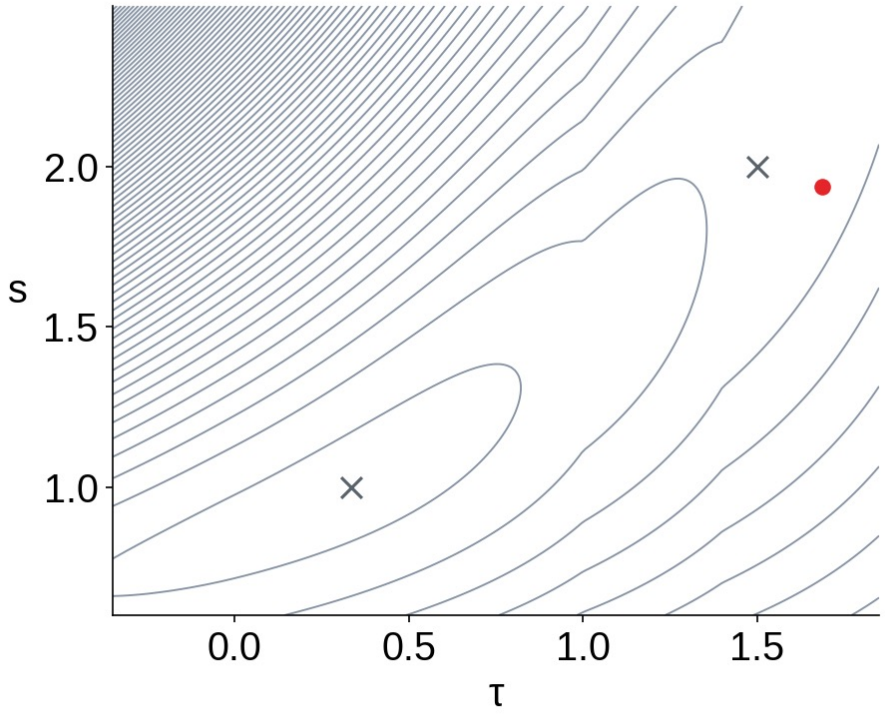
Gradient descent (solid) Momentum (dashed)

Update 0 / 400

Fit to the five points



Path over the loss contours



Contours: equal loss increments of 0.5. ×: the two minima.

GD: L = 1.25116

Momentum: L = 1.25116

RMSProp: scale each weight's step

Let $g_{t,j}$ be the minibatch gradient for weight j, including any penalty.

Average squared gradient (second moment)

$$v_{t,j} = \rho\, v_{t-1,j} + (1-\rho)\, g_{t,j}^2$$

Step

$$\theta_{t+1,j} = \theta_{t,j} - \eta_t \frac{g_{t,j}}{\sqrt{v_{t,j}} + \delta}$$

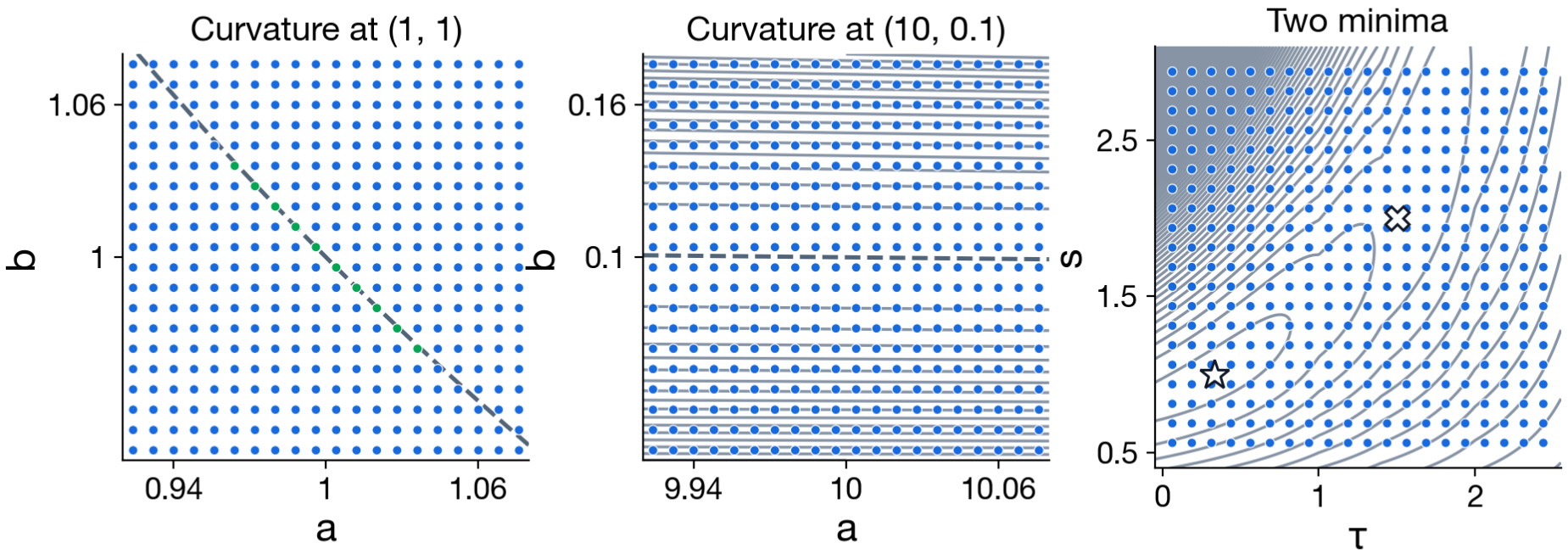
$v_{0,j} = 0$; $0 \le \rho < 1$ controls averaging.
$\delta > 0$ stabilises the denominator.

Large recent gradients reduce the effective step for that weight.

RMSProp: fast on curvature, not minima

Scaling each coordinate makes both curvature windows fast, but many balls end in the worse minimum.

RMSProp: update 0 of 100 (curvature), 0 of 400 (two minima)



Contours: ΔL = 0.02 (curvature); ΔL = 0.5 (two minima). Dashed: L = 0.

Green: at the best minimum now. Triangles: outside the view.

Click to play. At the end, balls return to their starts, coloured by outcome.

Adam: momentum and adaptive scaling

Adam combines averages of gradients and their squares for each weight j. Start both averages at zero.

Direction (first moment)

$$m_{t,j} = \beta_1 m_{t-1,j} + (1 - \beta_1)\, g_{t,j}$$

Scale (second moment)

$$v_{t,j} = \beta_2 v_{t-1,j} + (1 - \beta_2)\, g_{t,j}^2$$

Undo the zero start

$$\widehat{m}_{t,j} = \frac{m_{t,j}}{1 - \beta_1^t}, \qquad \widehat{v}_{t,j} = \frac{v_{t,j}}{1 - \beta_2^t}$$

Step

$$\theta_{t+1,j} = \theta_{t,j} - \eta_t \frac{\widehat{m}_{t,j}}{\sqrt{\widehat{v}_{t,j}} + \delta}$$

$0 \le \beta_1, \beta_2 < 1$ are decay rates; t counts updates.

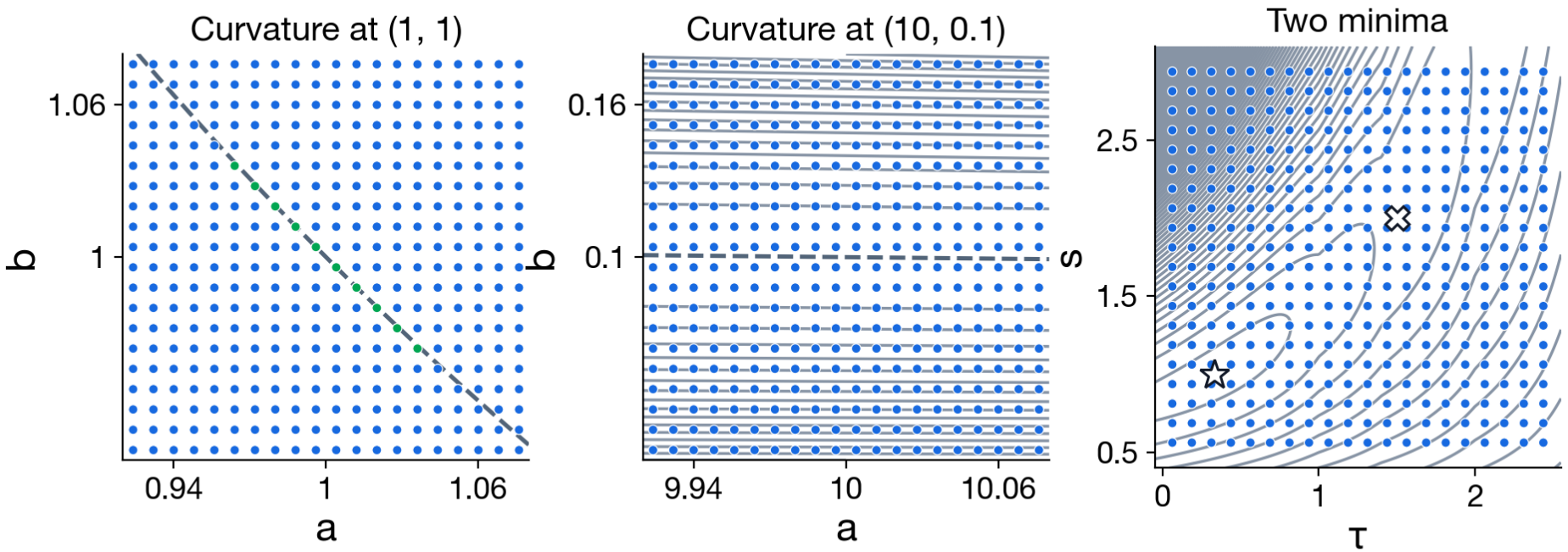
Dividing by $\sqrt{\hat{v}}$ scales the average gradient by its recent size. The step also depends on how consistently the gradients point in one direction.

Kingma & Ba, Adam (2015)

Adam handles all three here

Momentum plus per-coordinate scaling: every curvature start converges, and almost every ball reaches the better minimum.

Adam: update 0 of 100 (curvature), 0 of 400 (two minima)



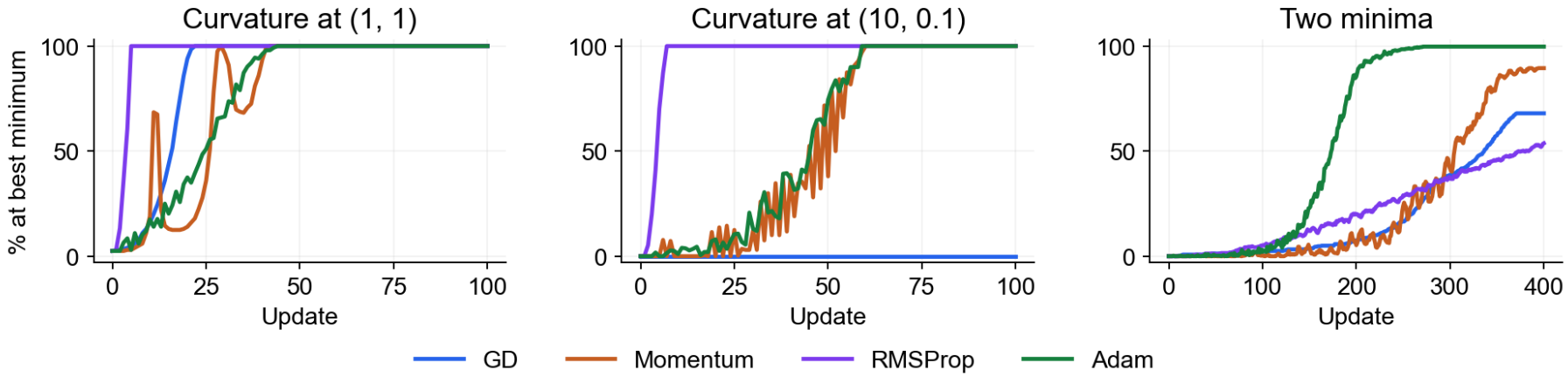
Contours: ΔL = 0.02 (curvature); ΔL = 0.5 (two minima). Dashed: L = 0.

Green: at the best minimum now. Triangles: outside the view.

Click to play. At the end, balls return to their starts, coloured by outcome.

Four optimisers, two obstacles

No optimiser is guaranteed to find the best minimum. Adam is a strong default here; tune the learning rate first.



Successful starts after the full update budget (out of 400):

Method / Task	Curvature at (1, 1)	Curvature at (10, 0.1)	Two minima
GD	400	0	272
Momentum	400	400	358
RMSProp	400	400	215
Adam	400	400	399

Curves: share at the best minimum after each update. Same 400 starts per task.

Choosing and checking an optimiser

Method	State carried between steps	Main change
SGD	None	Use this minibatch gradient
Momentum	Average gradient m (first moment)	Smooth the update direction
RMSProp	Average squared gradient v (second moment)	Scale each weight's step
Adam	Both m and v	Smooth and scale

Tune the learning rate and check validation curves.
A more elaborate optimiser does not guarantee better generalisation.

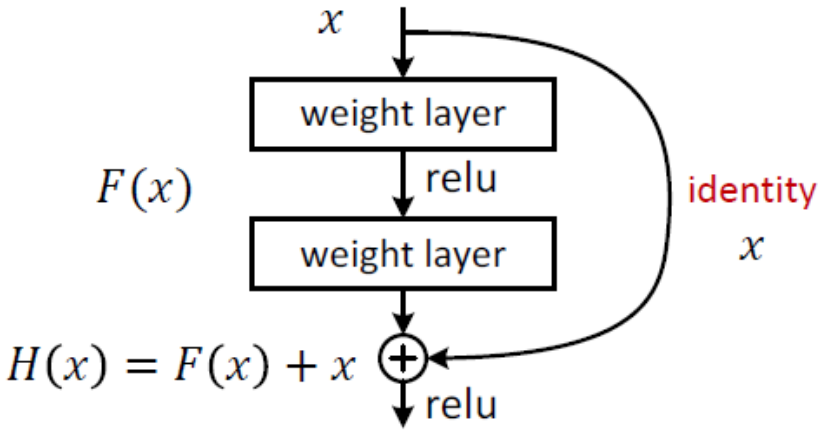
Depth Issues: ReLU and ResNets

Optimisation

- Gradient descent and SGD
- Momentum and Adam
- **Depth issues**
- Scale issues

Generalisation

Convolutional networks



Reading: Depth issues

K. Murphy (2022), §§13.4.2–13.4.3, 13.4.5: gradients and initialisation

Gradients vanish or explode with depth

Backpropagation multiplies one factor per layer. Over many layers the product shrinks or grows geometrically:

$$0.9^{30} \approx 0.04 \qquad\qquad 1.1^{30} \approx 17$$

Vanishing: early layers receive almost no gradient and barely learn.

Exploding: updates overshoot; the loss jumps or becomes NaN.

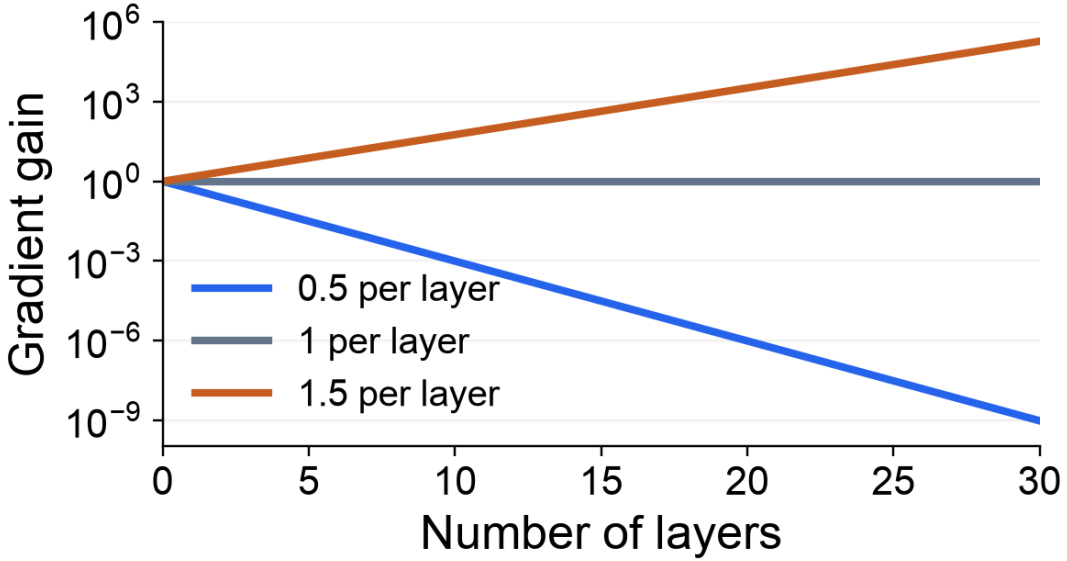
Remedies here: ReLU and residual connections (depth issues), and normalisation (scale issues).

Per-layer factors of 0.9 and 1.1 over 30 layers.

Depth multiplies gradient gains

Backpropagation multiplies one factor per layer: the weight times the slope of the activation. With one unit per layer:

$$\frac{\partial L}{\partial x} = w_1\, g'(u_1) \cdot w_2\, g'(u_2) \cdots w_D\, g'(u_D) \cdot \frac{\partial L}{\partial \hat{y}}$$



Small gains: early layers barely update.

Large gains: updates can become unstable.

Initialisation and normalisation help control scales. Residual paths give gradients an additional route through the network.

Sigmoids can cause vanishing gradients

Incoming gradient Sigmoid derivative: at most ¼

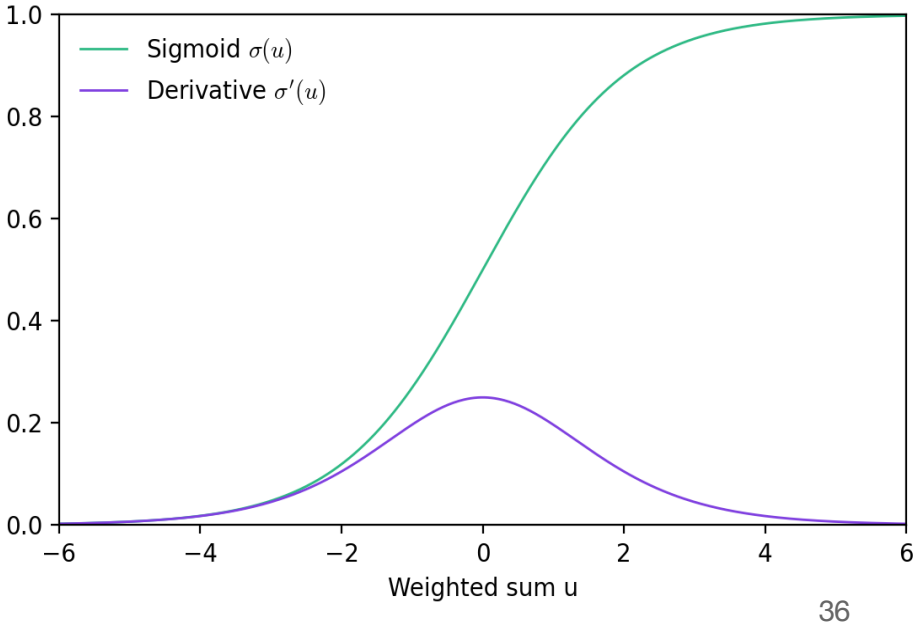
$$\frac{\partial L}{\partial u_k} = \frac{\partial L}{\partial a_k} \cdot \sigma'(u_k)$$

$$= \frac{\partial L}{\partial a_k} \cdot \sigma(u_k)\big(1 - \sigma(u_k)\big)$$

$$a_k = \sigma(u_k)$$

Repeated sigmoid derivatives can make gradients vanish in deep networks.

Weight factor: |w σ′(u)| ≤ |w| / 4.



Active ReLUs do not shrink the gradient

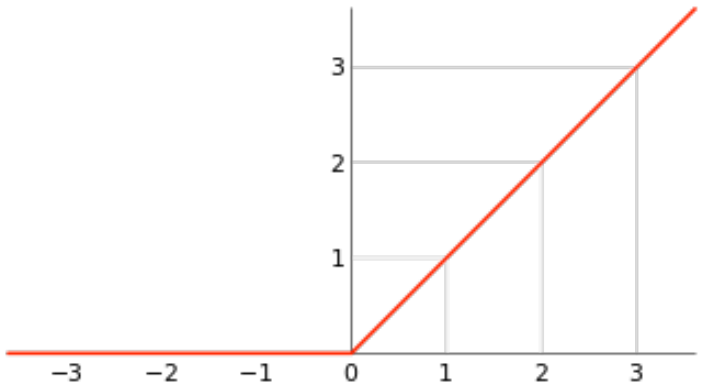
Incoming gradient ReLU derivative: 0 or 1

$$\frac{\partial L}{\partial u_k} = \boxed{\frac{\partial L}{\partial a_k}} \cdot \boxed{\mathrm{ReLU}'(u_k)}$$

Positive units avoid attenuation from the activation derivative.

Rectified linear unit

$$\mathrm{ReLU}(u) = \max(0, u)$$



$$\mathrm{ReLU}'(u) = \begin{cases} 1 & u > 0 \\ 0 & u \leq 0 \end{cases}$$

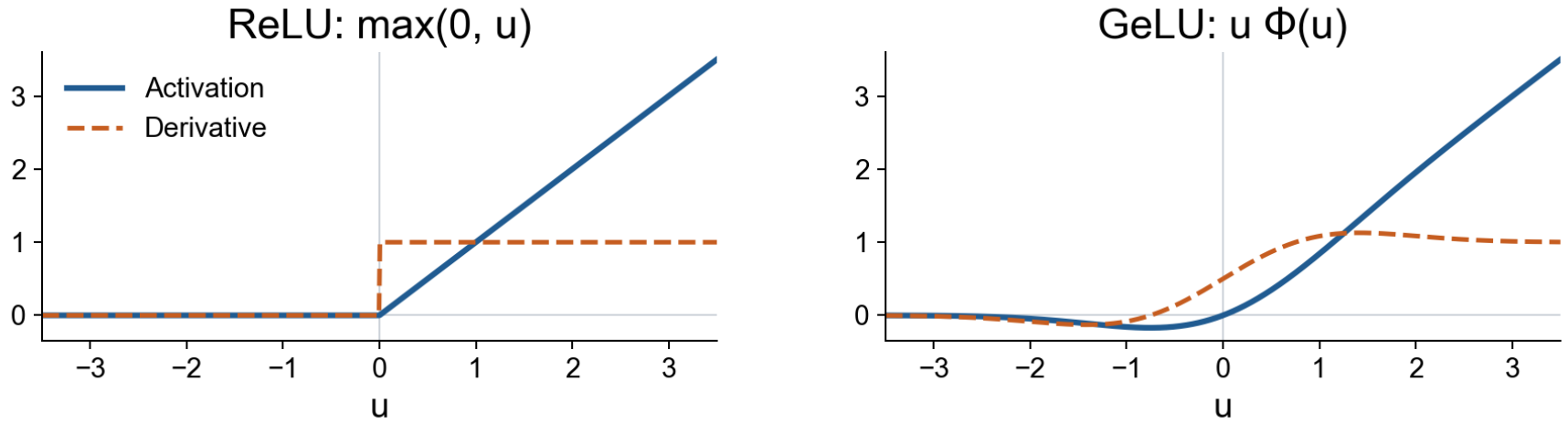
V. Nair and G. Hinton (2010). Rectified linear units improve restricted Boltzmann machines. ICML.

ReLU and GeLU: hard and smooth activation gates

Incoming gradient ReLU derivative: 0 or 1

$$\frac{\partial L}{\partial u_k} = \frac{\partial L}{\partial a_k} \cdot \mathrm{ReLU}'(u_k)$$

ReLU has derivative 1 for positive inputs; GeLU uses a smooth gate.

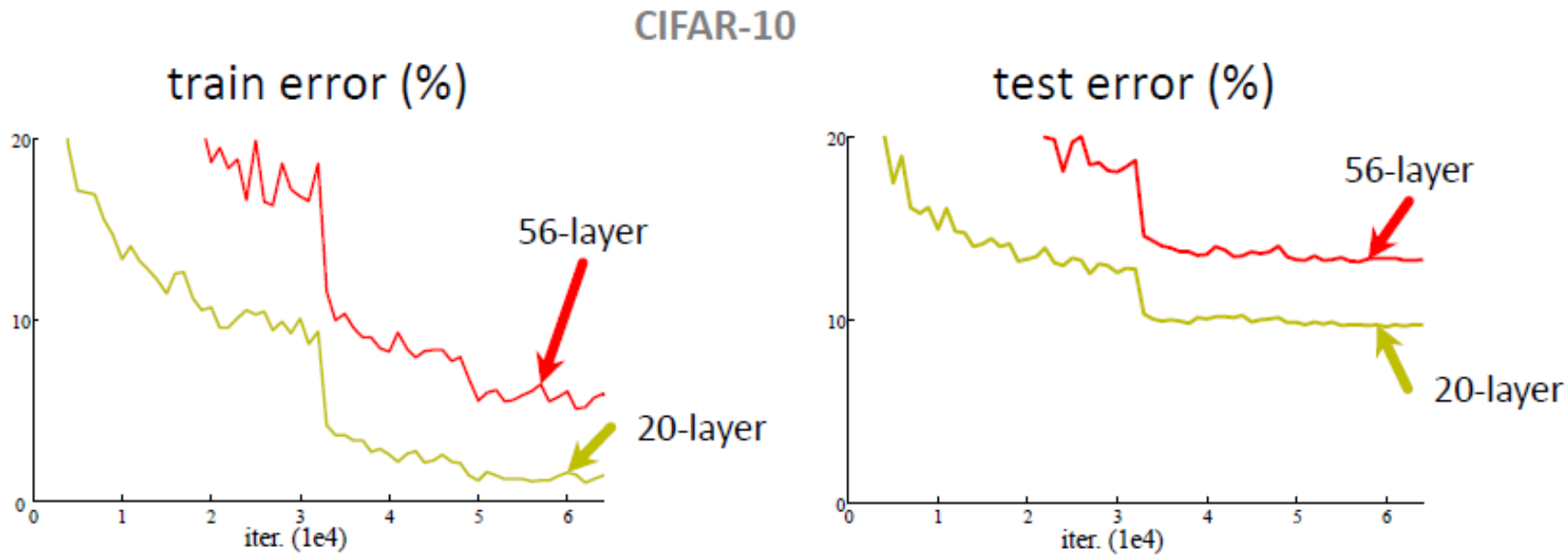


Φ(u) is the standard normal CDF. GeLU is common in Transformer MLPs (e.g. BERT): Lecture 5.

Some modern Transformers use SwiGLU. Sources: Nair & Hinton (2010); Hendrycks & Gimpel (2016); Devlin et al. (2019).

Deeper plain networks can train worse

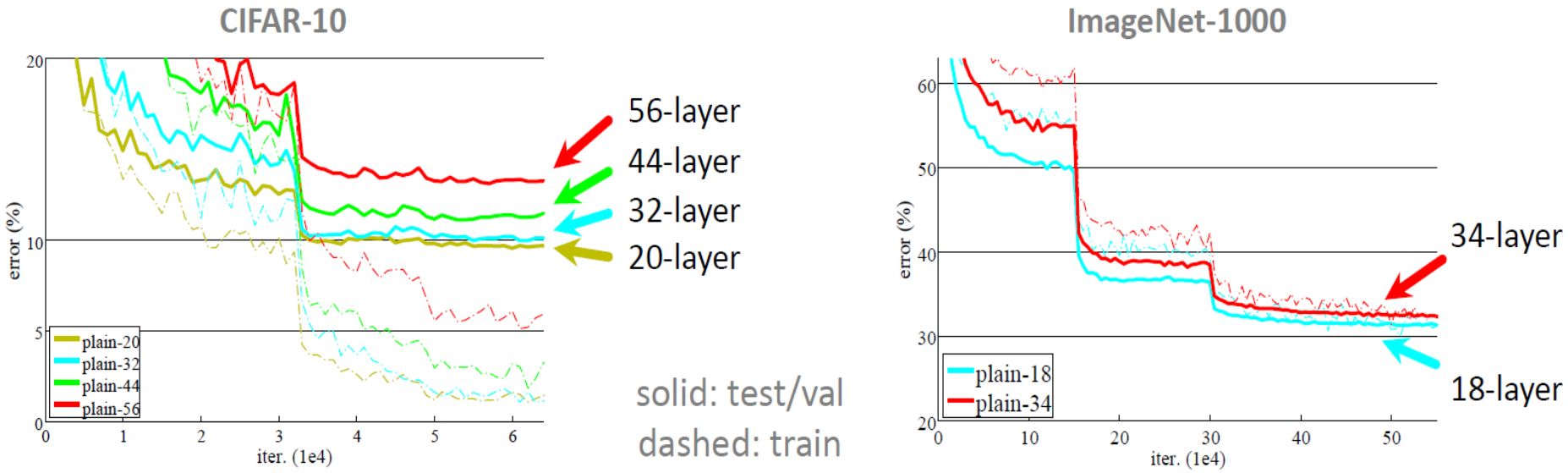
- Plain nets: stacking 3x3 conv layers
- 56-layer net has higher training error and test error than 20-layer net



He et al. (2016), Deep Residual Learning for Image Recognition, Fig. 1.

Degradation is an optimisation failure

- "Overly deep" plain nets have higher training error
- A general phenomenon, observed in many datasets
- Counterintuitive: deeper network should be stronger, right?
  - Idea: make sure deeper network can fall back to the shallow solution.



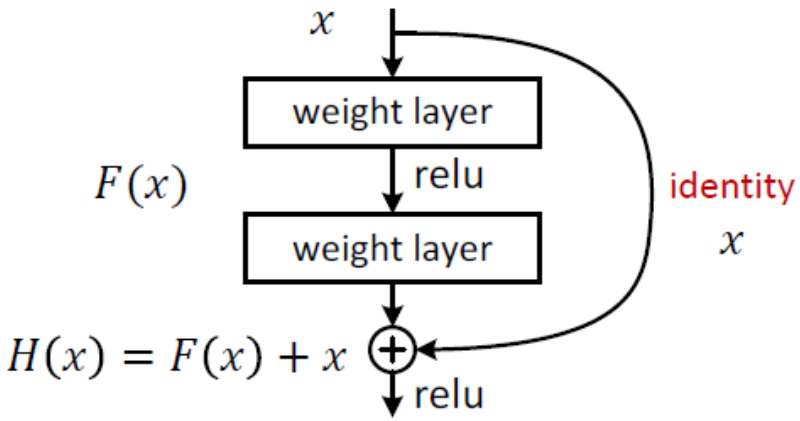
He et al. (2016), Deep Residual Learning for Image Recognition, Figs. 4 and 6.

A residual block learns a correction

A skip connection carries an earlier activation around intermediate layers.

A residual block adds a learned correction to the skipped signal.

A zero residual recovers identity before the final activation.

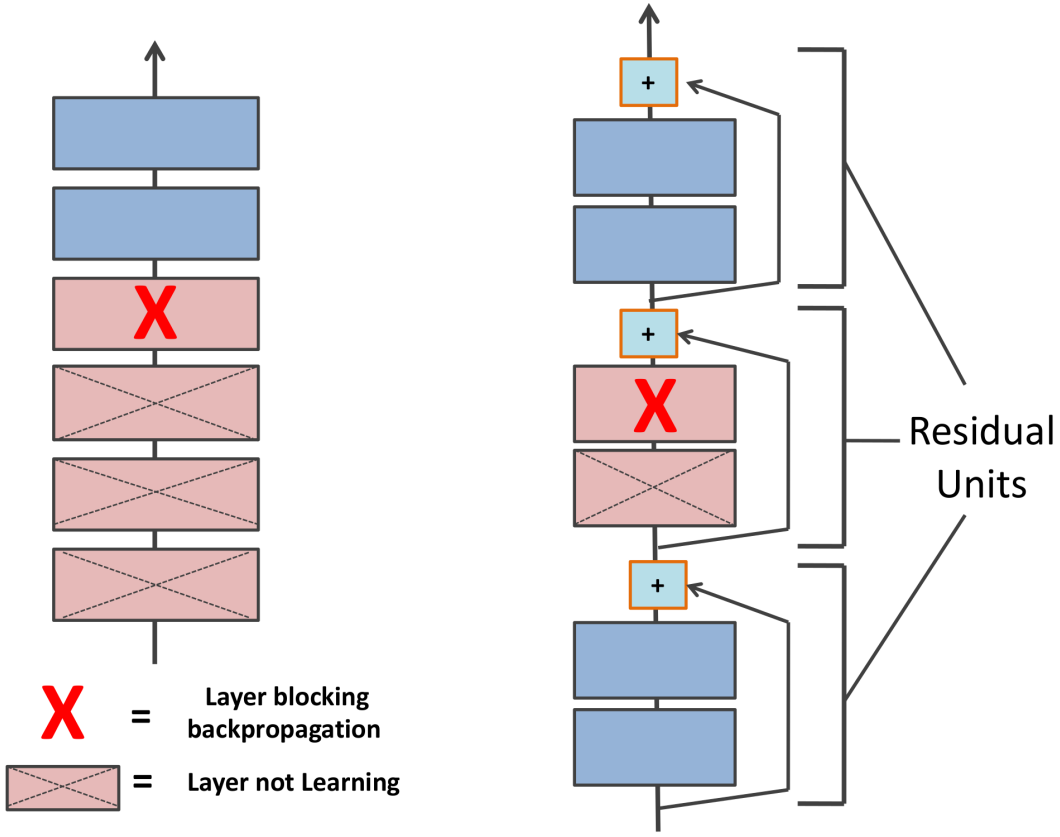


An additive skip connection

He et al. (2016), Deep Residual Learning for Image Recognition, Fig. 2

Skip paths route around stuck layers

In a plain stack, one layer that blocks the gradient stops every layer below it from learning.



With residual units, the skip path carries the gradient past the stuck layer, so earlier layers keep learning.

K. Murphy (2022), PML, Fig. 13.15(b), adapted from A. Géron (2019).

The skip path passes gradients unchanged

For an identity skip before any post-addition activation:

$$\mathbf{s}_{k+1} = \mathbf{s}_k + \mathbf{F}(\mathbf{s}_k)$$

$$\mathbf{g}_{\mathbf{s}_k} = \mathbf{g}_{\mathbf{s}_{k+1}} + \left(\frac{\partial \mathbf{F}(\mathbf{s}_k)}{\partial \mathbf{s}_k} \right)^T \mathbf{g}_{\mathbf{s}_{k+1}}$$

Here g is the column gradient of L with respect to the indicated state.

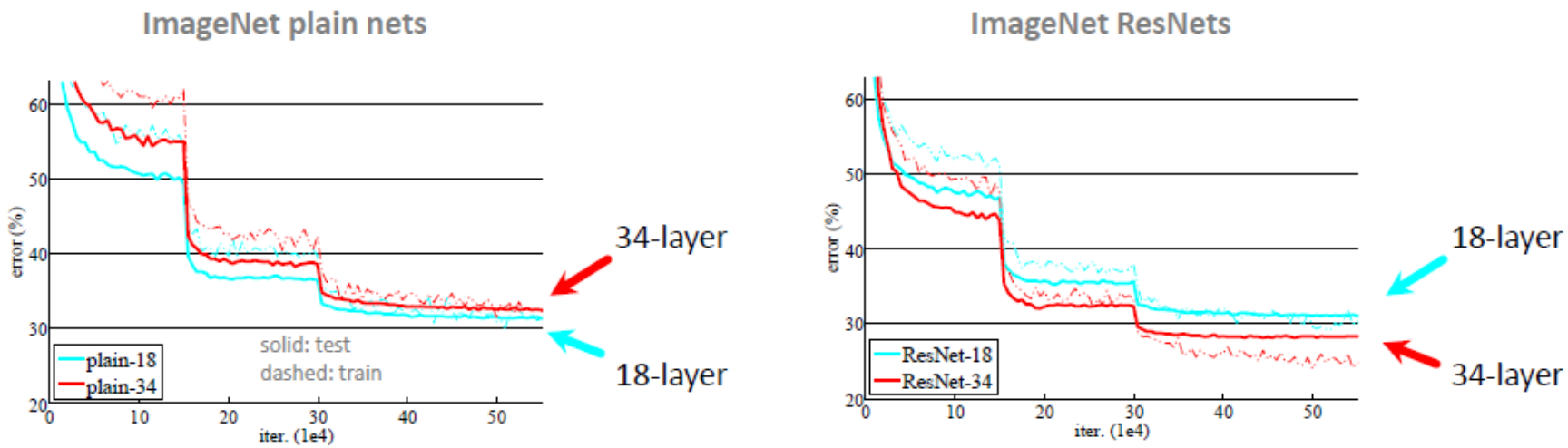
The identity branch contributes the incoming gradient directly.
The residual branch contributes an additional term.

A post-addition activation also multiplies this gradient by its derivative.

He et al. (2016), Identity Mappings in Deep Residual Networks

Residuals let deeper networks train better

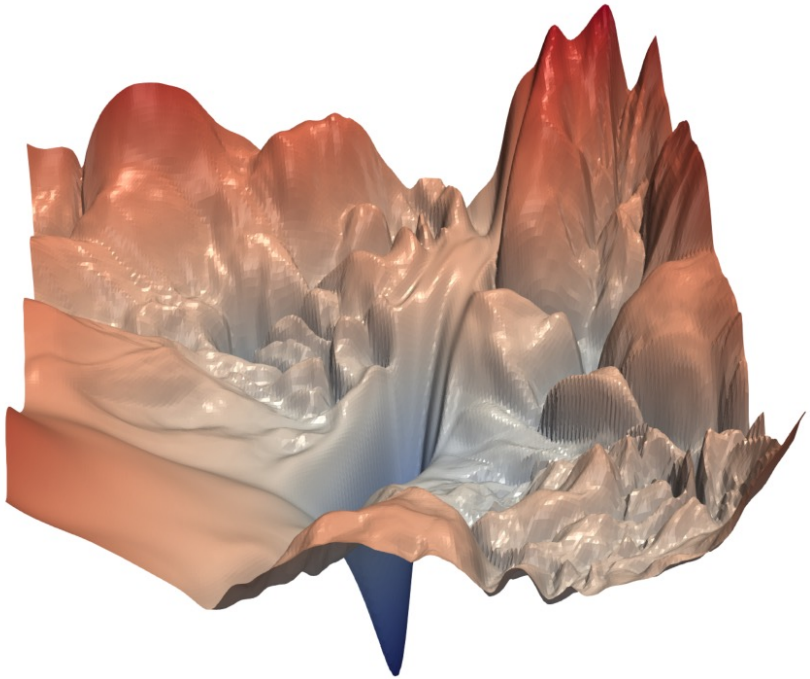
Residual connections make very deep networks easier to optimise.
Deeper ResNets reduce training and validation error (labelled “test” in the plot).



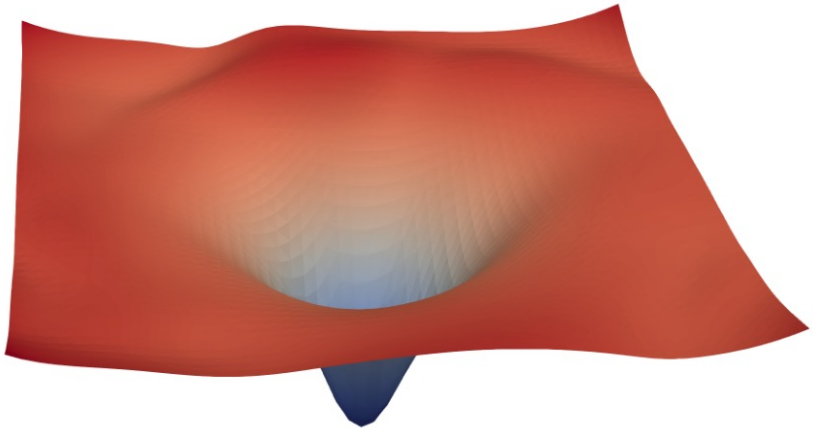
He et al. (2016), Deep Residual Learning for Image Recognition, Fig. 4.

Skip connections smooth the loss landscape

ResNet-56 on CIFAR-10



Without skip connections



With skip connections

Two-dimensional slices through parameter space, using filter normalisation.
The residual network has a smoother landscape in this comparison.

Li et al. (2018), Visualizing the Loss Landscape of Neural Nets, Fig. 1

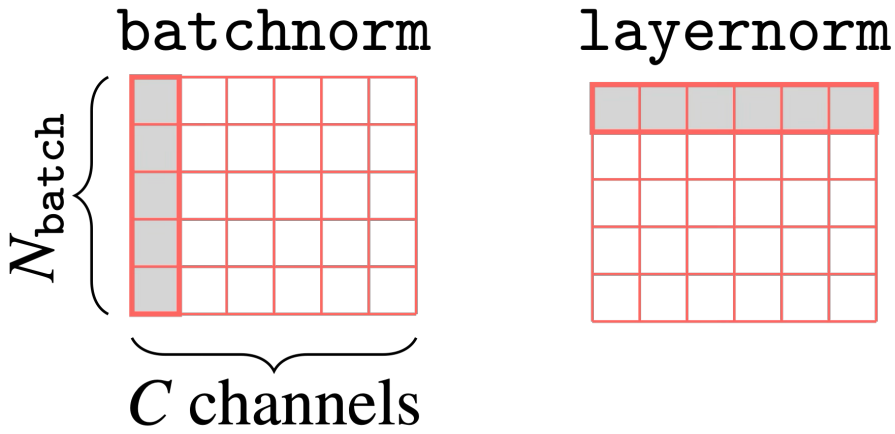
Scale Issues: Normalization and Initialization

Optimisation

- Gradient descent and SGD
- Momentum and Adam
- Depth issues
- **Scale issues**

Generalisation

Convolutional networks



Reading: Scale issues

K. Murphy (2022), §§13.4.2–13.4.3, 13.4.5: gradients and initialisation

Keep each layer’s gain near 1

A deep network multiplies layer gains, like a × b in the two-weight example.

Aim for stable signal and gradient scales through depth.

Initialisation sets the starting scale; normalisation helps control it during training.

$$\text{gain}_1 \times \text{gain}_2 \times \dots \times \text{gain}_k$$

Overall gain is a product: small imbalances compound with depth.

Standardise inputs so no feature dominates

Standardisation

Use that feature's training-set mean μ and standard deviation σ.

$$x' = \frac{x - \mu}{\sigma}$$

Feature scales affect weight updates: $\frac{\partial L}{\partial \mathbf{W}} = \frac{\partial L}{\partial \mathbf{u}} \mathbf{x}^T \quad (\mathbf{u} = \mathbf{W}\mathbf{x})$

Calculate scaling values on the training set only.
Apply the same transformation to validation and test data.

Normalise activations, then rescale them

A normalisation layer computes statistics over a chosen set $\mathcal{G}$ of activations, then learns an affine transformation.

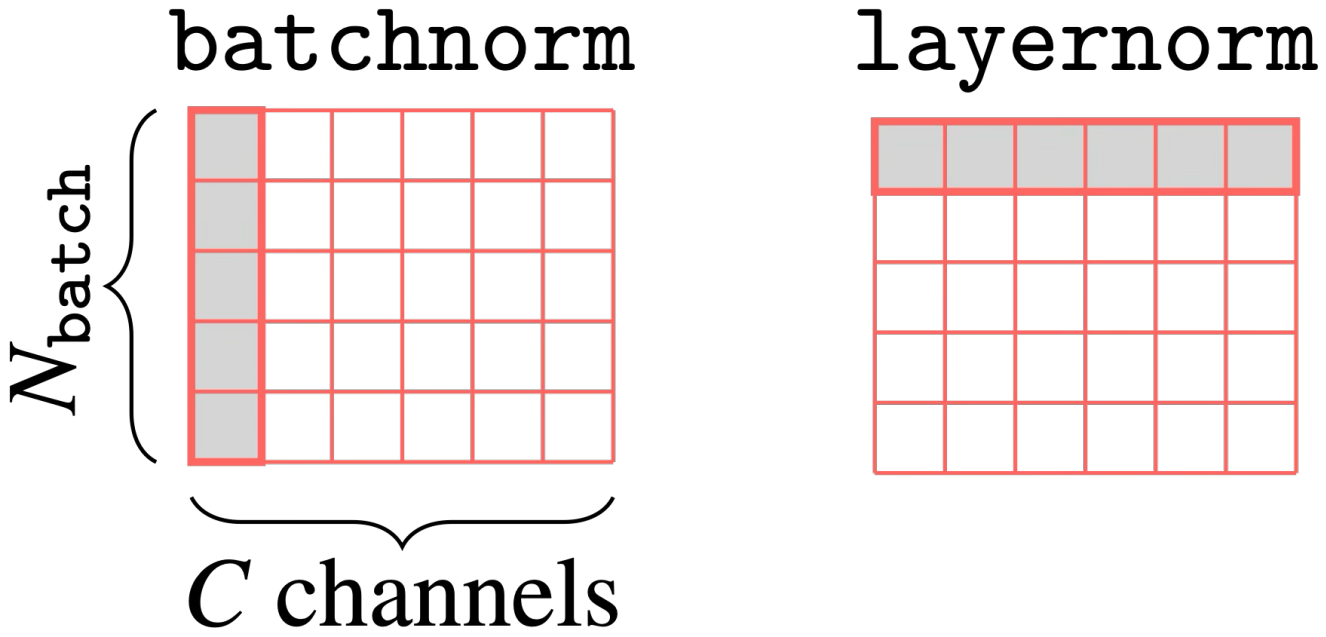
$$\widehat{x}_i = \frac{x_i - \mu_{\mathcal{G}}}{\sqrt{\sigma^2_{\mathcal{G}} + \epsilon}}, \qquad \tilde{h}_i = \gamma_i \widehat{x}_i + \beta_i$$

The chosen set determines which activations share a mean and variance.

Normalisation reduces drift in activation scale. Learned scale γ and offset β retain flexibility. The small positive ε stabilises the denominator.

Normalisation across examples or features

Rows are examples; columns are features. Highlighted cells share one mean and variance.



BatchNorm depends on the training minibatch. LayerNorm computes statistics separately for each example.

Adapted from K. Murphy (2022), Probabilistic Machine Learning: An Introduction, Fig. 14.14.

BatchNorm shares statistics across the batch

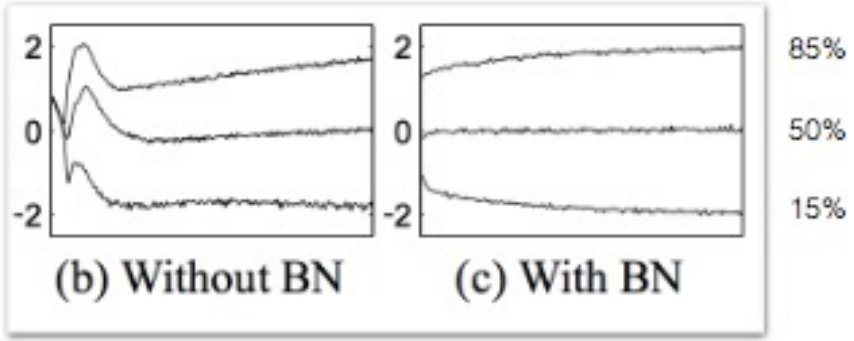
Each channel c gets one mean μ_c and variance σ_c^2, computed over the minibatch (and, in a CNN, over all positions).

For an activation x in channel c, with output $\tilde{h}$:

$$\widehat{x} = \frac{x - \mu_c}{\sqrt{\sigma_c^2 + \epsilon}}, \qquad \tilde{h} = \gamma_c \widehat{x} + \beta_c$$

At test time, use running averages of μ_c and σ_c^2 stored during training.

BN steadies one unit's input early on

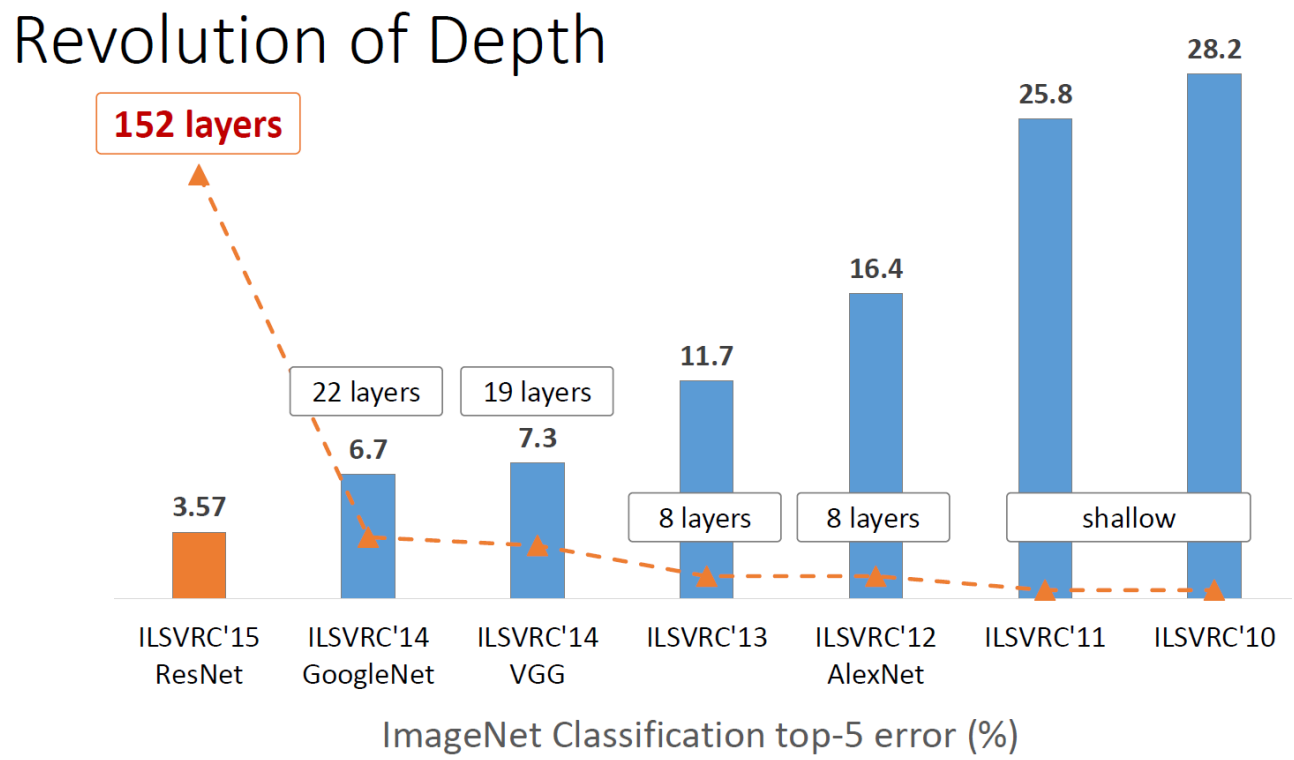


Training steps →

Ioffe and Szegedy (2015)

Trainable depth cut ImageNet error

ImageNet top-5 error fell as networks grew from 8 to 152 layers.



Training networks this deep relied on BatchNorm and residual connections (both 2015).

Figure: K. He, Deep Residual Networks tutorial (ICML 2016).

LayerNorm computes statistics per example

Compute statistics within each example over its chosen feature dimensions. Examples do not share statistics.

$$\mu_n = \frac{1}{D}\sum_{j=1}^{D} x_{nj}, \qquad \sigma_n^2 = \frac{1}{D}\sum_{j=1}^{D} (x_{nj} - \mu_n)^2$$

Transformers use LayerNorm by default.

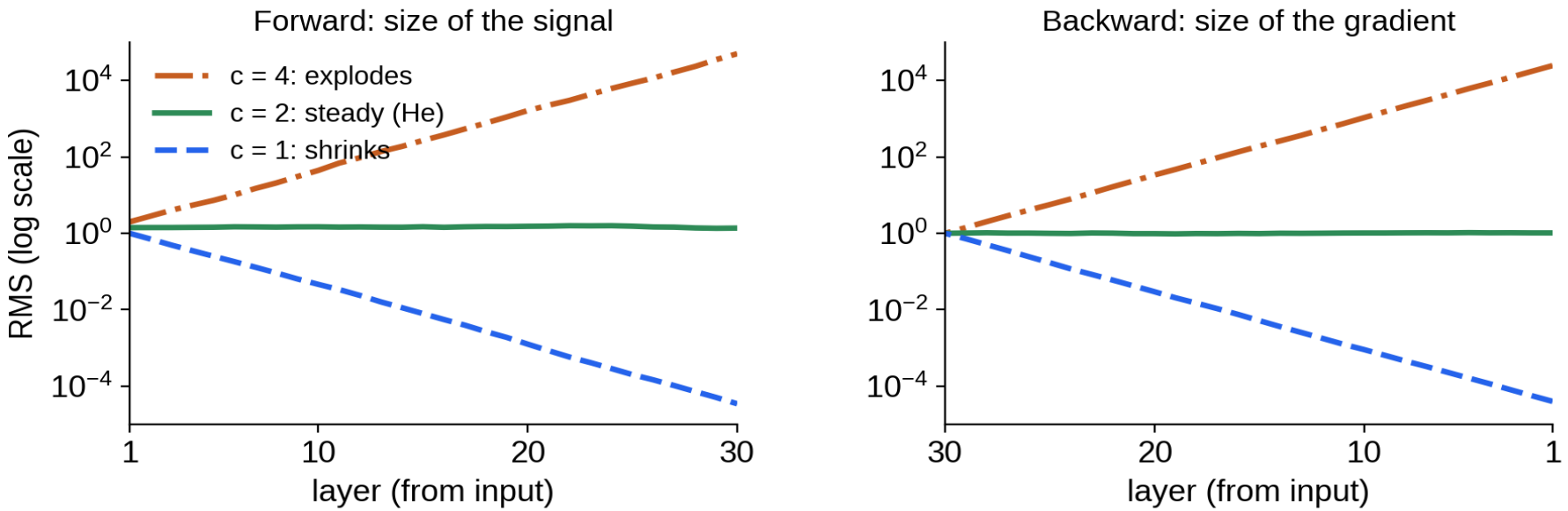
Training and inference use the same rule: there are no running batch statistics.

He initialisation: variance 2 / fan-in

He/Kaiming: use variance 2 / n_{in}, where n_{in} is the number of inputs.
Normal alternative: $W_{ij} \sim \mathcal{N}(0, 2/n_{in})$.

$$W_{ij} \sim \mathcal{U}\left(-\sqrt{6/n_{\text{in}}},\ \sqrt{6/n_{\text{in}}}\right) \quad \Longrightarrow \quad \text{Var}(W_{ij}) = \frac{2}{n_{\text{in}}}$$

Random ReLU network: 30 layers, width 256, weight variance c / fan-in.



The factor 2 compensates for the signal lost when ReLU suppresses negative inputs.

He et al. (2015), Delving Deep into Rectifiers; Glorot and Bengio (2010) for tanh.

Generalisation

Optimisation

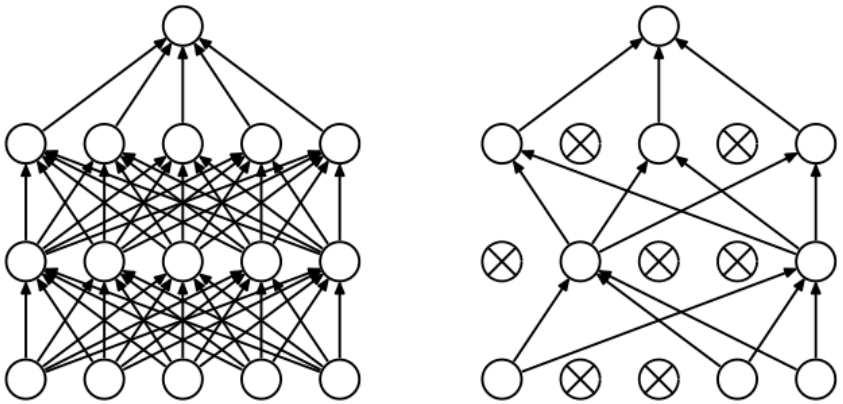
Generalisation

Dropout

Dataset augmentation

Early stopping

Convolutional networks



(a) Standard Neural Net

(b) After applying dropout.

Reading: Regularisation

K. Murphy (2022), §§13.5.2, 13.5.4 Weight decay and dropout

L2 regularisation gives weight decay

Penalise the weight matrices in each layer. Leave biases unpenalised.

Squared Frobenius norm

$$\lambda\Omega(\boldsymbol{\theta}) = \sum_l \lambda_l \|\mathbf{W}^l\|_F^2 = \sum_l \lambda_l \sum_{i,j} (W_{ij}^l)^2$$

Gradient of the penalty

$$\nabla_{\mathbf{W}^l}\left[\lambda\Omega(\boldsymbol{\theta})\right] = 2\lambda_l \mathbf{W}^l$$

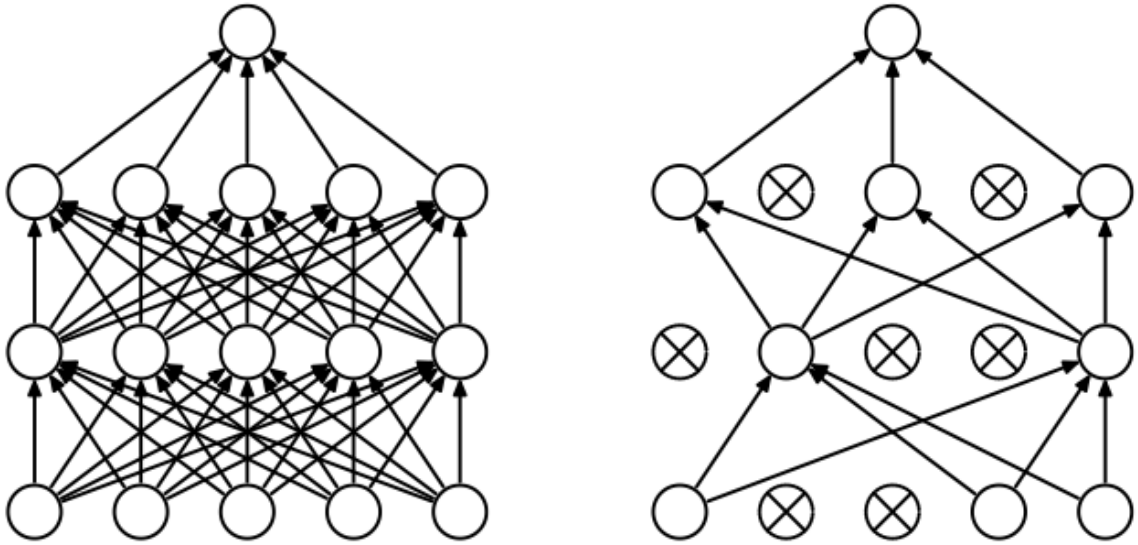
Minibatch SGD update

$$\begin{aligned}\mathbf{W}_{t+1}^l &= \mathbf{W}_t^l - \eta_t\left(\mathbf{G}_t^l + 2\lambda_l \mathbf{W}_t^l\right) \\ &= \underbrace{(1 - 2\eta_t\lambda_l)\mathbf{W}_t^l}_{\text{weight decay}} - \eta_t \mathbf{G}_t^l\end{aligned}$$

G_t^l is the minibatch data-loss gradient for the weight matrix W^l.

The decay term pulls the weights towards zero at each SGD update.

Dropout trains many thinned subnetworks



(a) Standard Neural Net (b) After applying dropout.

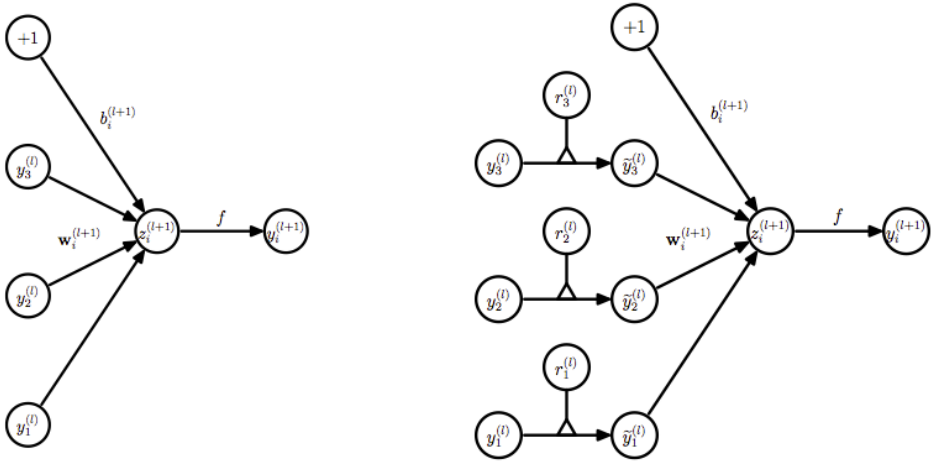
Each sample passes through a randomly thinned network.

What does this remind you of?

Random forests: different experts are averaged together.
Dropout approximates averaging subnetworks that share weights.

Srivastava et al. (2014), Dropout: A Simple Way to Prevent Neural Networks from Overfitting, JMLR, Fig. 1.

Dropout masks activations at random



(a) Standard network (b) Dropout network

Figure 3: Comparison of the basic operations of a standard and dropout network.

$$
\begin{aligned}
u_i^{l+1} &= \mathbf{w}_i^{l+1}\mathbf{a}^l + b_i^{l+1}, \\
a_i^{l+1} &= g(u_i^{l+1}),
\end{aligned}
$$

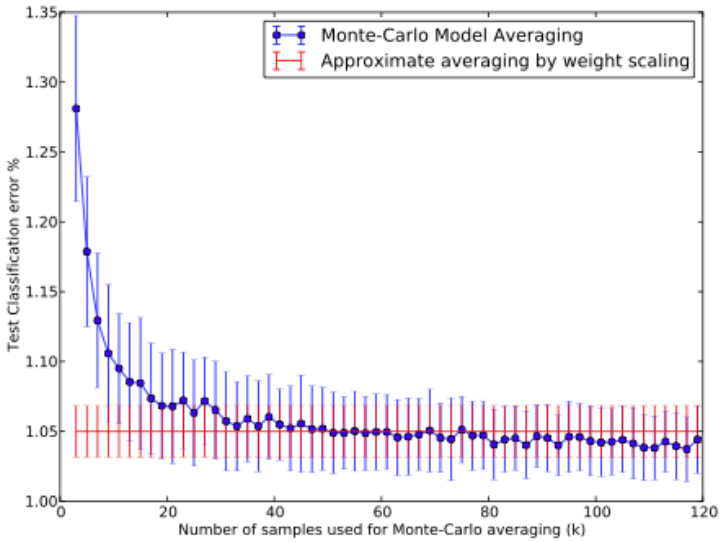
$$
\begin{aligned}
r_j^l &\sim \text{Bernoulli}(p), \\
\tilde{\mathbf{a}}^l &= \mathbf{r}^l \odot \mathbf{a}^l, \\
u_i^{l+1} &= \mathbf{w}_i^{l+1}\tilde{\mathbf{a}}^l + b_i^{l+1}, \\
a_i^{l+1} &= g(u_i^{l+1}).
\end{aligned}
$$

‘Feature noising’

p: keep probability; r: mask; g: activation function; ⊙: elementwise product.
Source figure: y → a, z → u, f → g. Equations use the notes’ notation.

Srivastava et al. (2014), Dropout, JMLR, Fig. 3; equations rewritten in this lecture’s notation.

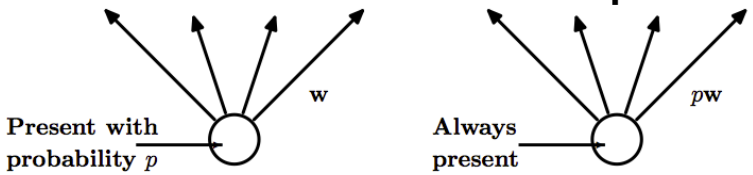
At test time, keep all units and rescale



MNIST test error, same ReLU network

Regulariser	Test error
L2	1.62%
Dropout + L2	**1.25%**
Max-norm	1.35%
Dropout + max-norm	**1.05%**

Dropout helps with either penalty.



(a) At training time (b) At test time

Figure 2: **Left**: A unit at training time that is present with probability p and is connected to units in the next layer with weights $\mathbf{w}$. **Right**: At test time, the unit is always present and the weights are multiplied by p. The output at test time is same as the expected output at training time.

Here p is the keep probability. At test time, keep every unit and scale outgoing weights by p. This is an approximation.

Srivastava et al. (2014), Dropout, JMLR: Figs. 2 and 11; Table 9 (784-1024-1024-2048-10 ReLU network).

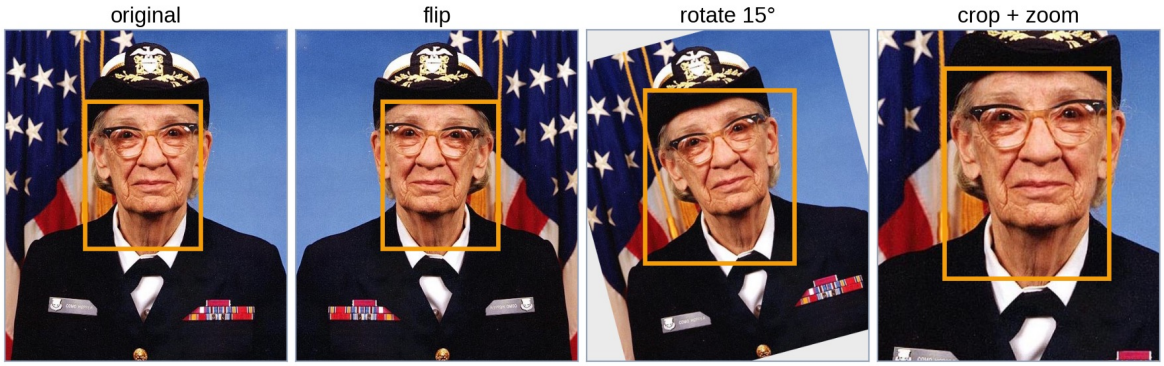
Generalisation

Optimisation

Generalisation

- Dropout
- **Dataset augmentation**
- Early stopping

Convolutional networks



Dataset augmentation: one image, many views

Transform training images in ways that keep the label valid, so the network sees more variety.

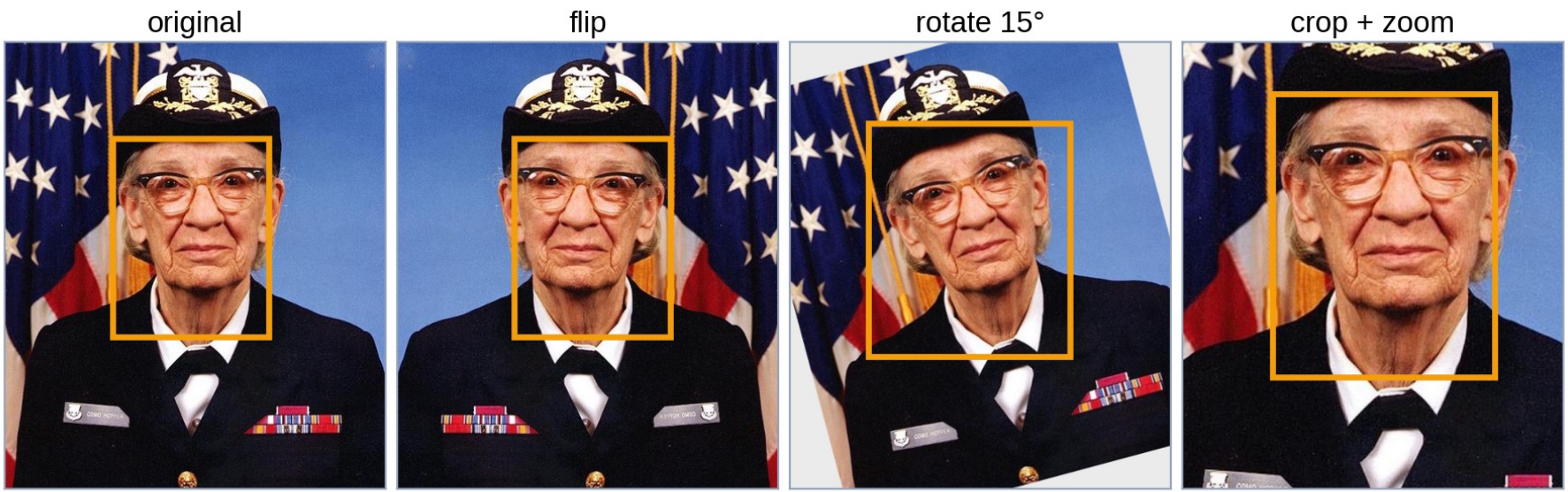


Which transformations are safe depends on the task: rotate a 6 by 180° and it becomes a 9.

Photo: Grace Hopper, official U.S. Navy photograph (public domain).

Detection: boxes move with the image

For detection the label is a box, so every geometric change must transform the box too.



A rotated box is replaced by the axis-aligned box around its corners, which is slightly larger.

Photo: Grace Hopper, official U.S. Navy photograph (public domain).

Data augmentation and label preservation

Choose transformations that preserve the task's target.

Task	Possible transformation	What must stay consistent
Image classification	Crop or colour change	The image label remains valid
Object detection	Flip or resize	Transform the boxes too
Semantic segmentation	Crop or flip	Transform the pixel mask too

An augmentation that removes the object can invalidate its label.

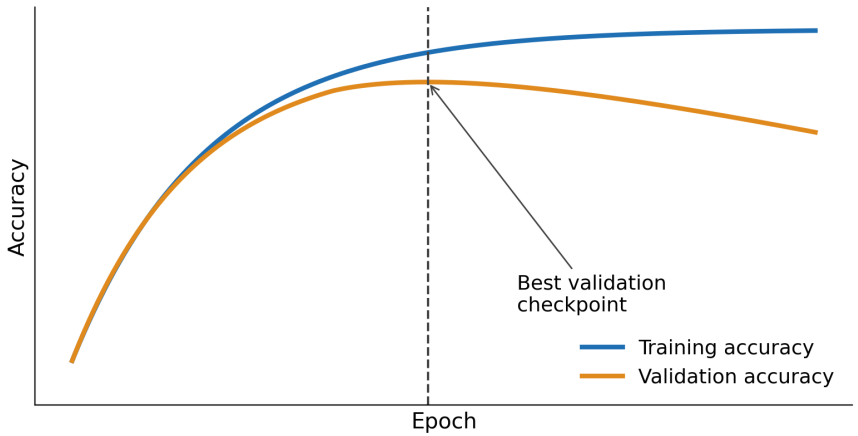
Generalisation

Optimisation

Generalisation

- Dropout
- Dataset augmentation
- Early stopping

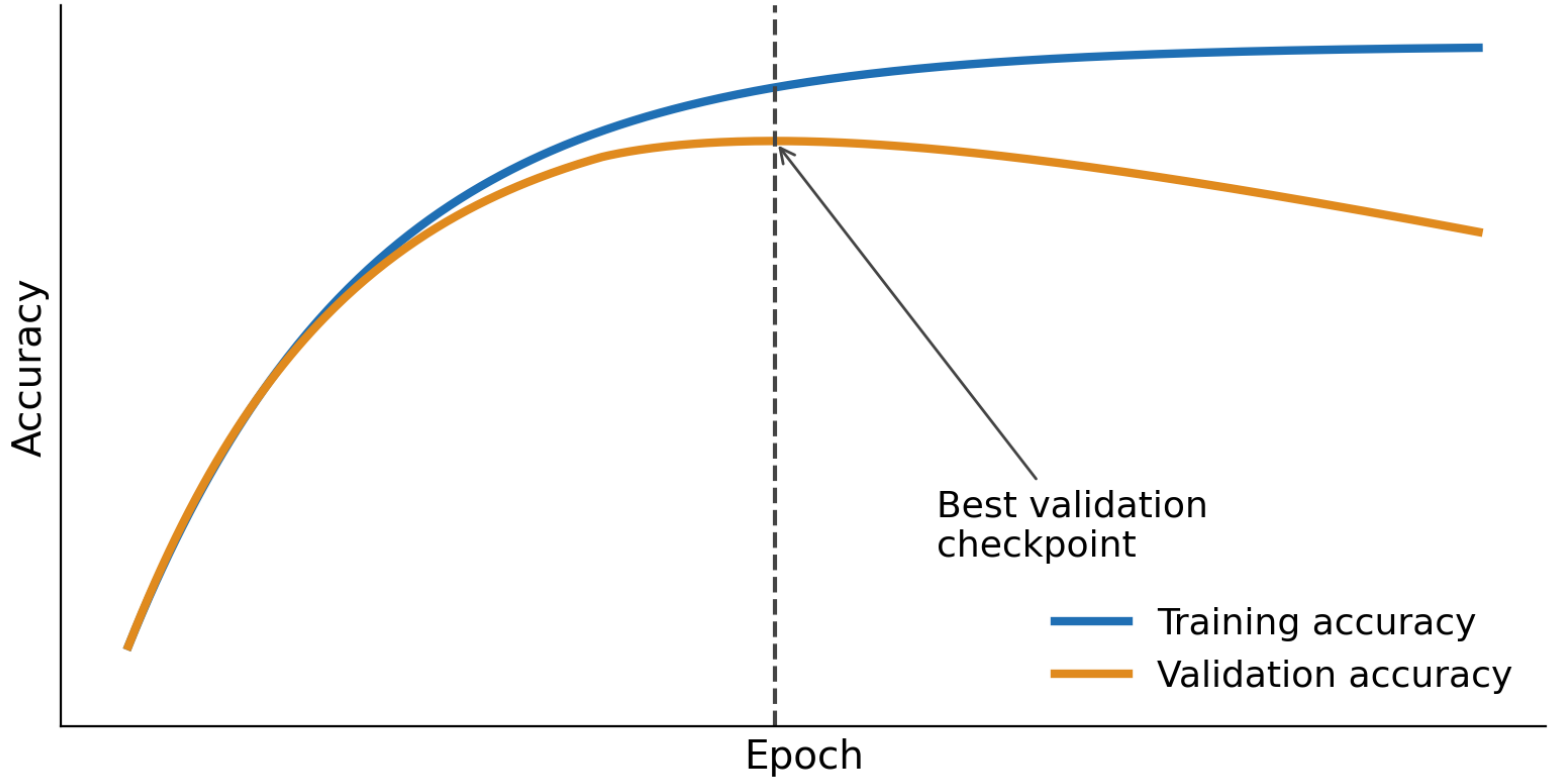
Convolutional networks



Reading: Early stopping

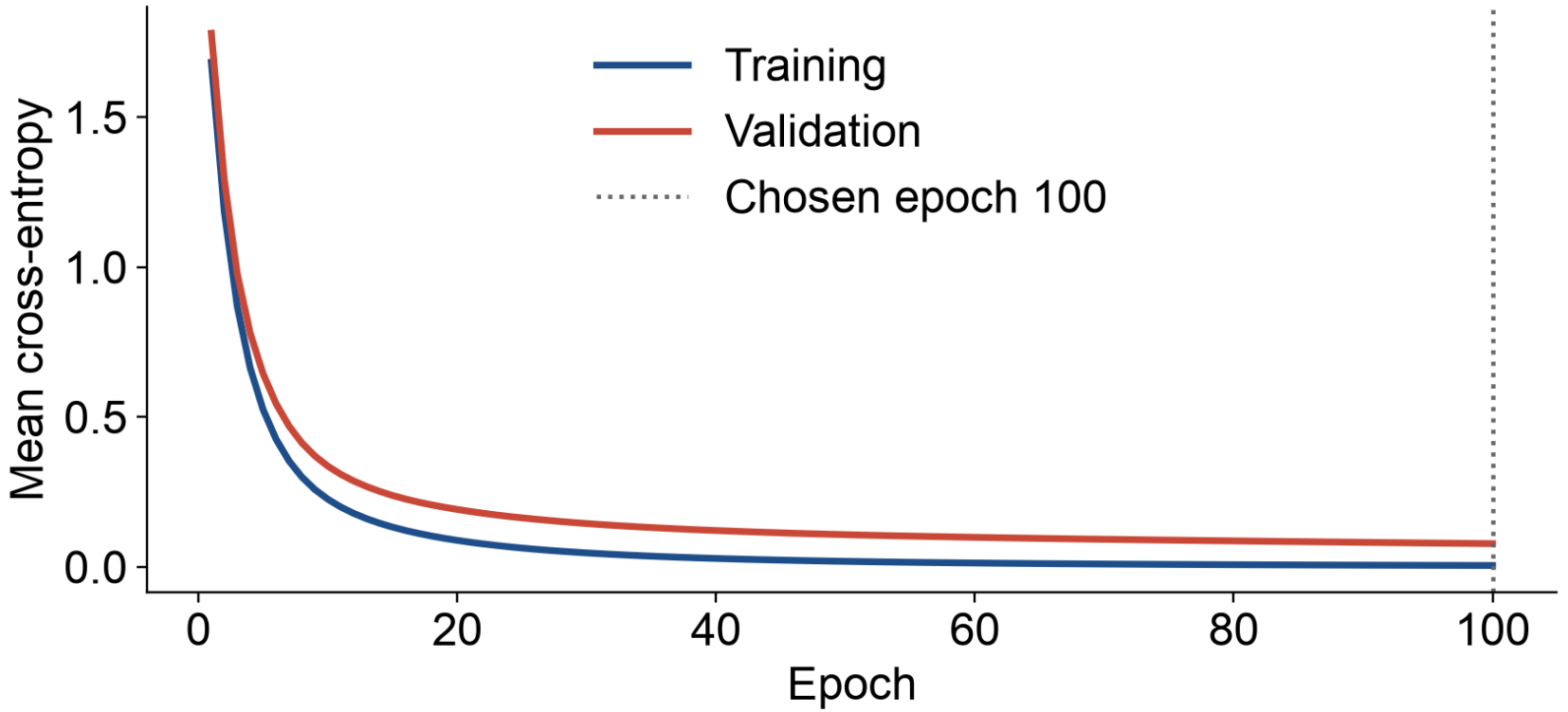
K. Murphy (2022), §13.5.1 Early stopping

Early stopping keeps the best validation point



- Monitor validation performance and retain the best checkpoint.
- Stop after a chosen period without improvement. Keep the test set separate.

Here validation never rises: keep the last epoch



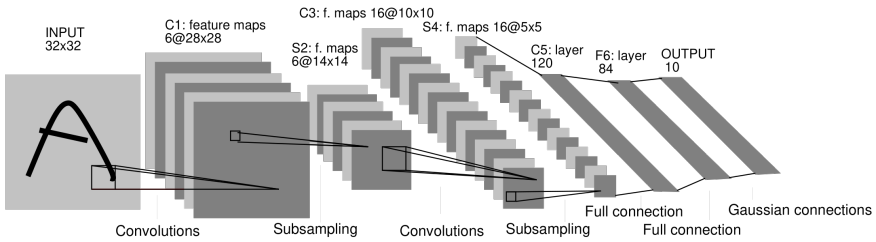
Validation selects epoch 100 here. Other runs may select an earlier epoch.

Convolutional networks

Optimisation

Generalisation

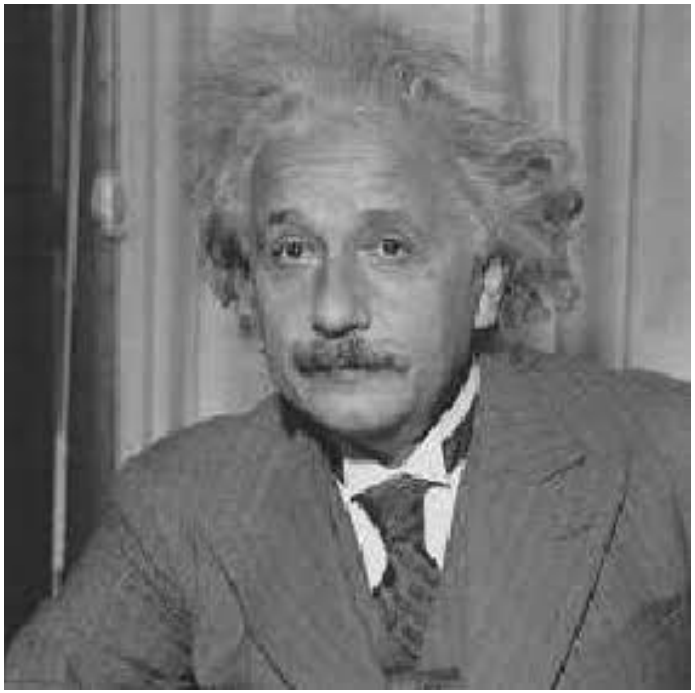
Convolutional networks



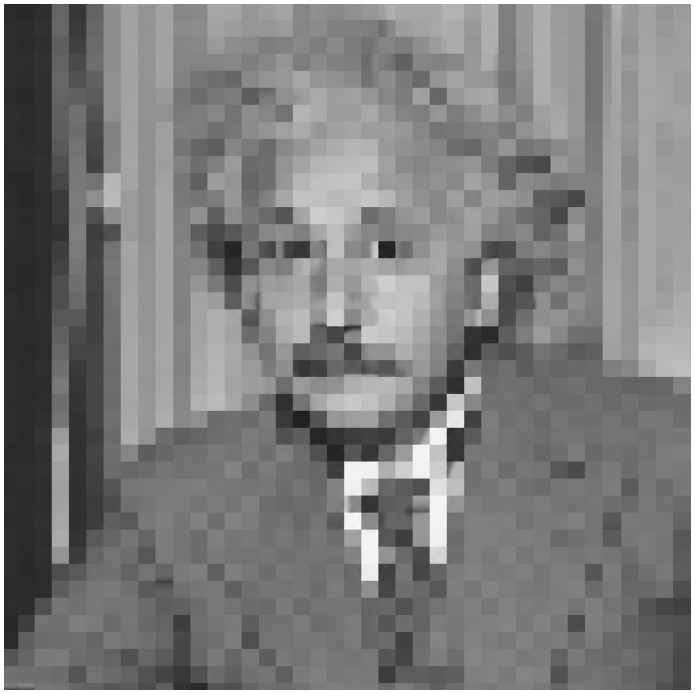
Reading: Convolutional networks

K. Murphy (2022), Chapter 14: Neural networks for images

Images as arrays of pixels



Original image

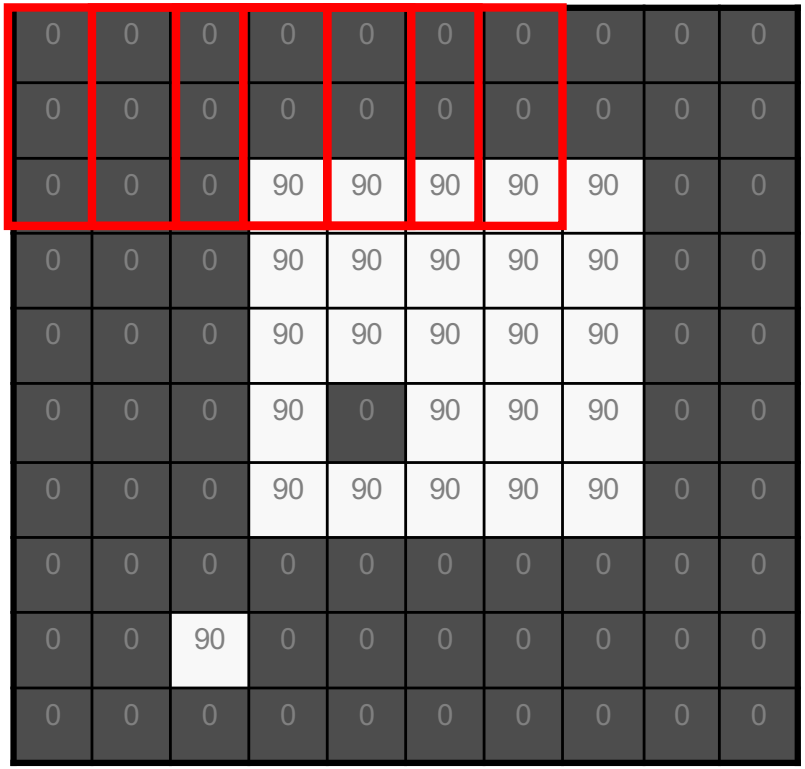


Coarse pixel view

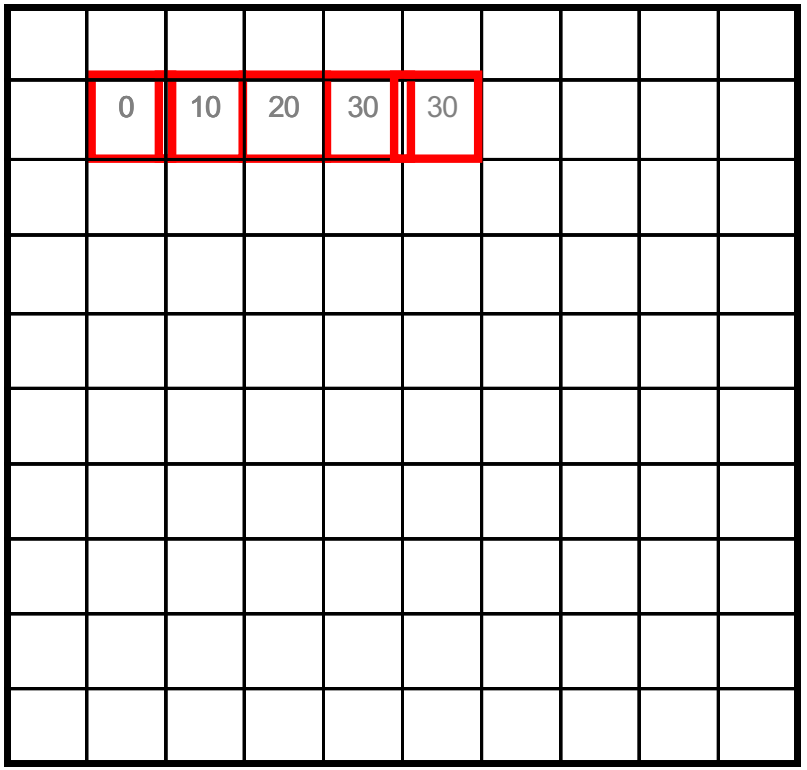
A greyscale image is a 2D array of intensity values.

Moving Average in 2D

$F[x, y]$



$G[x, y]$



Each click shifts the window and writes its average.

Moving Average in 2D

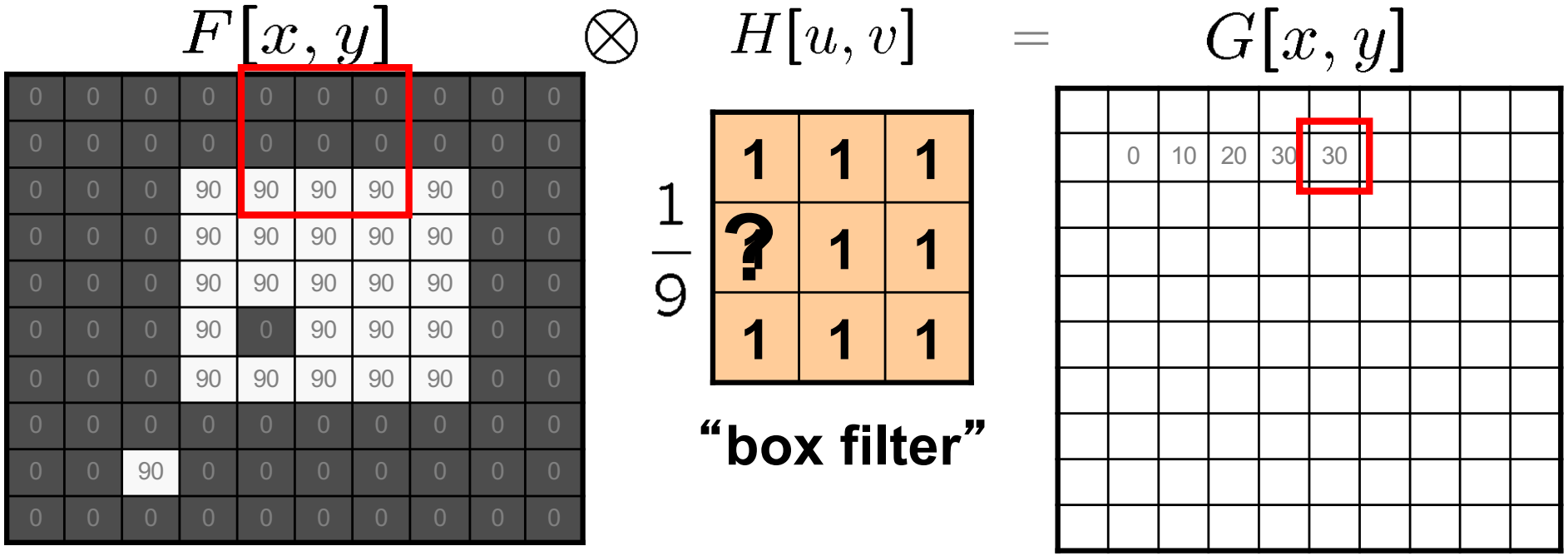
$F[x, y]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$G[x, y]$

	0	10	20				20	10	
	0	20	40	60	60	60	40	20	
	0		60	90	90	90	60		
	0		50	80	80	90	60		
	0		50	80	80	90	60		
	0	20		50	50	60	40	20	
	10	20					20	10	
	10	10	10	0	0	0	0	0	

Averaging = Filtering

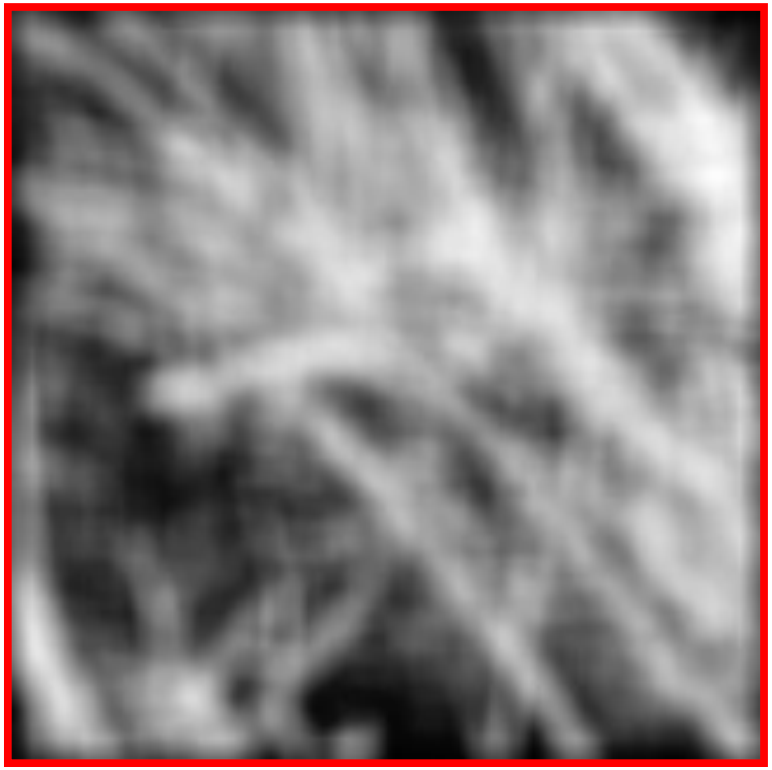


"box filter"

$$G = H \otimes F$$

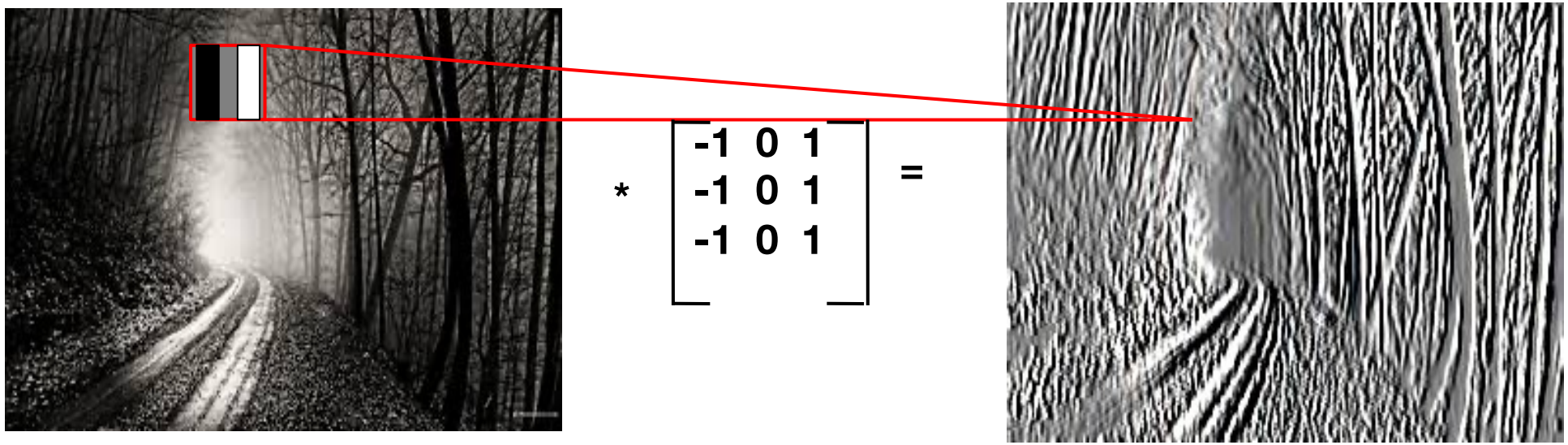


Original



Filtered

An edge-detection filter

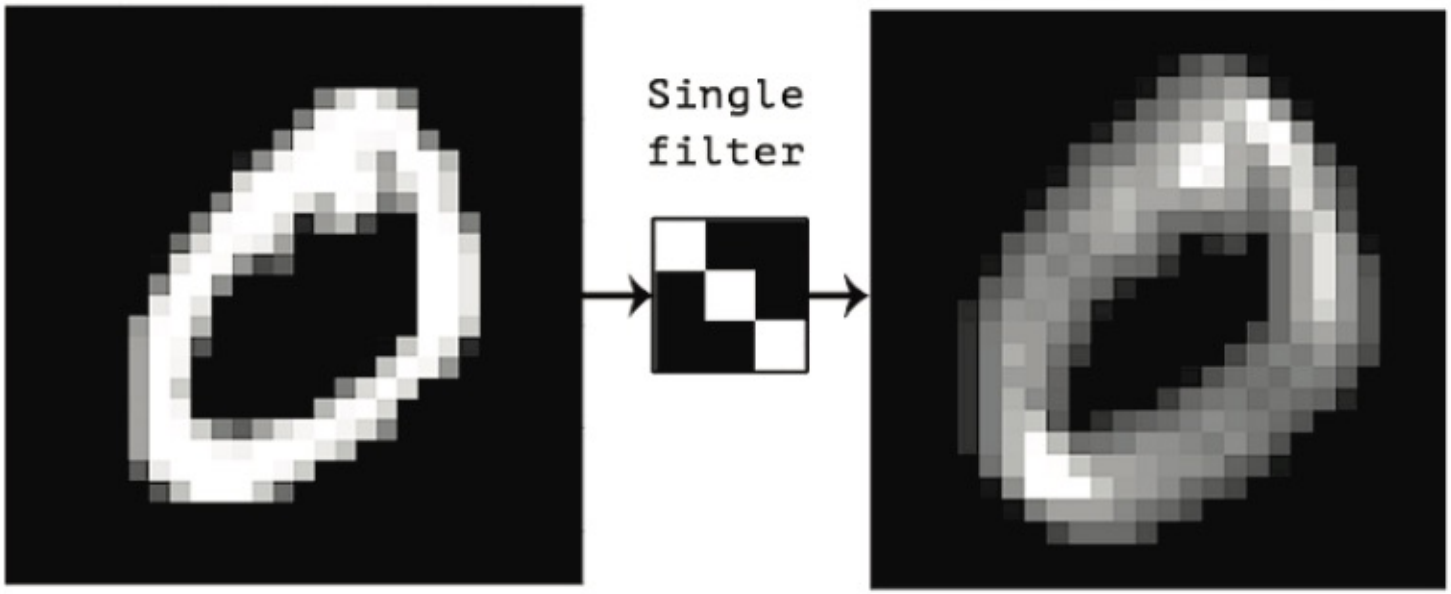


Filters find local patterns

A filter responds to a local pattern, such as an edge or a stroke.

Sliding the same filter across the image finds that pattern at different positions.

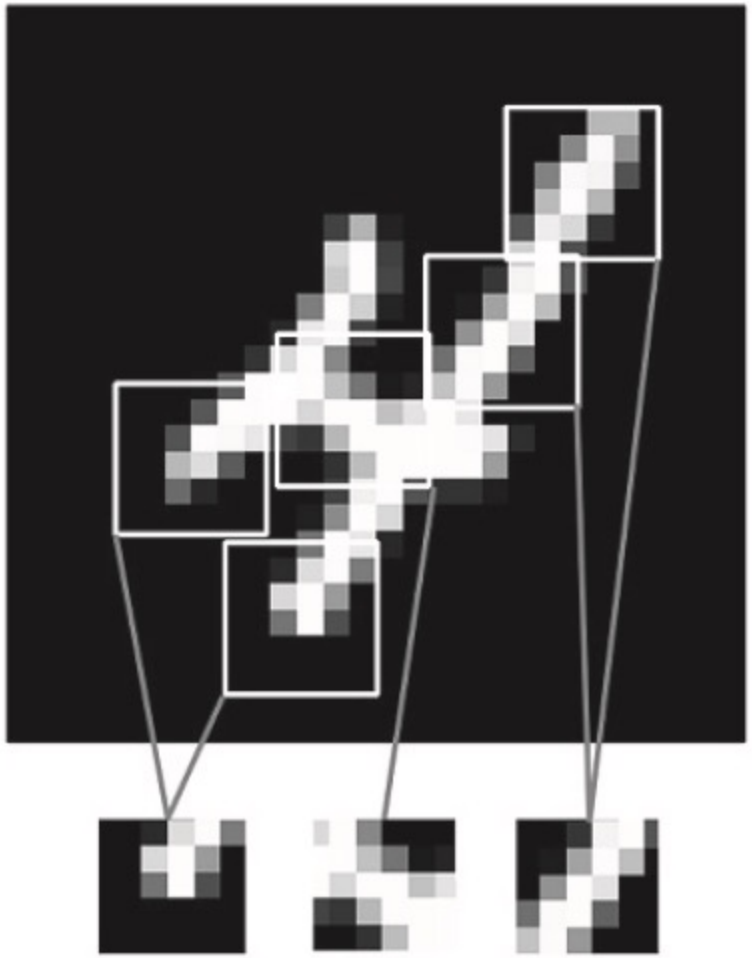
Hand-designed filters smooth images or detect edges. A CNN learns useful filters from the task loss.



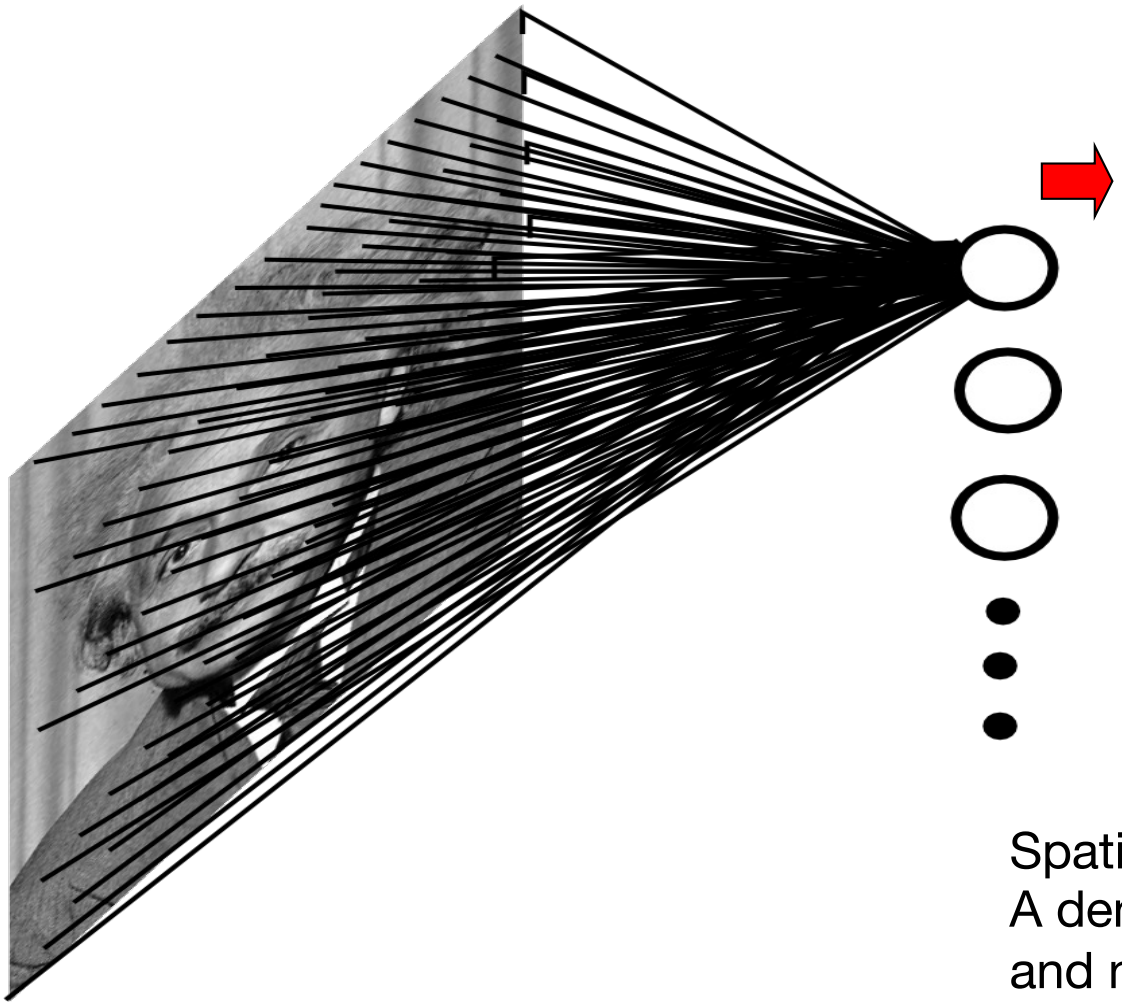
A digit needs many local patterns

Recognising a digit needs several patterns: stroke ends, curves and crossings, combined in the right positions.

How do we learn these patterns? Train several shared filters together, using backpropagation from the classification loss.



Dense layers: independent weights

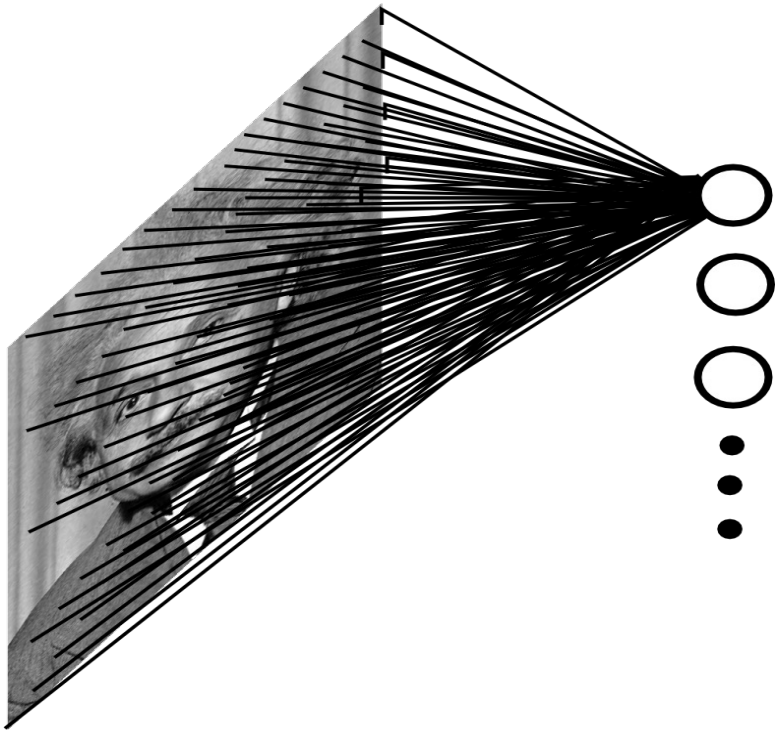


200 × 200 image
40,000 hidden units
About 1.6 billion weights

Spatial correlations are often local.
A dense layer uses many parameters
and needs substantial training data.

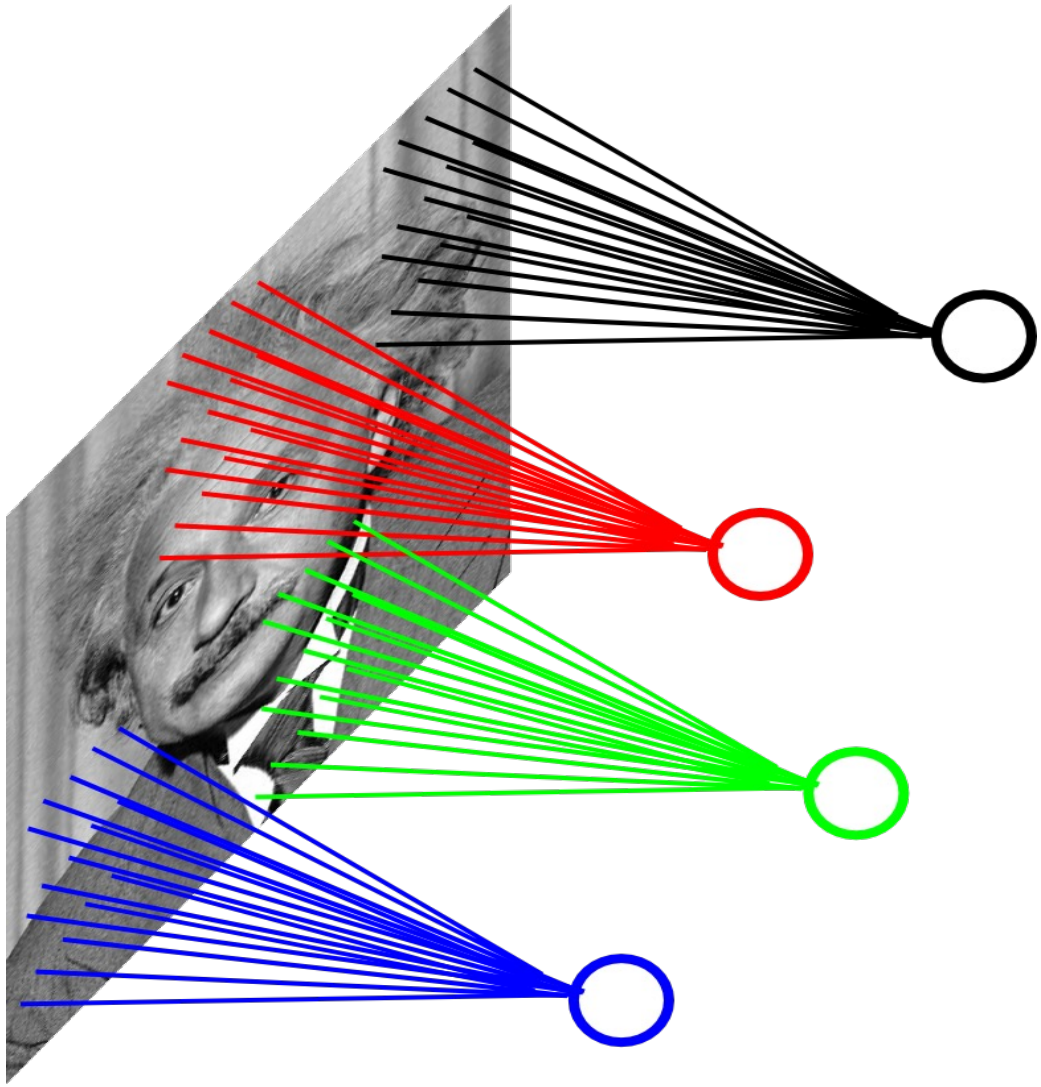
Fully connected layers: parameter count

of parameters:
K^2



$$\begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ \vdots \\ u_K \end{bmatrix} = \begin{bmatrix} w_{1,1} & w_{1,2} & w_{1,3} & w_{1,4} & \cdots & w_{1,K} \\ w_{2,1} & w_{2,2} & w_{2,3} & w_{2,4} & \cdots & w_{2,K} \\ w_{3,1} & w_{3,2} & w_{3,3} & w_{3,4} & \cdots & w_{3,K} \\ w_{4,1} & w_{4,2} & w_{4,3} & w_{4,4} & \cdots & w_{4,K} \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ w_{K,1} & w_{K,2} & w_{K,3} & w_{K,4} & \cdots & w_{K,K} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ \vdots \\ x_K \end{bmatrix}$$

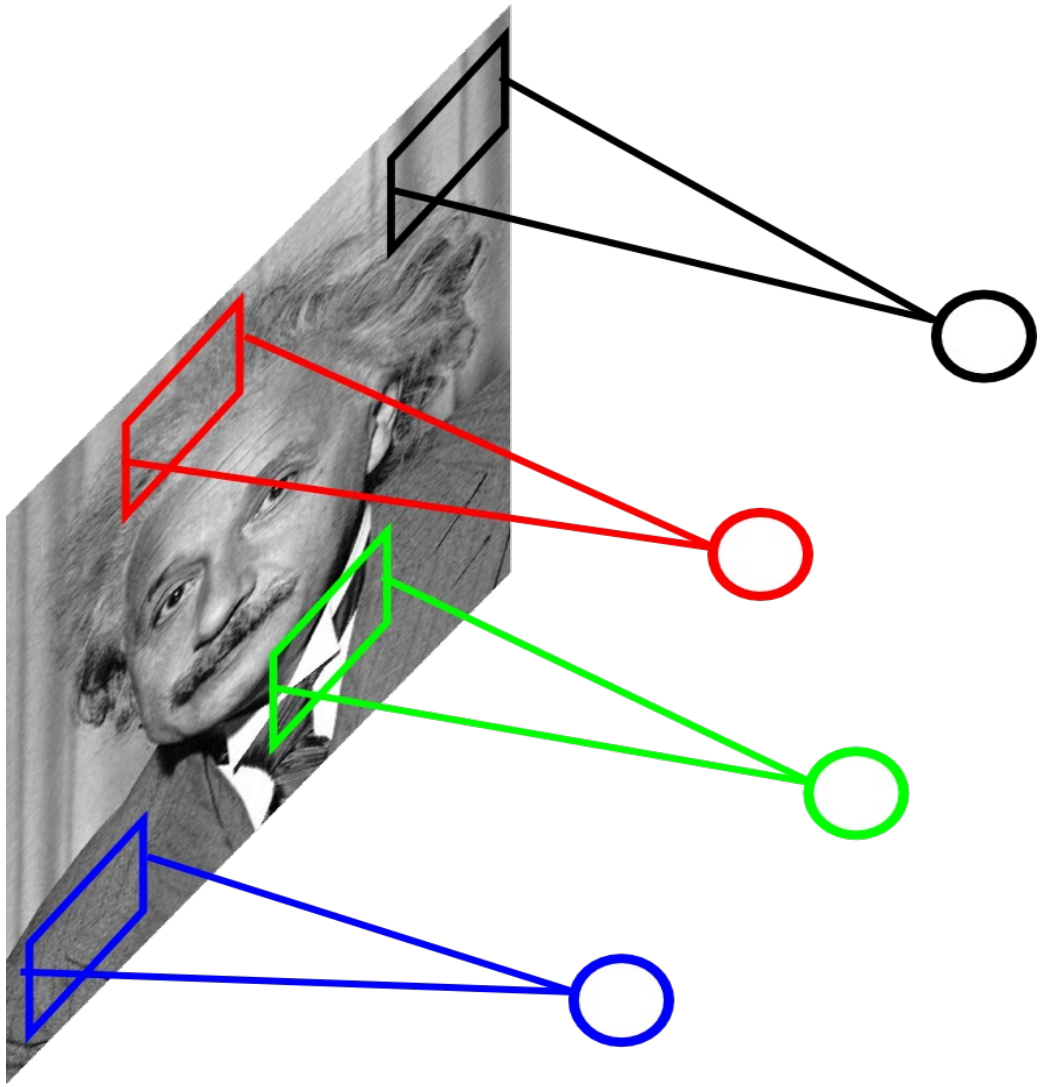
Locality: a sparse weight matrix



200 × 200 image
40,000 hidden units
10 × 10 local window
4 million weights

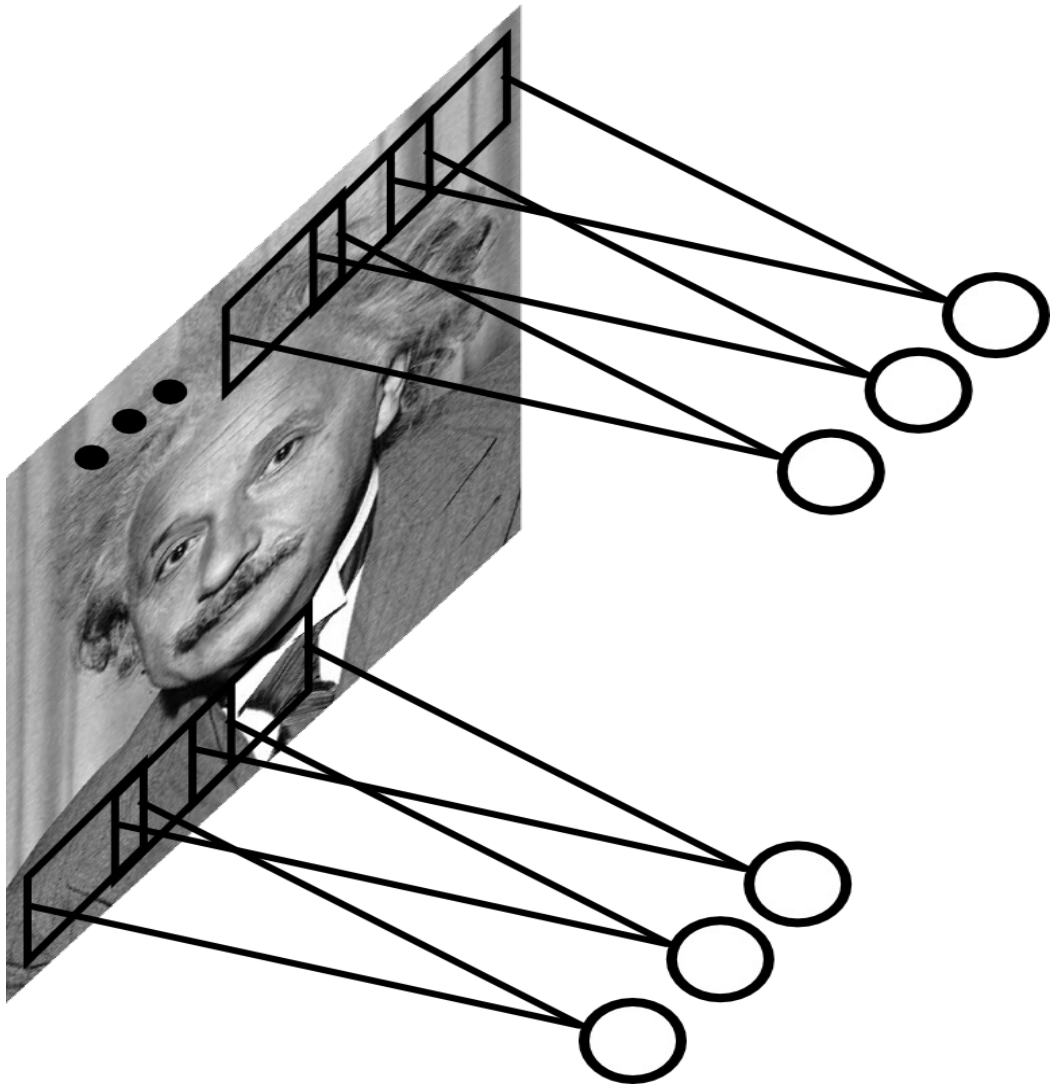
Fix weights outside each local window to zero.

Local weights can vary across locations



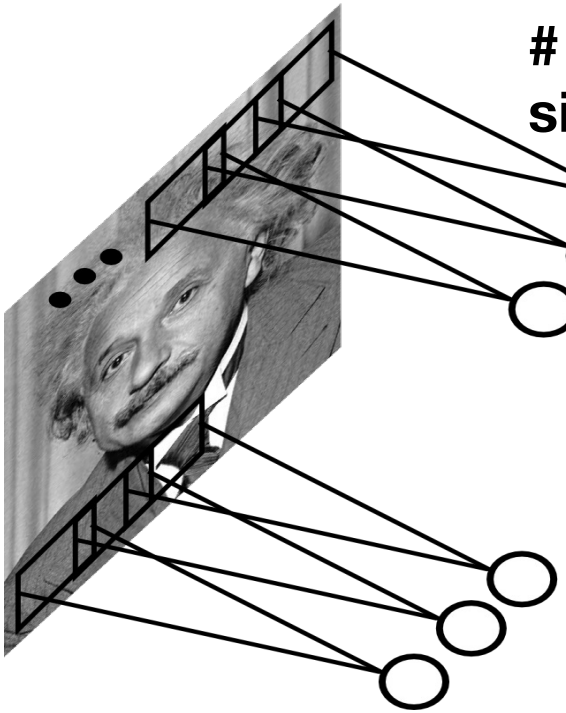
200 × 200 image
40,000 hidden units
10 × 10 local window
4 million weights

Convolution: local and shared weights



Reuse the same weights at every location.
Learn only the shared parameters.

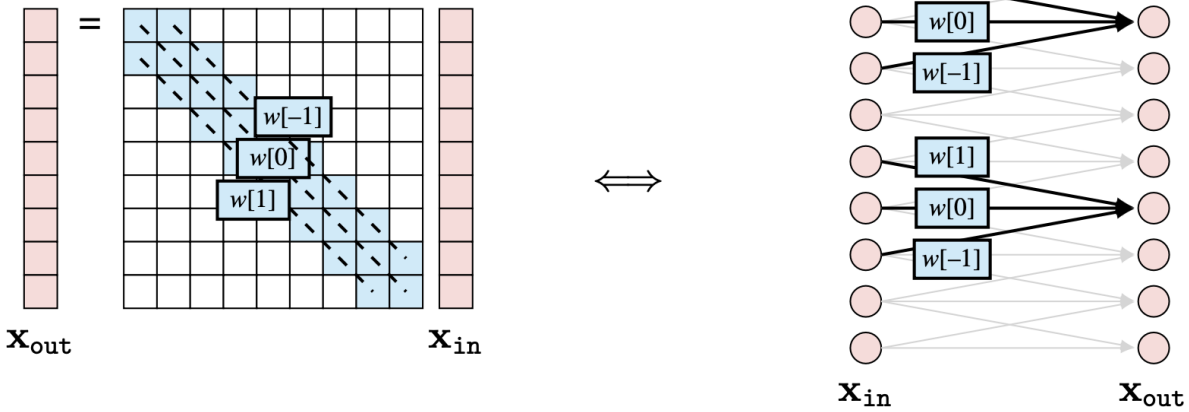
Convolutional layers: parameter count



of parameters:
size of window

The same filter as a matrix and a network

One input channel, one output channel



Three learned weights repeat across locations. Other connections are zero.

Try it: count the parameters

Input: a 32 × 32 colour image (3 channels). Output: 64 feature maps of size 32 × 32. Ignore biases.

a. How many weights does a fully connected layer need?

b. How many does a 3 × 3 convolutional layer need?

Pause: two minutes with your neighbour. Click to reveal the answers.

a. $(32 \cdot 32 \cdot 3) \times (32 \cdot 32 \cdot 64) = 3{,}072 \times 65{,}536 \approx 2 \times 10^8$. b. $3 \cdot 3 \cdot 3 \times 64 = 1{,}728$: more than 100,000 times fewer.

Convolution as architectural regularisation

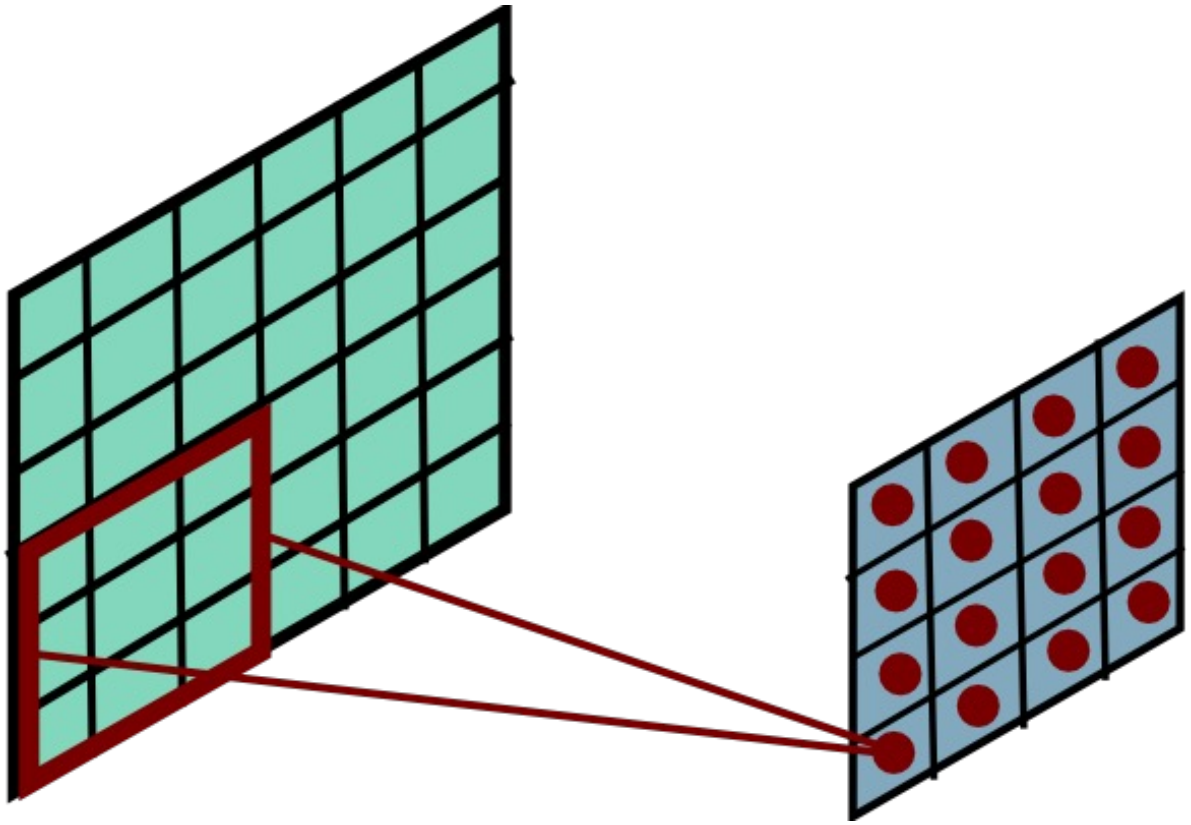
A penalty such as weight decay discourages some weight values.
An architecture constrains which weight matrices are possible.

Locality fixes many weights to zero.
Weight sharing ties multiple connections to the same parameter.

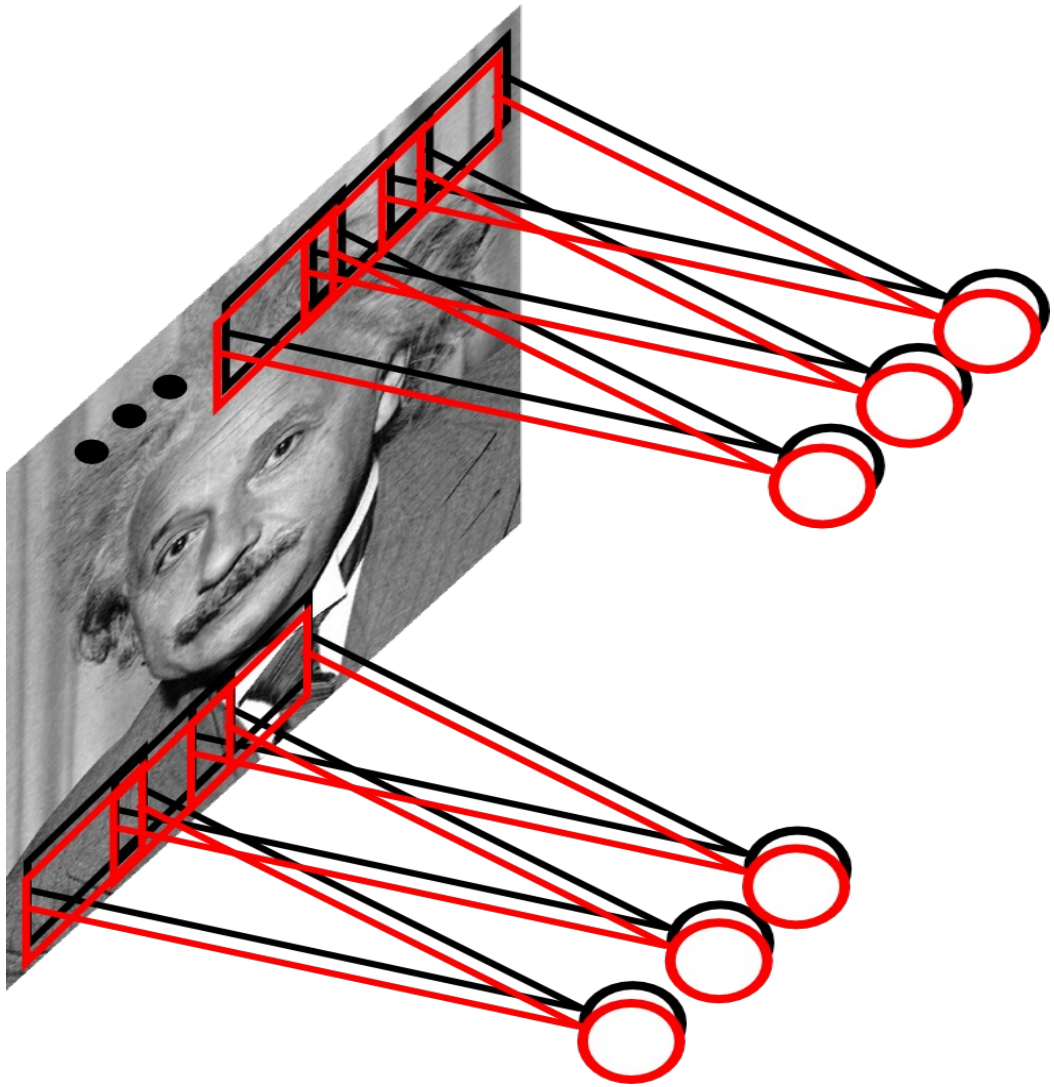
These constraints reduce the model's degrees of freedom and encode assumptions about the data.

Training still minimises a task loss using backpropagation.

Convolutional layers



Learning multiple convolutional filters

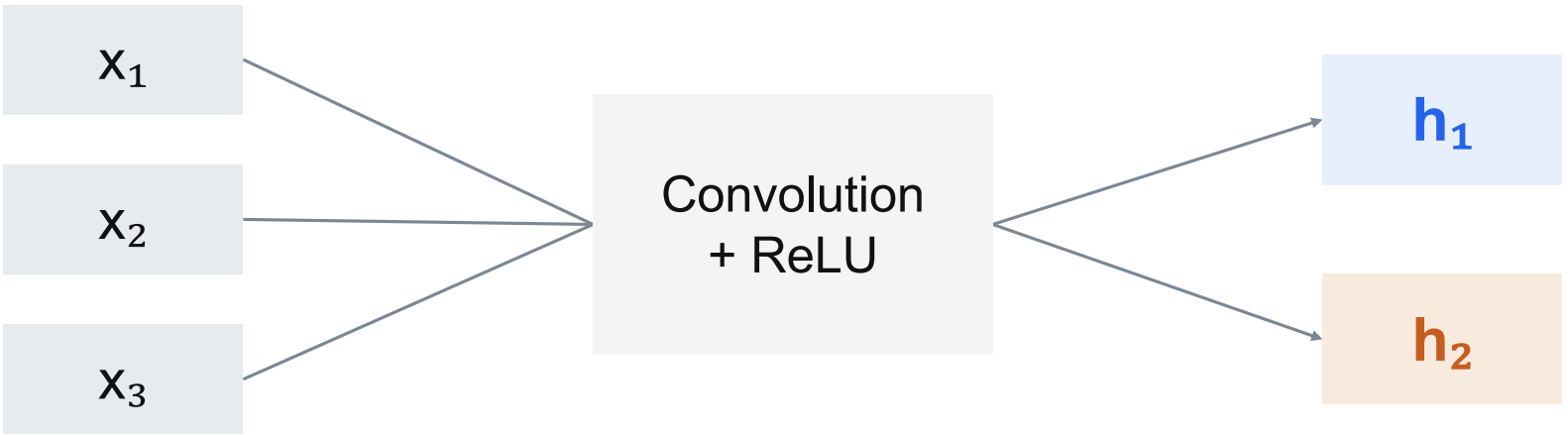


Multiple shared kernels

200 × 200 image
100 filters of size 10 × 10
10,000 weights for one input channel

Input and output feature maps

A convolutional layer produces one feature map per learned filter.

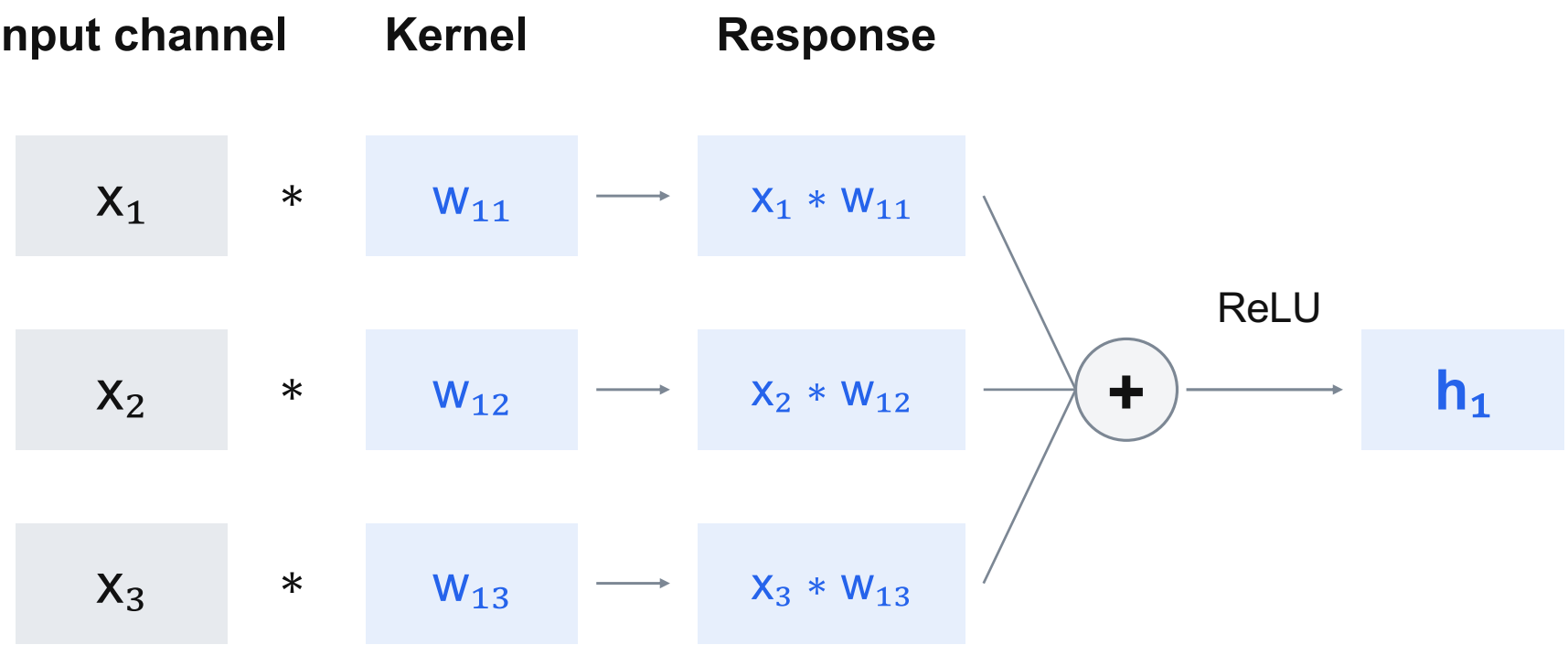


$$\mathbf{h}_i = \mathrm{ReLU}\left(\sum_{j=1}^{C_{\text{in}}} \mathbf{x}_j * \mathbf{w}_{ij}\right)$$

Each filter spans every input channel.

Computing one output channel

Convolve each input channel with its kernel, then sum the responses.



One filter contains one kernel per input channel.

Each output channel has its own kernels

The same inputs feed each output channel through different weights.

	Output channel 1	Output channel 2
Input channel 1	W_{11}	W_{21}
Input channel 2	W_{12}	W_{22}
Input channel 3	W_{13}	W_{23}

Each output sums its channel responses before ReLU.

3 input channels × 2 output channels = 6 kernels, each of size k × k.

Pooling over neighbouring responses

Summarise each local window within a feature map.

Input feature map

1	3	2	0
4	6	1	5
0	2	7	1
3	1	2	8

Max pooling

6	5
3	8

Keep the largest response.

Average pooling

3.5	2
1.5	4.5

Take the mean response.

2 × 2 windows, stride 2 halve the height and width.

Apply pooling independently to each channel. No learned weights.

Pooling and local robustness

A response can move within a pool without changing its maximum.



Maximum = 8

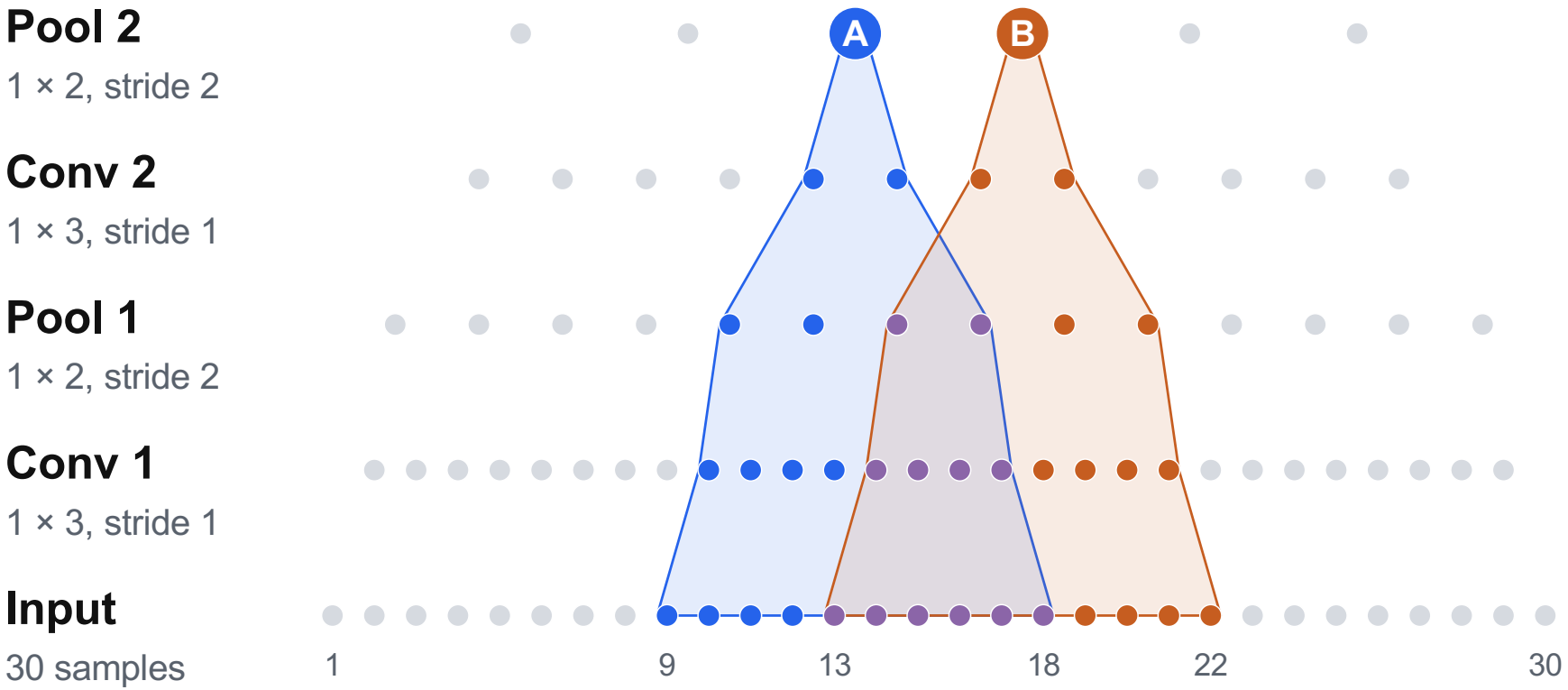


Maximum = 8

Moving across a pooling boundary can change the output.

Receptive fields in a 1D network

Convolution stride 1, pooling stride 2. No padding.



A: input samples 9–18 **B: input samples 13–22**

Each receptive field spans 10 samples. The two overlap on 6 samples.

Try it: receptive-field size

Convolutions have stride 1 and no padding.

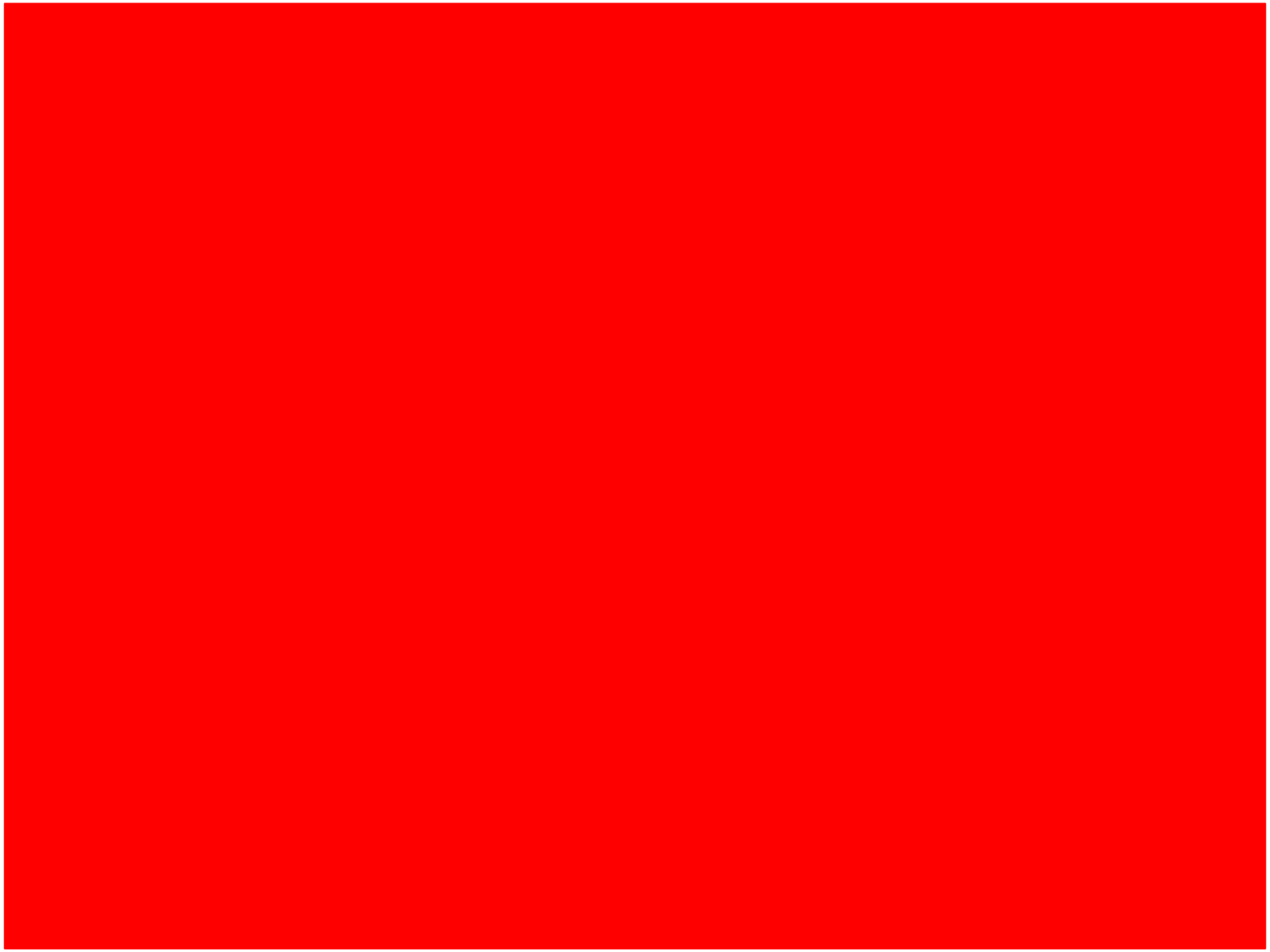
a. Three 3 × 3 convolutions in a row: how large is one output's receptive field?

b. A 3 × 3 convolution, a 2 × 2 max-pool with stride 2, then two more 3 × 3 convolutions?

Pause: two minutes with your neighbour. Click to reveal the answers.

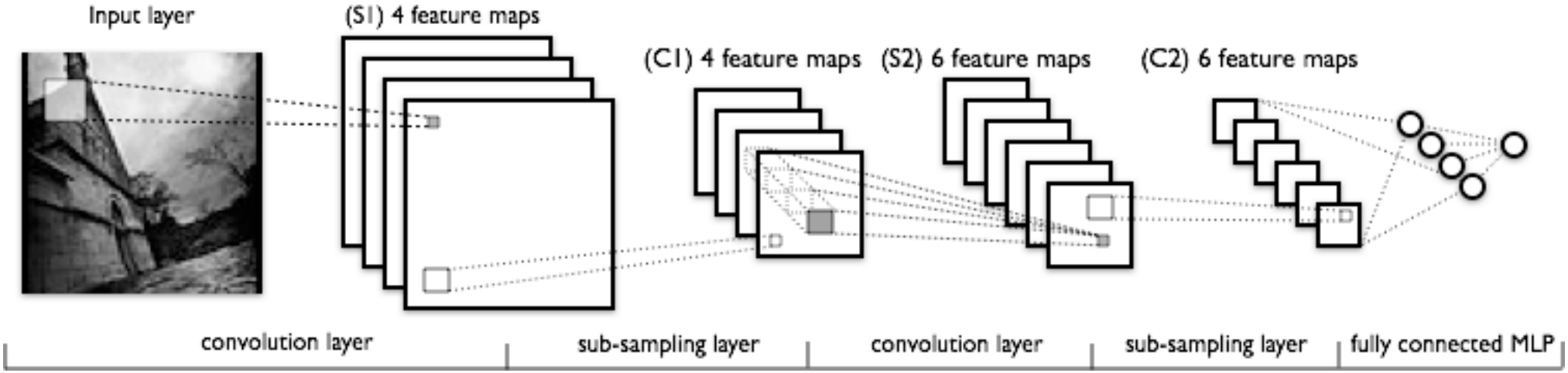
a. 7 × 7: each layer adds 2. b. 3, then 4 after the pool. The pool doubles the step, so each later convolution adds 4: 8, then 12 × 12.

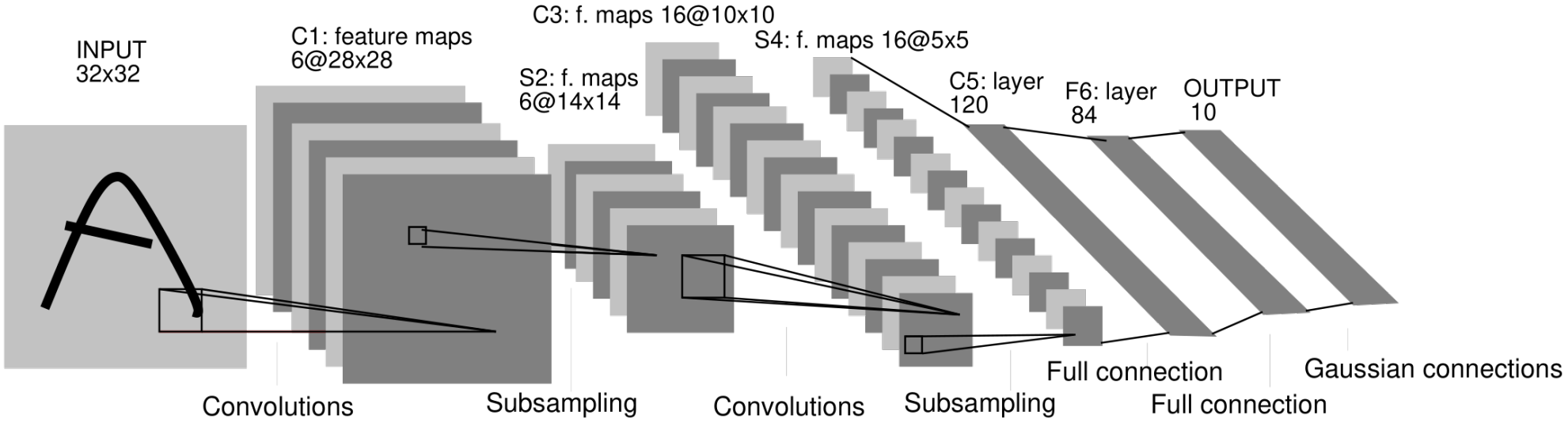
Field of view: Layer 8



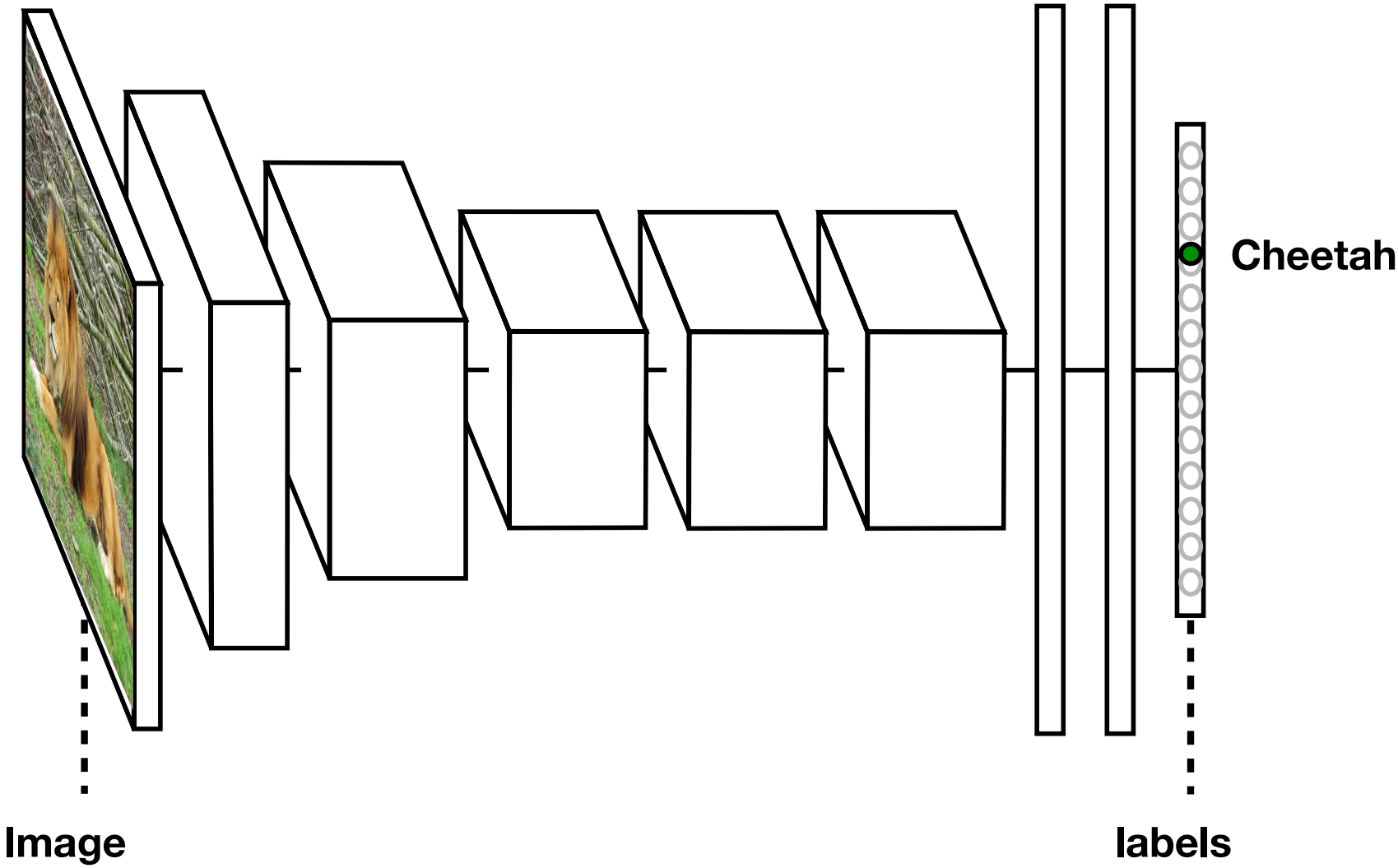
Slide credit: A. Harley

A convolutional neural network



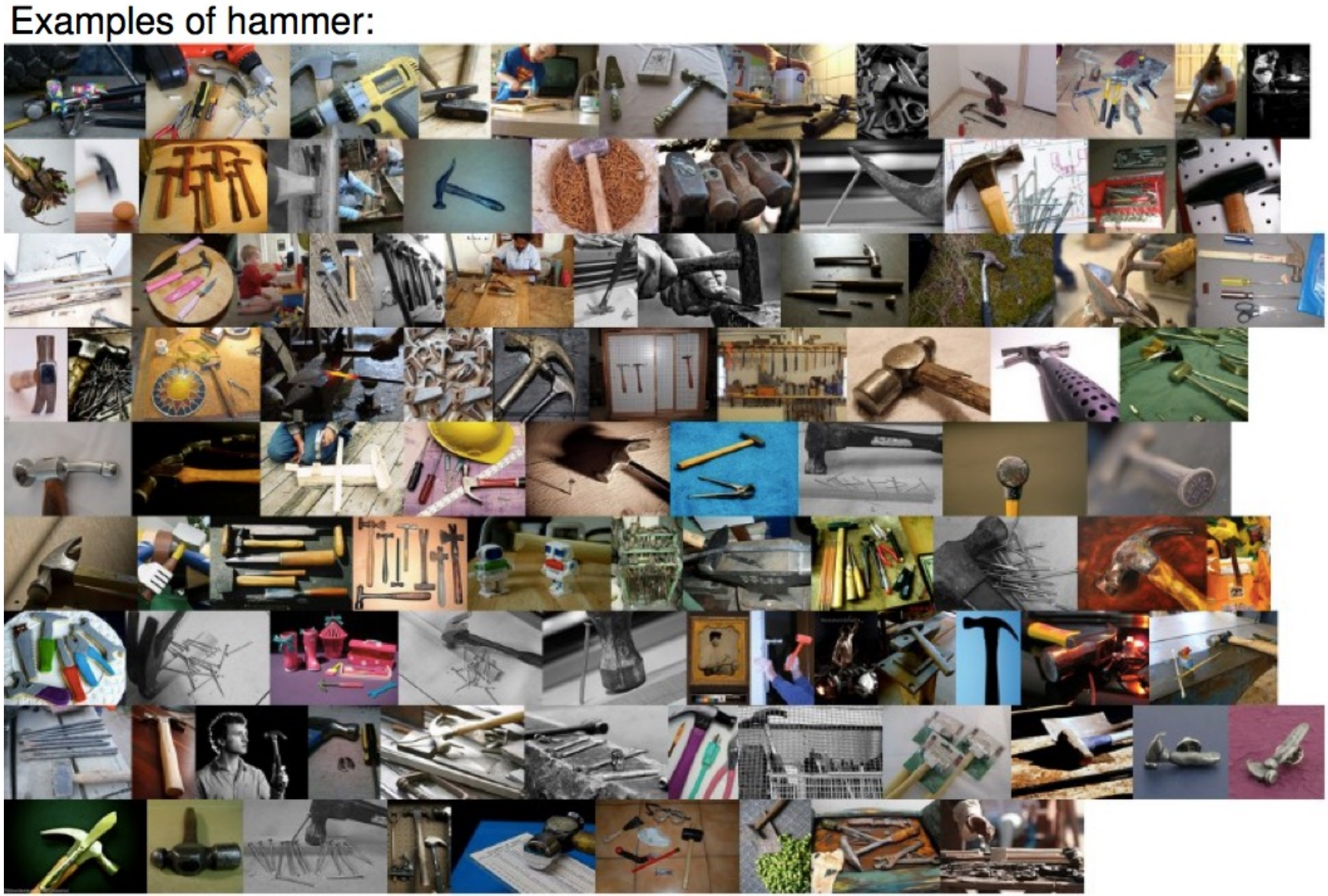


A CNN maps an image to class scores



A. Krizhevsky, I. Sutskever, and G. Hinton. ImageNet classification with deep convolutional neural networks. *NIPS*13

One category, enormous variation



Recognition spans thousands of classes



2012: a CNN cut the error by ten points

Team	Classification error
University of Tokyo	26.1%
Oxford University Computer Vision Group	26.9%
INRIA + XRCE (Xerox Research Centre Europe)	27.0%
University of Amsterdam	29.5%

And the winner:

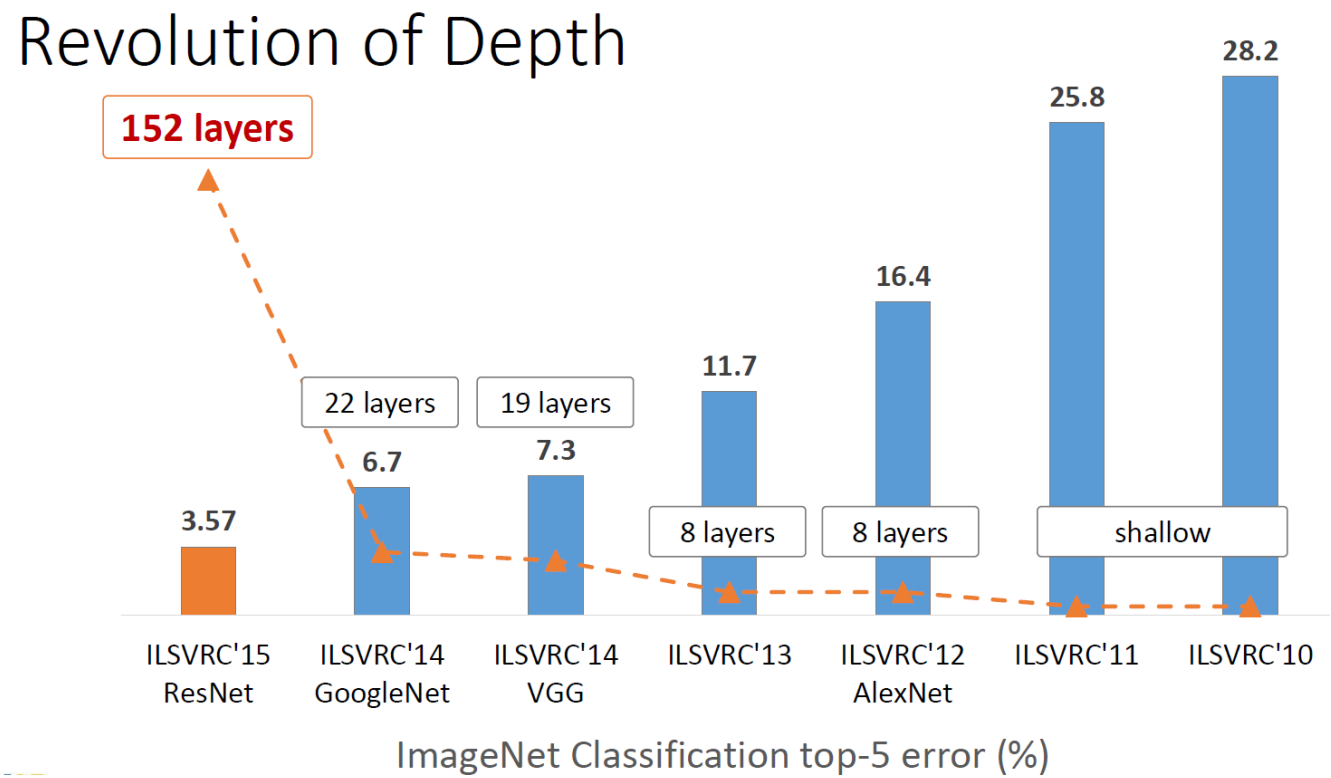
University of Toronto (Alex Krizhevsky)	**16.4%**

The deep convolutional network won by a wide margin.

ILSVRC 2012 results; Krizhevsky, Sutskever and Hinton (2012)

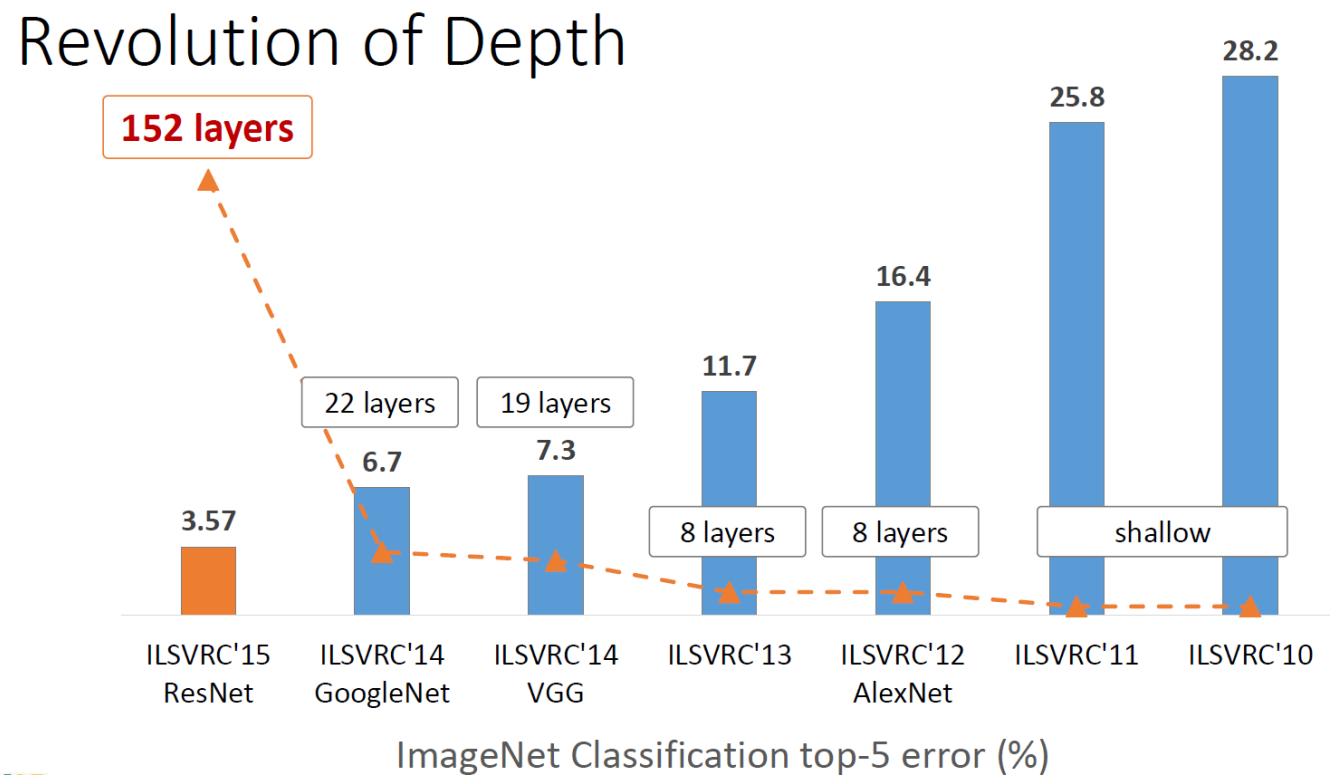
Why depth helps: context and richer features

- Larger receptive fields combine information from a wider area.
- Successive layers build more complex features from simpler ones.

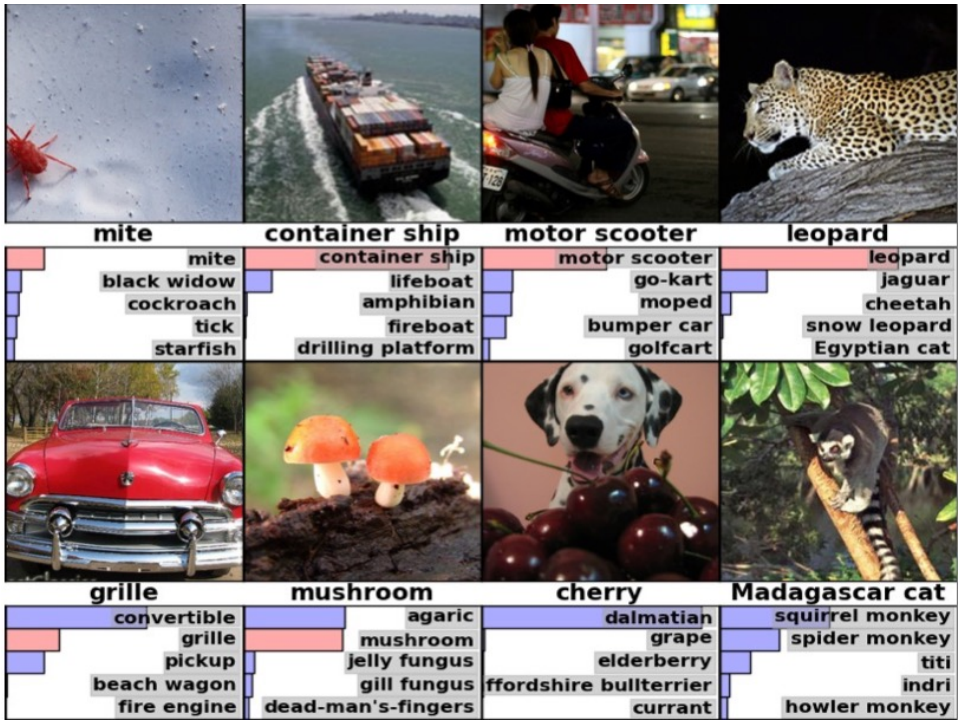


What made deeper networks trainable

- 2010–12: from 2 to 10 layers (ReLU, dropout)
- 2015: from 10 to 20 layers (batch normalisation)
- 2015 onwards: from 20 to 100/1000 layers (residual connections)



2015: CNNs beat the human estimate



Humans: 5.4%

A. Krizhevsky, I. Sutskever, and G. Hinton. ImageNet classification with deep convolutional neural networks. *NIPS*13 [16.4%] (best shallow competitor: 26%)

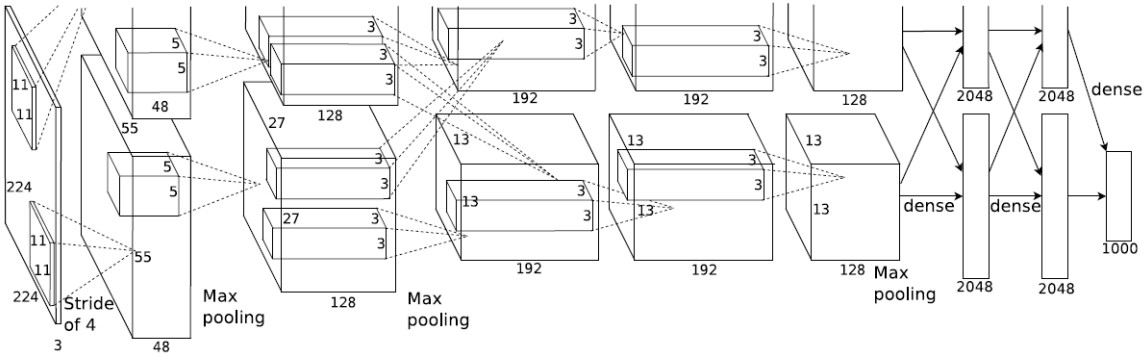
K. He, X. Zhang, S. Ren, J. Sun, Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification, ICCV 2015. [4.5%]

S. Ioffe, C. Szegedy, Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift, ICML 2015. [4.5%]

K. He, X. Zhang, S. Ren, J. Sun, Deep Residual Learning for Image Recognition, CVPR, 2016 [3.6%]

Five landmark CNN architectures

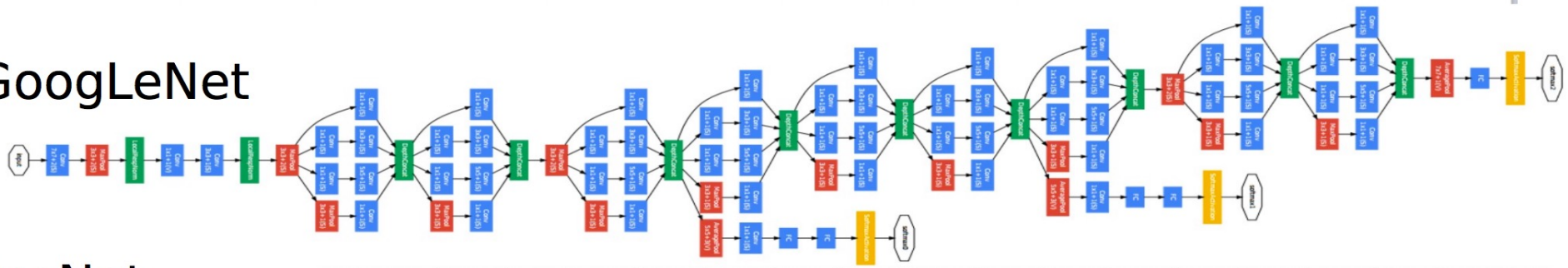
AlexNet



VGG

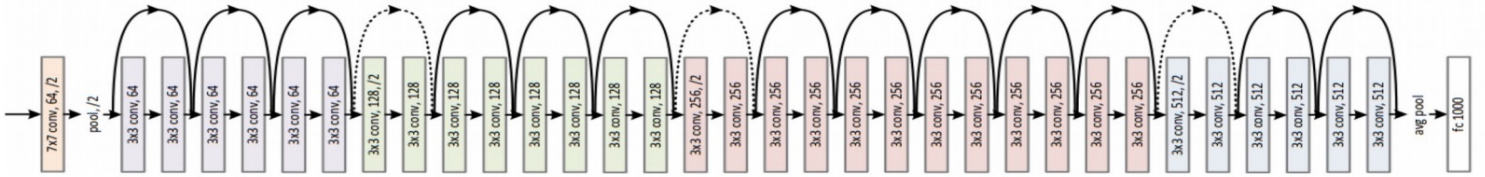


GoogLeNet



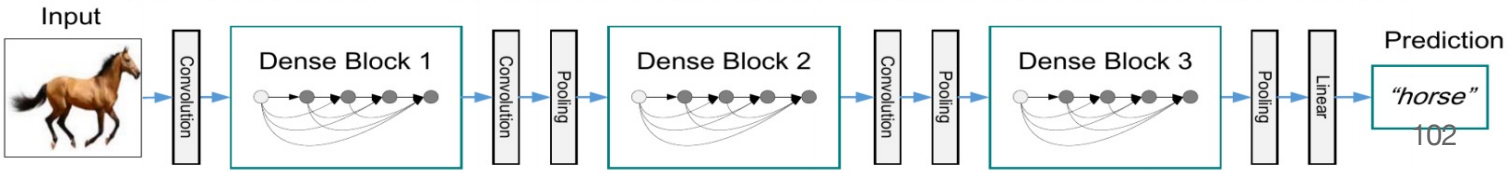
ResNet

[He et al. 2015]



DenseNet

[Huang et al 2017]

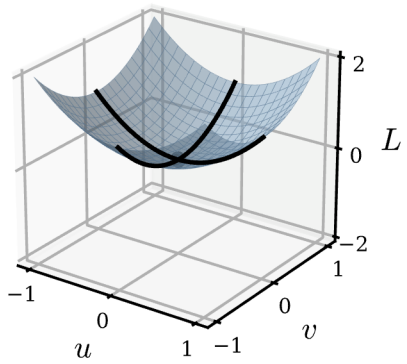


Hessian and local curvature

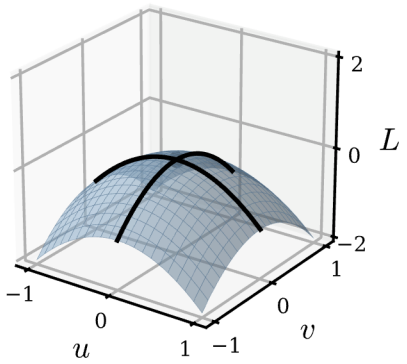
At a stationary point p, the gradient is zero. The Hessian gives the local quadratic shape.

$$H_{ij}(\mathbf{p}) = \frac{\partial^2 f}{\partial\theta_i\,\partial\theta_j}(\mathbf{p}) \qquad f(\mathbf{p}+\mathbf{d}) - f(\mathbf{p}) \;\approx\; \frac{1}{2}\mathbf{d}^T\mathbf{H}(\mathbf{p})\mathbf{d}$$

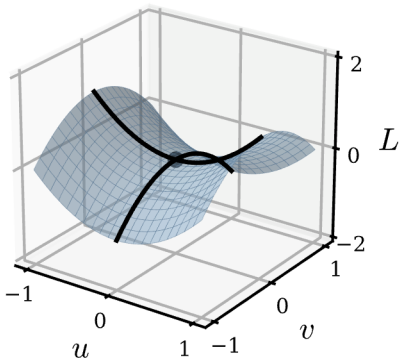
Minimum: $L = u^2 + v^2$
$\mathbf{H} = \mathrm{diag}(2, 2)$



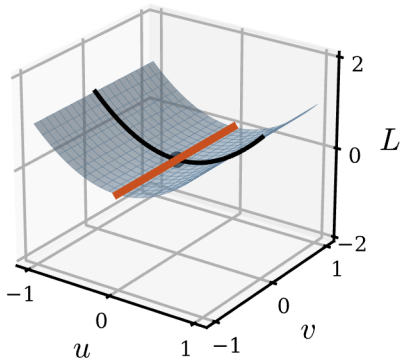
Maximum: $L = -u^2 - v^2$
$\mathbf{H} = \mathrm{diag}(-2, -2)$



Saddle: $L = u^2 - v^2$
$\mathbf{H} = \mathrm{diag}(2, -2)$



Flat valley: $L = u^2$
$\mathbf{H} = \mathrm{diag}(2, 0)$



$$\mathbf{H}\mathbf{v} = \lambda\mathbf{v}, \quad \|\mathbf{v}\| = 1 \quad \Longrightarrow \quad \left.\frac{d^2 f(\mathbf{p}+h\mathbf{v})}{dh^2}\right|_{h=0} = \lambda$$

Expanding the two-weight loss

$$L(a,b) = \frac{1}{2}(1-ab)^2, \qquad a>0,\ b>0$$

Differentiate with respect to both weights

$$\nabla L = (ab-1)\begin{bmatrix} b \\ a \end{bmatrix} \qquad \mathbf{H}(a,b) = \begin{bmatrix} b^2 & 2ab-1 \\ 2ab-1 & a^2 \end{bmatrix}$$

At a zero-loss point p = (a_0, b_0), with $a_0 b_0 = 1$, let d = (δa, δb).

$$L(\mathbf{p}+\mathbf{d}) = \frac{1}{2}\left(b_0\delta a + a_0\delta b + \delta a\,\delta b\right)^2$$

Keeping only second-order terms

$$L(\mathbf{p}+\mathbf{d}) \approx \frac{1}{2}(b_0\delta a + a_0\delta b)^2 = \frac{1}{2}\mathbf{d}^T\mathbf{H}(\mathbf{p})\mathbf{d}$$

The linear term vanishes because ∇L(p) = 0.

For (1 – ab)² without the ½, double every derivative and curvature shown here.

Equivalent fits, different curvature

Both points have ab = 1, zero loss and zero gradient.

Curvature at (1, 1): (1, 1)

$$\mathbf{H}(1,1) = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

$$\lambda_1 = 0, \qquad \lambda_2 = 2$$

$$L \approx \frac{1}{2}\delta a^2 + \delta a\,\delta b + \frac{1}{2}\delta b^2$$

Curvature couples a and b equally

Curvature at (10, 0.1): (0.1, 10)

$$\mathbf{H}(0.1,10) = \begin{bmatrix} 100 & 1 \\ 1 & 0.01 \end{bmatrix}$$

$$\lambda_1 = 0, \qquad \lambda_2 = 100.01$$

$$L \approx 50\delta a^2 + \delta a\,\delta b + 0.005\delta b^2$$

Largest curvature is almost along a

$$\frac{\lambda_{\max}(0.1,10)}{\lambda_{\max}(1,1)} = \frac{100.01}{2} = 50.005$$

The steepest curvature is about 50× higher for the same fit.

Appendix: softmax and cross-entropy

A multiclass forward and backward pass

Linear scores, softmax probabilities and cross-entropy loss

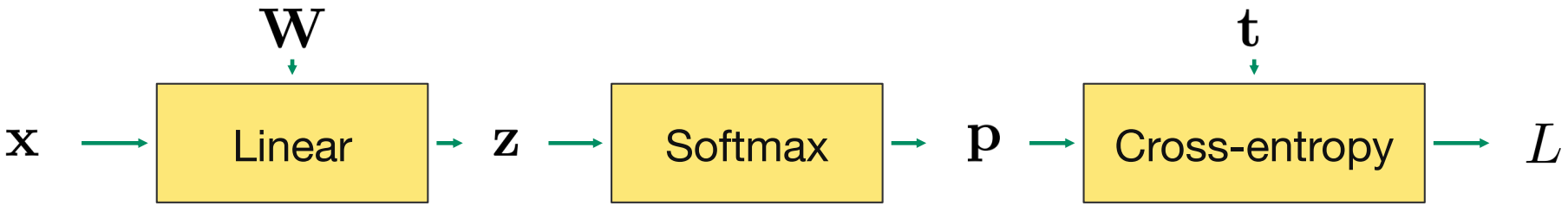
The softmax Jacobian and the gradient with respect to scores

Weight gradients and gradients sent to the previous layer

The following slides retain the worked derivation.

Multiclass classification: forward pass

One example, C classes, and a one-hot target t. Biases are omitted.



Linear $\mathbf{z} = \mathbf{W}\mathbf{x}, \qquad \mathbf{W} \in \mathbb{R}^{C \times D}$

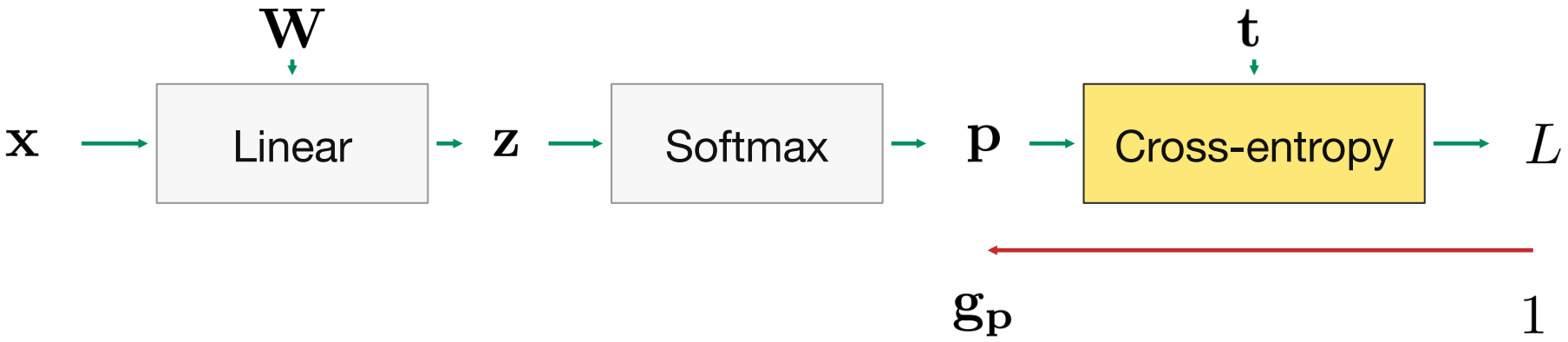
Softmax $\mathbf{p} = \text{softmax}(\mathbf{z}), \quad p_c = \dfrac{e^{z_c}}{\sum_{k=1}^{C} e^{z_k}}$

Loss $L = -\sum_{c=1}^{C} t_c \log p_c = -\log p_{c^*}$

c* is the correct class. Cache x and the softmax probabilities.

Backward through cross-entropy

Start with the scalar seed ∂L/∂L = 1.



$$L = -\sum_{c=1}^{C} t_c \log p_c$$

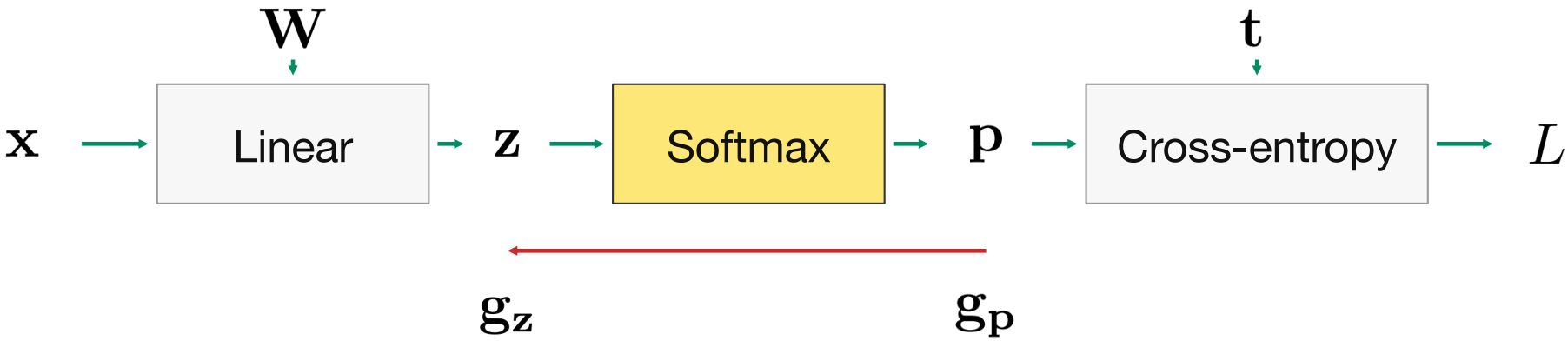
$$\frac{\partial L}{\partial p_c} = -\frac{t_c}{p_c}, \quad [\mathbf{g_p}]_c = -\frac{t_c}{p_c}$$

Example: probabilities (0.6, 0.3, 0.1), correct class 2.

At the probabilities instead: $\mathbf{g_p} = (0, -10/3, 0)^T$.

The softmax Jacobian

Every softmax probability depends on every score.



$$\frac{\partial p_c}{\partial z_k} = p_c(\delta_{ck} - p_k)$$

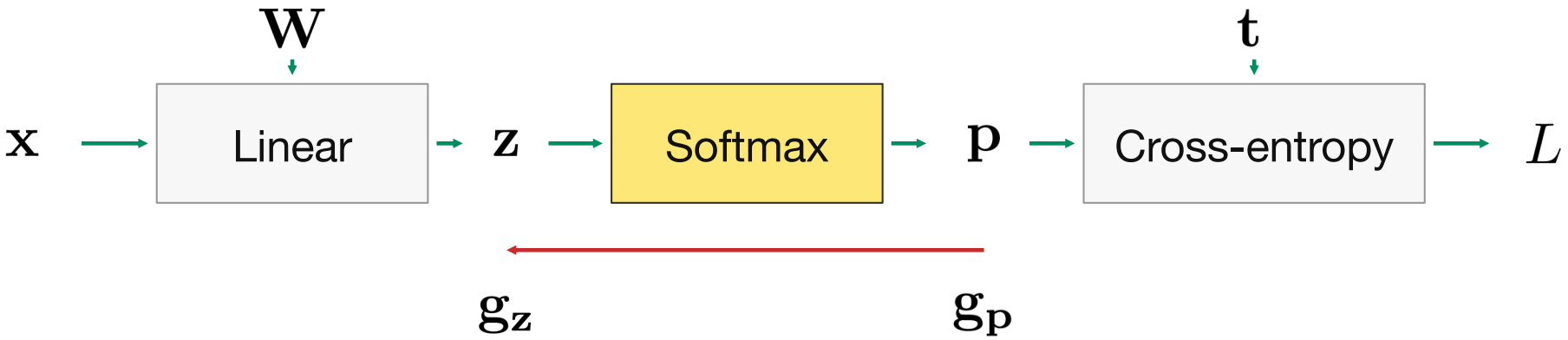
The indicator equals 1 when c = k, and 0 otherwise.

$$\mathbf{J}_{\text{softmax}} = \text{diag}(\mathbf{p}) - \mathbf{p}\mathbf{p}^T \in \mathbb{R}^{C \times C}$$

$$\mathbf{g_z} = \mathbf{J}_{\text{softmax}}^T \mathbf{g_p}$$

Backward through softmax

The chain rule converts probability sensitivity into score sensitivity.



$$\frac{\partial L}{\partial z_k} = \sum_{c=1}^{C} \left(-\frac{t_c}{p_c} \right) p_c(\delta_{ck} - p_k)$$

$$= -t_k + p_k \sum_{c=1}^{C} t_c = p_k - t_k$$

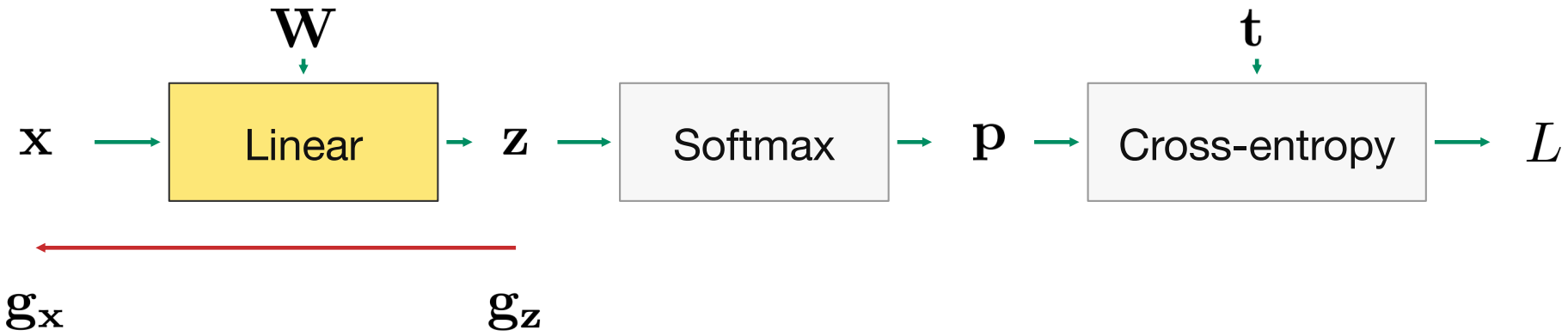
$$\mathbf{g_z} = \mathbf{p} - \mathbf{t}$$

The target entries sum to one. The result differentiates the scores z.

$$\mathbf{p} = (0.6, 0.3, 0.1)^T, \quad \mathbf{t} = (0, 1, 0)^T \quad \Longrightarrow \quad \mathbf{g_z} = (0.6, -0.7, 0.1)^T$$

Backward through the linear layer

The incoming column gradient is the prediction minus the target.



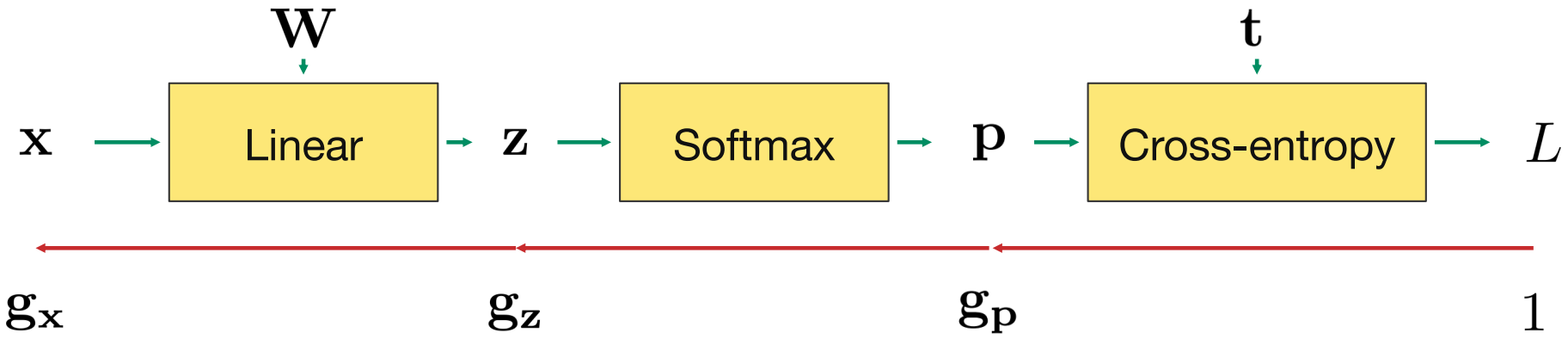
$$\nabla_{\mathbf{W}} L = \underbrace{\mathbf{g_z}}_{C \times 1} \underbrace{\mathbf{x}^T}_{1 \times D} \in \mathbb{R}^{C \times D}$$

$$\mathbf{g_x} = \underbrace{\mathbf{W}^T}_{D \times C} \underbrace{\mathbf{g_z}}_{C \times 1} \in \mathbb{R}^{D \times 1}$$

Store the weight gradient. Send the input gradient to the previous layer.

The complete multiclass backward pass

The same forward values supply all the gradients.



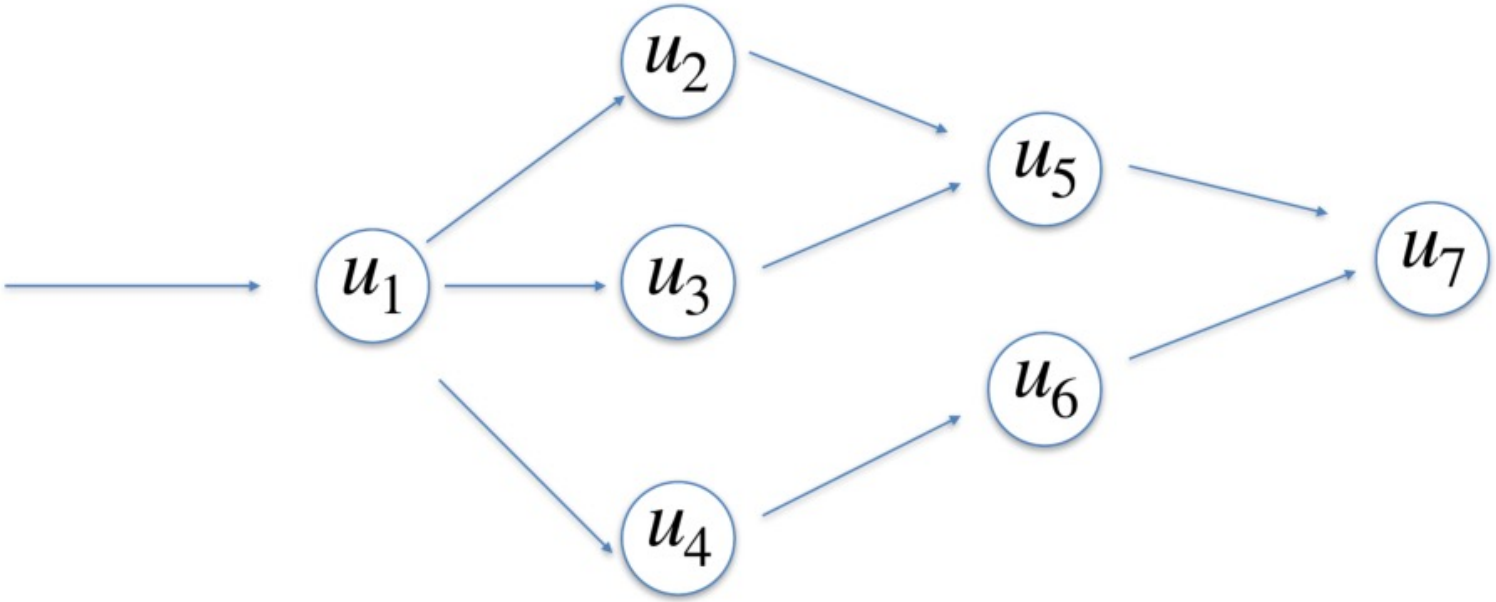
$$\mathbf{g}_\mathbf{z} = \mathbf{p} - \mathbf{t}$$

$$\nabla_\mathbf{W} L = (\mathbf{p} - \mathbf{t})\mathbf{x}^T$$

$$\mathbf{g}_\mathbf{x} = \mathbf{W}^T(\mathbf{p} - \mathbf{t})$$

For a mean data loss, average the per-example weight gradients.

Appendix: computation graphs

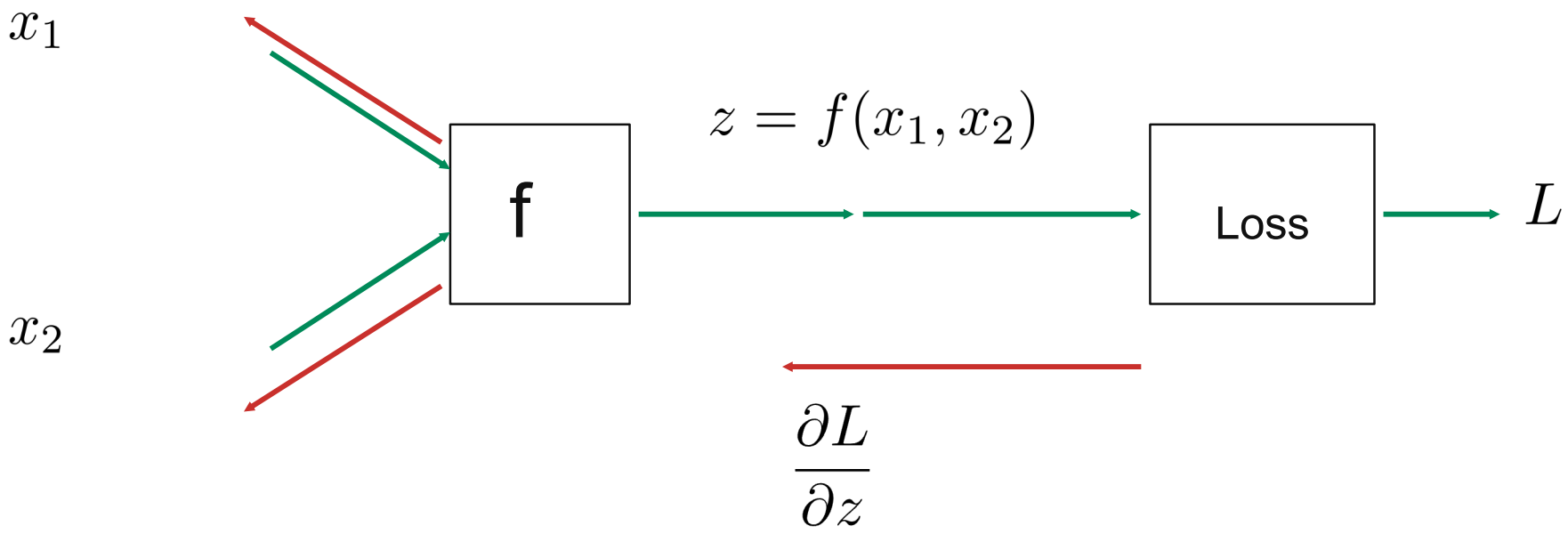


Reading: Computation graphs

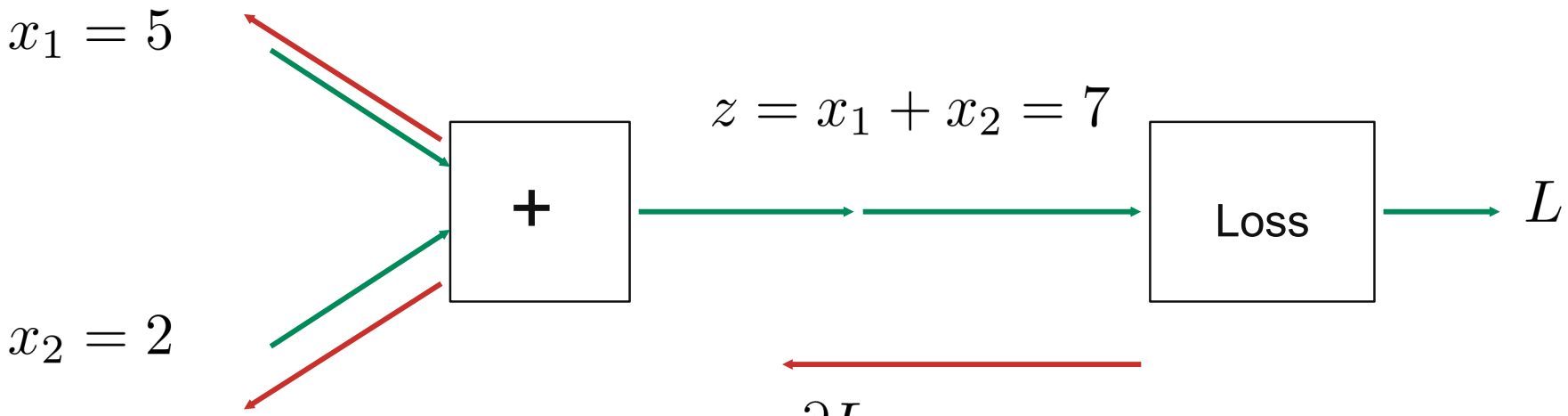
K. Murphy (2022), §13.3.4: Computation graphs
COMP0169 lecture slides, pp. 24–32

Backpropagation through one operation

x_1 and x_2 are scalar inputs.



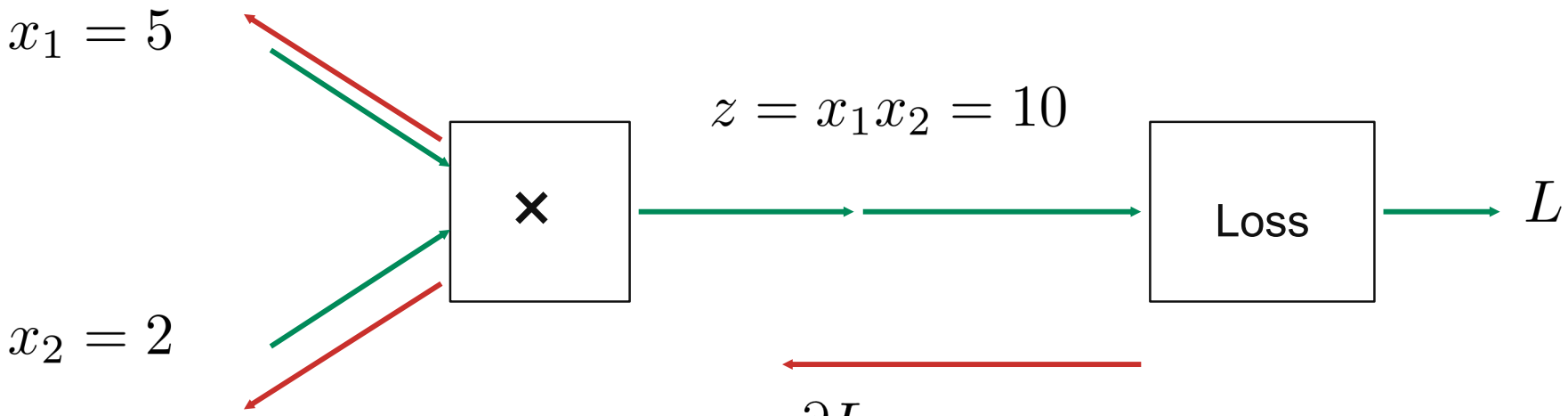
Backpropagation: addition



$$\frac{\partial z}{\partial x_1} = 1, \qquad \frac{\partial z}{\partial x_2} = 1$$

$$\frac{\partial L}{\partial x_1} = \frac{\partial L}{\partial z} \cdot 1 = 15 \qquad \frac{\partial L}{\partial x_2} = \frac{\partial L}{\partial z} \cdot 1 = 15$$

Backpropagation: multiplication

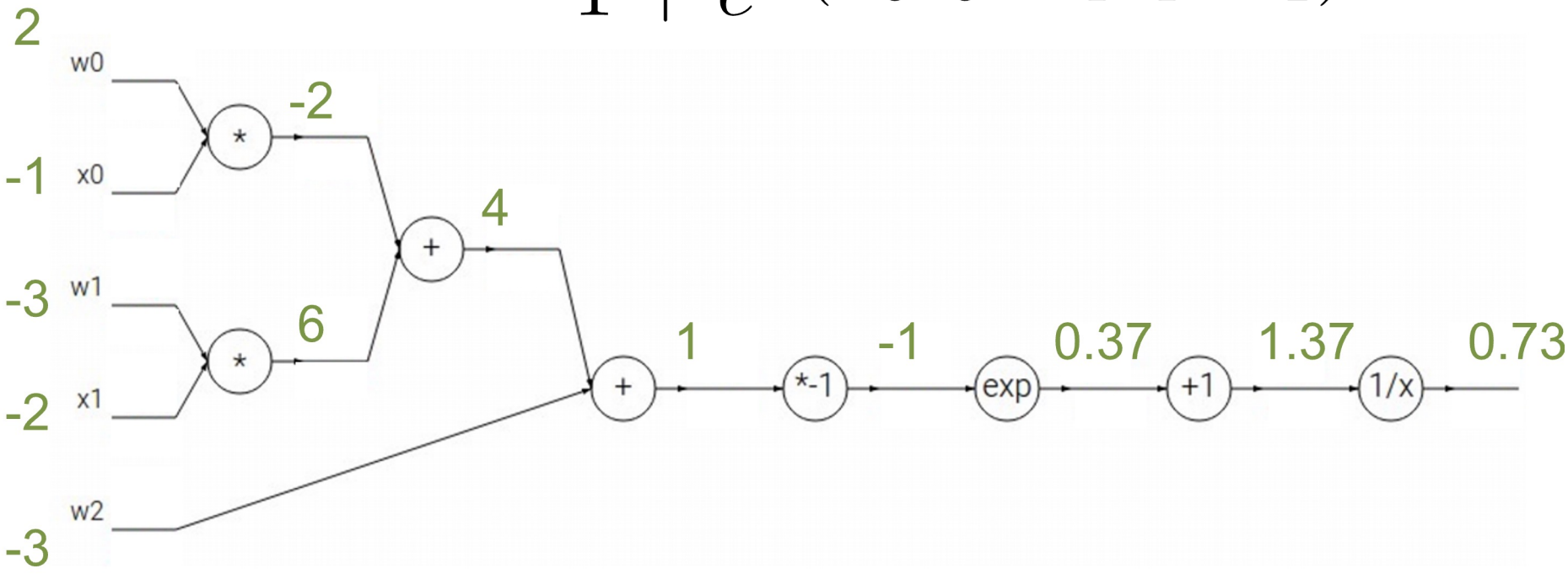


$$\frac{\partial z}{\partial x_1} = x_2 = 2, \qquad \frac{\partial z}{\partial x_2} = x_1 = 5$$

$$\frac{\partial L}{\partial x_1} = \frac{\partial L}{\partial z} x_2 = 7 \cdot 2 = 14 \qquad \frac{\partial L}{\partial x_2} = \frac{\partial L}{\partial z} x_1 = 7 \cdot 5 = 35$$

Backpropagation through a sigmoid neuron

$$f(\mathbf{x}, \mathbf{w}) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$$



Source notation: w_2 is the bias b; inputs are indexed 0 and 1.

Backpropagation on a small computation

$$f = (r + s)t, \quad r = 3,\ s = 5,\ t = -2$$

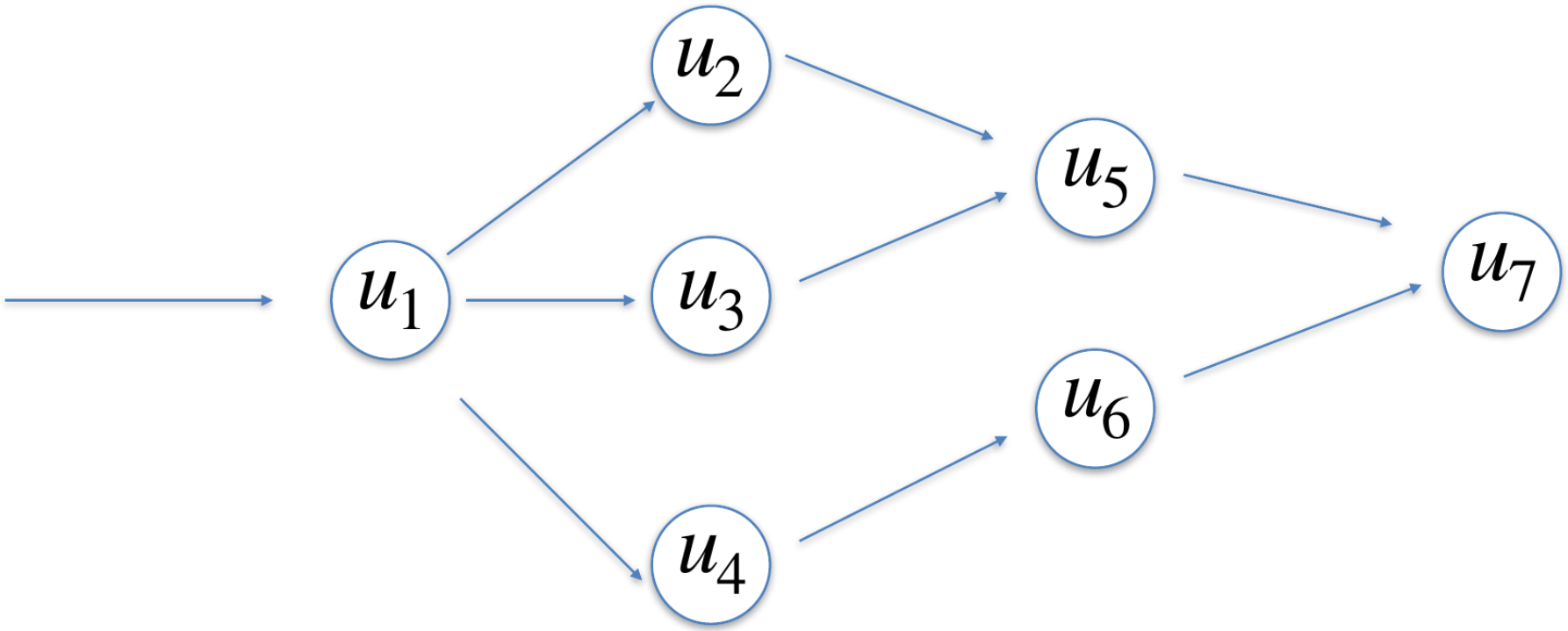
$$q = r + s = 8, \qquad f = qt = -16$$

$$\frac{\partial f}{\partial q} = t = -2, \quad \frac{\partial f}{\partial t} = q = 8$$

$$\frac{\partial f}{\partial r} = \frac{\partial f}{\partial q} \cdot 1 = -2, \quad \frac{\partial f}{\partial s} = -2$$

Multiplication uses the other input. Addition sends the gradient to both inputs.

Ordering a computation graph



A forward pass visits each operation after its inputs.
Several topological orders can be valid.

Backpropagation on a graph

```
class ComputationalGraph(object):
    #...
    def forward(inputs):
        # 1. [pass inputs to input gates...]
        # 2. forward the computational graph:
        for gate in self.graph.nodes_topologically_sorted():
            gate.forward()
        return loss # the final gate in the graph outputs the loss
    def backward():
        for gate in reversed(self.graph.nodes_topologically_sorted()):
            gate.backward() # little piece of backprop (chain rule applied)
        return inputs_gradients
```

The backward pass reverses the forward order.
Each operation contributes gradients to its inputs.